Modul Praktikum

Algoritma Pemrograman 1

S1 Informatika

Published by school of computing

Our official instagram

@informaticslab_telu

Telkom University

Fakultas Informatika
School of Computing
Telkom University

informatics lab

LEMBAR PENGESAHAN

Saya yang bertanda tangan di bawah ini:

Nama	: Lidya Ningsih, S.T., M.Kom.
NIP	: 23950047
Koordinator Mata Kuliah	: Algoritma dan Pemrograman 1
Prodi	: S1 Informatika

Menerangkan dengan sesungguhnya bahwa modul ini digunakan untuk pelaksanaan praktikum di Semester Ganjil Tahun Ajaran 2026/2027 di Laboratorium Informatika, Fakultas Informatika, Universitas Telkom.

Bandung, 24 Agustus 2026

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Mengesahkan,	Mengetahui,
Koordinator Mata Kuliah	Kaprodi S1 Informatika
Algoritma Pemrograman 1	
Lidya Ningsih, S.T., M.Kom.	Mahmud Dwi Sulistiyo, S.T., M.T., Ph.D
NIP. 23950047	NIP. 13880017

DAFTAR ISI

LEMBAR PENGESAHAN ..... **ii**
DAFTAR ISI ..... **iii**
DAFTAR GAMBAR ..... **vi**
DAFTAR TABEL ..... **vii**
PRAKATA ..... **viii**
MODUL 1. RUNNING MODUL **1**
1.1 Pengantar Praktikum 1
1.2 Peraturan Praktikum Laboratorium Informatika 2
1.3 Penyampaian Keluhan Praktikum IFLAB 4
1.4 Aturan Penulisan dan Submisi Tugas 5
1.5 Instalasi Program Penunjang Praktikum 8
1.6 Struktur Program menggunakan Go 9
MODUL 2. I/O, TIPE DATA & VARIABEL **12**
2.1 Input dan Output (I/O) 12
2.2 Data, Variabel, dan Instruksi Dasar 13
2.3 Konstanta Simbolik 17
2.4 Tipe Data Character 17
2.5 Contoh Soal Modul 2 18
2.6 Soal Latihan Modul 2 20
MODUL 3. I/O, TIPE DATA & VARIABEL (LATIHAN 1) **22**
3.1 Integer Division dan Modulo 22
3.2 Casting atau Konversi Tipe Data 23
3.3 Contoh Soal Modul 3 23
3.4 Soal Latihan Modul 3 26
MODUL 4. I/O, TIPE DATA & VARIABEL (LATIHAN 2) **28**
4.1 Contoh Soal Modul 4 28
4.2 Soal Latihan Modul 4 30
MODUL 5 & 6. FOR-LOOP **32**
5.1 Paradigma Perulangan 32
5.2 Karakteristik For-Loop (Perulangan berdasarkan iterasi) 32
5.3 Implementasi menggunakan Go 33

Fakultas Informatika
School of Computing
Telkom University
informatics lab

5.4 Contoh Soal Modul 5 33
5.5 Soal Latihan Modul 5 36
MODUL 7. ASESMEN PRAKTIKUM 1 39
7.1 Asesmen Praktikum 1 39
7.2 Contoh Soal Asesmen Praktikum 1 39
MODUL 9. IF-THEN 43
9.1 Paradigma Percabangan 43
9.2 Karakterisik If-Then 43
9.3 Implementasi menggunakan Go 43
9.4 Contoh Soal Modul 9 44
9.5 Soal Latihan Modul 9 47
MODUL 10. ELSE-IF 49
10.1 Paradigma Percabangan 49
10.2 Karakterisik Else-If 49
10.3 Implementasi menggunakan Go 50
10.4 Contoh Soal Modul 10 51
10.5 Soal Latihan Modul 10 54
MODUL 11. SWITCH-CASE 57
11.1 Karakterisik Switch-Case 57
11.2 Implementasi menggunakan Go 57
11.3 Contoh Soal Modul 11 59
11.4 Soal Latihan Modul 11 63
MODUL 12. WHILE-LOOP 66
12.1 Paradigma Perulangan 66
12.2 Karakterisik While-Loop 66
12.3 Implementasi menggunakan Go 67
12.4 Contoh Soal Modul 12 68
12.5 Soal Latihan Modul 12 70
MODUL 13. REPEAT-UNTIL 73
13.1 Paradigma Perulangan 73
13.2 Karakteristik Repeat-Until 73
13.3 Implementasi menggunakan Go 74
13.4 Contoh Soal Modul 13 75

Fakultas Informatika
School of Computing
Telkom University
informatics lab

13.5 Soal Latihan Modul 13 79
MODUL 14. KOMPOSISI 80
14.1 Pengantar Komposisi 80
14.2 Contoh Soal Modul 14 80
14.3 Soal Latihan Modul 14 83
MODUL 15. ASESMEN PRAKTIKUM 2 87
15.1 Asesmen Praktikum 2 87
15.2 Contoh Soal Asesmen Praktikum 2 87
MODUL 16. SKEMA PEMROSESAN SEKUENSIAL (PENGAYAAN) 92
16.1 Pengantar Skema Pemrosesan Sekuensial 92
16.2 Pembacaan Data Tanpa Marker pada Akhir Rangkaian Data 92
16.3 Pembacaan Data dengan Marker pada Akhir Rangkaian Data 93
16.4 Kemungkinan Rangkaian Data Kosong, Hanya Ada Marker 93
16.5 Elemen Pertama Perlu Diproses Tersendiri/Kasus Khusus 94
16.6 Soal Latihan Modul 16 96
DAFTAR PUSTAKA 100
LAMPIRAN 101
A. Paket-Paket Penting 101
B. Perintah Go 113
C. Instalasi Go Perkakas 114
D. Instalasi Paket Pihak ke-3 (Contoh paket GUI) 115
E. Contoh Program 115

Fakultas Informatika
School of Computing
Telkom University
informatics lab

DAFTAR GAMBAR

Gamber 16-1 Ilustrasi denah daerah yang digunakan untuk mengukur curah hujan 96

Fakultas Informatika
School of Computing
Telkom University
informatics lab

DAFTAR TABEL

Tabel 1-1 Rencana agenda praktikum selama satu semester 1
Tabel 1-2 Bentuk Percabangan 7
Tabel 2-1 instruksi_input(variabel1, variabel2, variabel3, ...) 12
Tabel 2-2 instruksi_output(variabel1/data1, variabel2/data2, ...) 12
Tabel 2-3 Pemetaan Tipe Data Dasar Algoritmik ke Tipe Data Bahasa Go 13
Tabel 2-4 Daftar Operator dalam Bahasa Go dan Tipe Data Terkait 14
Tabel 2-5 Contoh Operasi Tipe Data dan Hasil Evaluasinya dalam Bahasa Go 15
Tabel 2-6 Contoh Penyesuaian (Casting) Tipe Data dalam Bahasa Go 15
Tabel 2-7 Metode Konversi Tipe Data (Type Casting dan Format String) dalam Bahasa Go 15
Tabel 2-8 Notasi Deklarasi Variabel Algoritmik dan Penulisannya dalam Bahasa Go 16
Tabel 2-9 Notasi Instruksi Dasar Algoritmik dan Penulisannya dalam Bahasa Go 17
Tabel 2-10 Representasi Tipe Data Karakter (Char) dalam Bahasa Go 17
Tabel 2-11 Contoh potongan tabel ASCII untuk beberapa bilangan integer 17
Tabel 3-1 Simulasi Ekstraksi Digit Menggunakan Operasi div dan mod 23
Tabel 5-1 Perbandingan Penulisan Kode Program Tanpa dan Menggunakan Perulangan 32
Tabel 5-2 Notasi Perulangan For-Loop pada Pseudocode dan Bahasa Go 32
Tabel 9-1 Notasi Percabangan Tunggal (If-Then) pada Pseudocode dan Bahasa Go 43
Tabel 10-1 Notasi Percabangan If-Else pada Pseudocode dan Bahasa Go 49
Tabel 10-2 Notasi Percabangan Bertingkat (Nested If-Else) pada Pseudocode dan Bahasa Go 49
Tabel 10-3 Notasi Percabangan Jamak (Else-If) pada Pseudocode dan Bahasa Go 50
Tabel 11-1 Notasi Pemilihan Switch-Case pada Pseudocode dan Bahasa Go 57
Tabel 12-1 Notasi Perulangan While-Loop pada Pseudocode dan Bahasa Go 66
Tabel 12-2 Contoh Implementasi While-Loop untuk Menghitung Akar Bilangan dalam Algoritma dan Bahasa Go 67
Tabel 13-1 Notasi Perulangan Repeat-Until pada Pseudocode dan Bahasa Go 73
Tabel 13-2 Contoh Implementasi Repeat-Until untuk Mencetak Deret Fibonacci dalam Algoritma dan Bahasa Go 74
Tabel 16-1 Pola Pembacaan Data Tanpa Marker pada Notasi Algoritma dan Bahasa Go 92
Tabel 16-2 Contoh Penerapan Pola Pembacaan Data untuk Pencarian Nilai Maksimum dalam Notasi Algoritma dan Bahasa Go 92
Tabel 16-3 Pola Pembacaan Data Menggunakan Marker pada Notasi Algoritma dan Bahasa Go 93
Tabel 16-4 Contoh Penerapan Pola Pembacaan Data Menggunakan Marker untuk Pencarian Nilai Maksimum dalam Algoritma dan Bahasa Go 93
Tabel 16-5 Pola Pembacaan Data Menggunakan Marker dengan Penanganan Kasus Data Kosong pada Notasi Algoritma dan Bahasa Go 94
Tabel 16-6 Contoh Penerapan Pola Pembacaan Data Berbasis Marker dengan Penanganan Data Kosong untuk Pencarian Nilai Maksimum 94
Tabel 16-7 Pola Pembacaan Data Berbasis Marker dengan Pemrosesan Data Pertama dan Penanganan Kasus Kosong 95
Tabel 16-8 Contoh Penerapan Pola Pembacaan Berbasis Marker dengan Inisialisasi Data Pertama untuk Pencarian Nilai Maksimum 95

Fakultas Informatika
School of Computing
Telkom University
Informatics Lab

PRAKATA

Buku modul praktikum **Algoritma dan Pemrograman 1 (kode mata kuliah CAK1BAB3)** ini merupakan panduan mahasiswa dalam pelaksanaan praktikum dalam mata kuliah **Algoritma dan Pemrograman 1** di Prodi S1 Infromatika, Fakultas Informatika, Universitas Telkom. Buku modul ini berisi materi, contoh soal, dan kumpulan soal-soal latihan yang bisa digunakan mahasiswa untuk belajar sebelum pelaksanaan praktikum dimulai. Bahasa pemrograman yang digunakan adalah **Go Language (GoLang)**.

Buku modul praktikum ini terdiri dari materi terkait tipe data, struktur kontrol percabangan dan perulangan, proses sekuensial. Materi tersebut diturunkan menjadi 16 modul praktikum yang disusun sedemikian rupa sehingga diharapkan mampu membantu mahasiswa dalam mempelajari *problem solving* sederhana menggunakan bahasa pemrograman GoLang. Setiap pekan praktikum, mahasiswa akan melaksanakan kegiatan praktikum sesuai dengan urutan modul yang terdapat pada buku modul praktikum ini, mulai dari pengerjaan tugas, latihan dan juga asesmen. Walaupun buku modul ini berisi materi yang mengajarkan algoritma dengan pendekatan praktek pemrograman dengan GoLang, tetapi buku modul ini tidak bisa dijadikan sebagai referensi utama untuk pembelajaran pemrograman yang lebih luas. Hal ini dikarenakan fokus kedalaman materi dan tingkat kesulitan soal yang diberikan menyesuaikan pada *Course Learning Outcome* (CLO) pada rencana pembelajaran semester yang telah ditetapkan pada mata kuliah **Algoritma dan Pemrograman 1** di Prodi S1 Informatika, Fakultas Informatika, Universitas Telkom.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Buku modul ini diharapkan bisa memberikan manfaat kepada mahasiswa atau praktikan dalam menguasai materi yang diberikan pada mata kuliah Algoritma Pemrograman.

Bandung, 24 Agustus 2026

Tim Penulis

MODUL 1. RUNNING MODUL

1.1 Pengantar Praktikum

Praktikum Algoritma dan Pemrograman 1 merupakan salah satu aktivitas dalam rangkaian perkuliahan pada mata kuliah **Algoritma dan Pemrograman 1** (kode mata kuliah **CAK1BAB3**). Oleh karena itu susunan materi pada modul ini dibuat sedemikian rupa sehingga inline dengan materi yang diajarkan di kelas teori. Penilaian pada praktikum Algoritma Pemrograman ini menjadi salah satu komponen penilaian pada CLO-03-4 *Mampu menerapkan prinsip-prinsip algoritma dalam menyelesaikan masalah di bidang informatika*, dan Sub-CLO-03-4-3 *Mahasiswa mampu mengimplementasikan proses sekuensial dasar ke dalam suatu bahasa pemrograman*.

Selain itu, terdapat tiga jenis penugasan pada kegiatan praktikum ini, yaitu:

1) **Tugas Pendahuluan (TP)**, merupakan tugas yang diberikan sebelum praktikum dimulai, estimasi pengerjaan tugas ini adalah 60 menit di luar perkuliahan/praktikum. Tugas ini biasanya diumumkan di akhir pekan dan dikumpulkan pada hari Senin pagi.
2) **Tugas Jurnal**, merupakan tugas utama praktikum dengan estimasi pengerjaan selama 100 menit. Pada saat pengerjaan jurnal ini, asisten praktikum akan membimbing praktikan dalam mengerjakan tugas dan asisten juga akan memberikan tutorial pada beberapa contoh soal. Praktikan diperbolehkan bertanya terkait materi, ide solusi, dan notasi pemrograman.
3) **Asesmen Praktikum**, merupakan sesi yang praktikan mengerjakan soal secara mandiri tanpa bantuan dari asisten praktikum. Pertanyaan hanya boleh terkait hal teknis, kejelasan soal dan notasi pemrograman. Estimasi pengerjaan selama 100 menit.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Adapun rencana agenda praktikum selama satu semester ini dapat dilihat pada tabel[1] berikut:

Tabel 1-1 Rencana agenda praktikum selama satu semester

No	Modul Praktikum	TP	Jurnal	Asesmen
1	Running Modul	-	-	-
2	I/O, Tipe Data &Variabel	√	√	-
3	I/O, Tipe Data &Variabel (Latihan 1)	√	√	-
4	I/O, Tipe Data &Variabel (Latihan 2)	√	√	-
5	Struktur Kontrol : For-Loop 1	√	√	-
6	Struktur Kontrol : For-Loop 2	√	√	-
7	Asesmen Praktikum 1	√	-	√
8	Asistensi Praktikum	-	-	-
9	Struktur Kontrol : If-Then	√	√	-
10	Struktur Kontrol : Else-If	√	√	-
11	Struktur Kontrol : Switch-Case	√	√	-

[1] susunan materi dan penugasan bisa berubah sesuai dengan kondisi pelaksanaan praktikum

12	Struktur Kontrol : While-Loop	√	√	-
13	Struktur Kontrol : Repeat-Until	√	√	-
14	Struktur Kontrol : Komposisi	√	√	-
15	Asesmen Praktikum 2	√	-	√
16	Praktikum Susulan	-	-	-

1.2 Peraturan Praktikum Laboratorium Informatika

1) Praktikum diampu oleh dosen kelas dan dibantu oleh asisten laboratorium dan asisten praktikum.
2) Praktikum dilaksanakan di Gedung Telkom University Landmark Tower (TULT) Lantai 6 dan Lantai 7 sesuai jadwal yang ditentukan.
3) Praktikan wajib membawa KTM (Kartu Tanda Mahasiswa).
4) Durasi kegiatan praktikum S-1 = 2 jam (100 menit).
5) Jumlah pertemuan praktikum:
    - 16 kali pertemuan
6) Praktikan wajib hadir minimal 75% dari seluruh pertemuan praktikum di lab.
7) Praktikan yang datang terlambat :
    a. <= 20 menit : diperbolehkan mengikuti praktikum tanpa tambahan waktu.
    b. > 30 menit : tidak diperbolehkan mengikuti praktikum.
8) Saat praktikum berlangsung, asisten praktikum dan praktikan:
    a. Wajib menggunakan pakaian sesuai aturan yang berlaku.
    b. Wajib mematikan atau mengkondisikan semua alat komunikasi.
    c. Wajib menjaga seluruh fasilitas yang berada di ruangan praktikum.
    d. Dilarang membuka aplikasi yang tidak berhubungan dengan praktikum yang berlangsung.
    e. Dilarang mengubah pengaturan *software* maupun *hardware* komputer tanpa izin.
    f. Dilarang membawa makanan maupun minuman di ruang praktikum.
    g. Dilarang memberikan jawaban ke praktikan lain.
    h. Dilarang menyebarkan soal praktikum.
    i. Dilarang membuang sampah di ruangan praktikum.
    j. Wajib meletakkan alas kaki dengan rapi pada tempat yang telah disediakan.
9) Setiap praktikan dapat mengikuti praktikum susulan maksimal dua modul untuk satu mata kuliah praktikum.
    a. Praktikan yang dapat mengikuti praktikum susulan hanyalah praktikan yang memenuhi syarat sesuai ketentuan institusi, yaitu: sakit (dibuktikan dengan surat

Fakultas Informatika
School of Computing
Telkom University
informatics lab

keterangan medis), tugas dari institusi (dibuktikan dengan surat dinas atau dispensasi dari institusi), atau mendapat musibah atau kedukaan (menunjukkan surat keterangan dari orangtua/wali mahasiswa.)

b. Persyaratan untuk praktikum susulan diserahkan sesegera mungkin kepada asisten laboratorium untuk keperluan administrasi.

c. Praktikan yang diizinkan menjadi peserta praktikum susulan ditetapkan oleh lab informatika dan tidak dapat diganggu gugat.

10) Sanksi pelanggaran pada saat pelaksanaan praktikum

a. Ketidakhadiran pada kelas praktikum

- **Nilai jurnal dan/atau tes awal & akhir = 0**

b. Meminta, mendapatkan, dan menyebarluaskan soal dan atau kunci jawaban praktikum:

- **Penyebar soal dan kunci jawaban: Pengajuan sanksi kepada Komisi Disiplin Fakultas**
- **Penerima soal dan kunci jawaban: Nilai '0' pada (seluruh asesmen) praktikum**

c. Lupa menghapus file praktikum

- **Pengurangan nilai jurnal 20%**

Fakultas Informatika School of Computing Telkom University
informatics lab

d. Memicu kegaduhan, sehingga membuat situasi tidak kondusif (jalan-jalan, mengganggu teman, mengobrol, dll), asisten praktikum diwajibkan menegur sebanyak 3x

- **Pengurangan nilai jurnal 50%**

e. Tidak menggunakan pakaian sesuai aturan yang berlaku

- **< modul 10: pengurangan nilai jurnal sebesar (Modul ke-) x 10%.**

  Contoh:

  Modul ke-6, pengurangan nilai sebesar **6 x 10% = 60%.**

  Pengurangan nilai tidak diberlakukan apabila praktikan mengganti pakaian sesuai ketentuan sebelum menit ke-30 pelaksanaan praktikum, dan tidak diberikan tambahan waktu.

- **≥ modul 10: tidak diperkenankan mengikuti kegiatan praktikum.**

f. Terbukti menggunakan AI pada saat pengerjaan tugas pendahuluan (TP) dan/atau jurnal praktikum

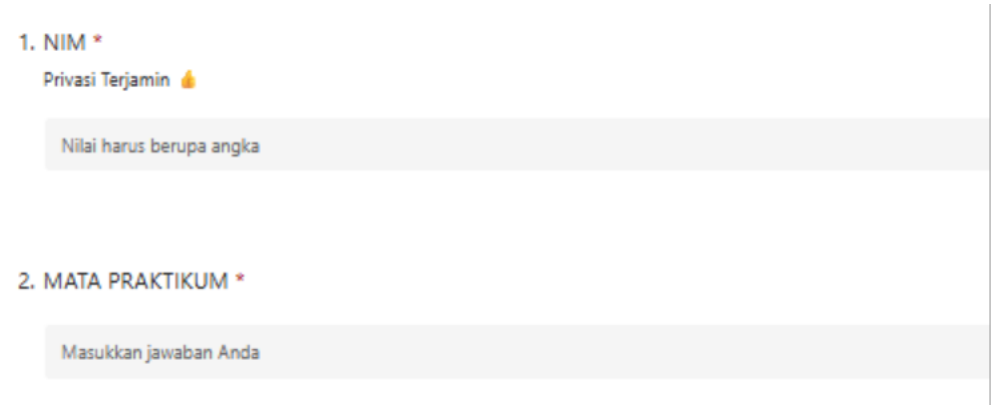
- **Nilai '0' pada tugas pendahuluan dan/atau jurnal modul terkait.**
- **Jika hanya terindikasi: Pengurangan nilai minimal 50% dan diserahkan ke asisten praktikum terkait. Praktikan dapat mengajukan komplain nilai jika tidak terbukti.**

g. Menyalahgunakan fitur LMS dan perangkat pendukung kegiatan praktikum lainnya

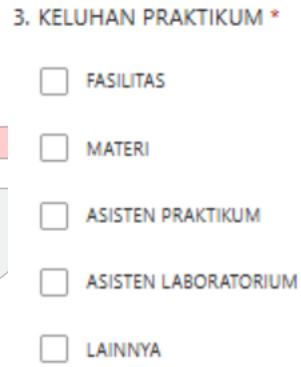
- **Siap menerima sanksi dari lab**

1.3 Penyampaian Keluhan Praktikum IFLAB

1) Buka tautan http://tiny.cc/FEEDBACK2627-1 melalui peramban (browser) Anda.
2) Isi data yang diperlukan seperti NIM dan mata praktikum. Kerahasiaan data Anda dijamin.



3) Pilih kategori keluhan yang sesuai.



4) Tulis keluhan Anda secara jelas dan rinci.



7. KELUHAN KEPADA ASPRAK/ASLAB *

"-" jika tidak ada.

Masukkan jawaban Anda

8. KELUHAN LAINNYA *

"-" jika tidak ada.

Masukkan jawaban Anda

5) Berikan saran dan solusi terkait permasalahan yang Anda sampaikan.

9. SARAN DAN SOLUSI *

"-" jika tidak ada.
Silakan berikan solusi terhadap keluhan anda yang sekiranya dapat diterima oleh IFLAB.

Masukkan jawaban Anda

6) Lampirkan bukti pendukung bila ada.

10. BUKTI

Lampirkan bukti jika ada

Unggah file

Batas jumlah file: 1 Batas ukuran satu file: 10MB Tipe file yang diizinkan: Word, Excel, PPT, PDF, Gambar, Video, Audio

7) Kirim (submit) jawaban Anda.

School of Computing
Telkom University
informatics lab

1.4 Aturan Penulisan dan Submisi Tugas

1) Setiap program harus disimpan dalam file dengan ekstensi/akhiran **.go**.
2) Program harus dibuat rapi dan terstruktur.
    a. Buat indentasi yang benar (Instruksi pada level yang sama dimulai dari kolom yang sama)

    Instruksi di dalam suatu blok, misal di dalam struktur while-loop, harus masuk 4 spasi ke dalam dibandingkan dengan struktur kontrolnya.

    Gunakan perintah **go fmt** sebelum submisi untuk memastikan program Anda mempunyai struktur yang konsisten. Pada contoh di bawah, spasi digambarkan dengan simbol ▌ agar lebih jelas.

    Gunakan perintah **go clean** sebelum submisi untuk membuang semua file yang tidak diperlukan di dalam folder, seperti executable code, sehingga hanya menyisakan file program sumbernya/source code saja.

```
i := 1
for i < 100 {
▌▌▌▌ if i%10 == 0 {
▌▌▌▌▌▌▌▌ fmt.Println( i*3 )
▌▌▌▌ }
```

```
    i = i + 1
}
fmt.Println( “Selesai” )
```

b. Setiap baris hanya boleh berisi satu instruksi saja, walaupun instruksi tersebut sangat pendek

3) Pemilihan nama variabel harus sesuai dengan kebutuhan.

a. **JANGAN** menggunakan kata kunci sebagai bagian dari nama variabel, seperti: **_true, _false, _for, forloop, ...**

b. Pilih nama variabel yang berhubungan dengan nilai yang disimpan dalam variabel tersebut, seperti: **jumlah, rerata, terbesar, ganjil, ketemu, mhs, skor, min, max, ...**

c. Gunakan variabel satu huruf atau akronim untuk manfaat variabel yang sudah umum untuk huruf/akronim tersebut, seperti:

- **i, j, k**, ... untuk indeks, iterator loop, dsb.
- **t, t1, temp**, ... untuk variabel temporer
- **p, q, r**, ... untuk pointer
- **n, m**, ... untuk jumlah data, ukuran array, dsb.
- **u, v, w, x, y, z**, ... untuk nilai dari suatu koleksi data

d. Gunakan penulisan nama variabel yang konsisten.

- **CamelCase**: pada nama berbentuk kata majemuk, setiap kata dimulai dengan huruf besar, kecuali kata pertama. Kata pertama (atau hanya satu kata) maka tetap dimulai dengan huruf kecil. Dalam bahasa Go, nama dimulai dengan huruf besar merupakan nama yang diekspor oleh paket.

  Contoh: **nilaiTerkecil**, **mhsBaru**, **mhsLama**, ...

- Dengan **_underscore**: pada nama berbentuk kata majemuk, kata-kata digabungkan dengan “_”.

  Contoh **nilai_terkecil**, **mhs_baru**, **mhs_lama**, ...

- **HURUF BESAR/KAPITAL** digunakan untuk konstanta simbolik, yaitu nilainya tidak pernah, tidak mungkin, dan tidak boleh diubah dalam program.

  Contoh: **MAX_MHS**, **NMAX**, ...

- Pada tugas pemrograman AP **HINDARI** pemilihan nama yang dimulai dengan underscore.

  Contoh yang dilarang: **_mhs**, **_temp**, ...

4) Setiap algoritma dan program sumbernya selalu berlaku prinsip satu pintu masuk satu pintu keluar.

Fakultas Informatika
School of Computing
Telkom University
Informatics Lab

a. Sekuens instruksi dimulai dari instruksi pertama, eksekusi secara berurutan, dan berakhir pada satu instruksi terakhir.

b. Bentuk perulangan dimulai dari instruksi pertama, masuk ke dalam dan mengulang iterasi badan loop, dan kemudian keluar dari **SATU** lokasi saja, yaitu dari kondisi iterasi tersebut.

   - Jika bentuk struktur **while-loop**, maka loop berakhir dan pintu keluar dari instruksi **while** (bukan dari instruksi **endwhile**).
   - Jika bentuk struktur **repeat-until**, maka loop berakhir dan pintu keluar dari instruksi **until** (bukan dari instruksi **repeat**).
   - Bentuk lain juga menggunakan prinsip yang sama, satu pintu keluar saja, sehingga tidak diperkenankan bentuk **while** dicampur dengan adanya instruksi **break**, atau menggunakan beberapa perintah **break** dalam satu loop, atau menggunakan perintah **continue** di dalam suatu loop.

c. Bentuk percabangan dimulai dari instruksi pertama, kemudian mengeksekusi blok instruksi sesuai dengan kondisinya, dan berakhir pada instruksi penutup (**endif** atau **endcase**).

   - Tidak diperkenankan merenteng beberapa blok instruksi, misalkan menggunakan instruksi **fallthrough**.
   - Tidak diperkenankan keluar dari tengah blok **if** atau **switch** menggunakan perintah **break**, kecuali untuk kebutuhan keluar dari loop yang memenuhi syarat bentuk perulangan di atas.

Fakultas Informatika
School of Computing
Telkom University
Informatics Lab

5) Kelas ini mempelajari konsep pemrograman konvensional. Dalam tugas yang diberikan, tidak diperlukan instruksi yang bersifat konkurensi dan/atau paralel. Ini termasuk untuk **JANGAN** menggunakan instruksi assignment paralel.

6) Algoritma dan program sumbernya harus jelas dan menggunakan tipe data yang sesuai peruntukannya. Beberapa **contoh penyalahgunaan tipe data**:

   a. Tipe **integer**, tetapi nilai yang dipakai hanya 0 dan 1 (atau sejenisnya), yang manfaatnya adalah untuk menyatakan suatu keadaan (ada/tidak ada, ketemu/tidak ketemu), dsb.. Tipe yang tepat yang sebaiknya digunakan sudah jelas adalah tipe **boolean**.

   b. Tipe **real**, tetapi untuk semua operasi yang diterapkan pada variabel tersebut tidak pernah berkaitan dengan pecahan. Tipe yang sebaiknya digunakan adalah tipe **integer**.

7) Algoritma dan program sumber implementasinya harus selalu bersifat efektif, tidak menggunakan instruksi yang pada dasarnya tidak memberikan efek neto yang penting. Beberapa contoh bentuk ekspresi yang berlebihan:

Tabel 1-2 Bentuk Percabangan

Bentuk yang TIDAK TEPAT	Bentuk yang SEHARUSNYA
if found == **true then** .. **if** found != **false then** ..	**if** found **then** ..

Bentuk yang TIDAK TEPAT	Bentuk yang SEHARUSNYA
if found == **false then** .. **if** found != **true then** ..	**if not** found **then** ..
if suatu_kondisi **then** abc = **true** **else** abc = **false** **endif**	abc = suatu_kondisi
abc = def * 1	abc = def
abc = def – 0	abc = def
abc = def * -1	abc = -def

8) Proses debugging adalah bagian dari kegiatan untuk membuat program sumber yang berjalan dengan baik dan sesuai dengan algoritma yang dirancang. Dalam proses debugging mungkin ditambahkan perintah yang input dan output untuk mencari kesalahan yang ada dalam program.

   a. Perintah seperti **fmt.Scanln()**, misalnya sebagai instruksi terakhir dalam program, digunakan untuk menghentikan eksekusi agar keluaran dapat diverifikasi.

   b. Perintah **fmt.Println("...")** digunakan di berbagai lokasi program untuk mencetak isi variabel dan/atau ekspresi tertentu untuk memastikan kebenaran proses yang sedang terjadi.

   c. **Akan tetapi pada saat submisi**, semua perintah tambahan tersebut harus dihapus dari program yang akan diserahkan!

   d. Pastikan semua perintah **fmt.Print*** sudah memberikan **keluaran yang sesuai dalam format**, **bentuk, dan hasil seperti yang diminta** dalam tugas.

   e. Pastikan semua perintah **fmt.Scan*** memang membaca yang harus dibaca, tidak kurang dan tidak lebih.

9) Patuhi etika pendidikan.

   Jika diperintahkan sebagai tugas mandiri, maka selayaknya untuk tidak mengutip, sebagian atau pun seluruhnya, apa adanya atau dengan modifikasi, hasil karya orang lain, rekan lain, atau tim lain.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

1.5 Instalasi Program Penunjang Praktikum

Kode program dengan bahasa pemrograman Go tidak bisa langsung dijalankan oleh komputer, perlu diterjemahkan terlebih dahulu menjadi sebuah program yang dimengerti oleh komputer. Proses penerjemahan ini dikenal dengan istilah kompilasi, yang pada proses ini kode program dicek struktur atau tata bahasanya, kemudian diterjemahkan menjadi suatu program dengan bahasa yang dimengerti oleh komputer yaitu bahasa mesin. Program yang digunakan untuk

menerjemahkan ini disebut Compiler, artinya setiap bahasa pemrograman memiliki compiler-nya masing-masing.

Selain compiler diperlukan juga sebuah program *text editor* khusus atau yang biasa disebut **Integrated Development Environment (IDE)**. Program ini digunakan untuk menuliskan kode program, dan biasanya sudah dilengkapi fitur-fitur yang membantu programmer dalam membuat suatu program.

(1) Instalasi Go Language (GoLang)

Compiler dari program Go dapat diunduh pada tautan berikut https://go.dev/dl/, dan proses installasi dapat dilakukan dengan mengikuti petunjuk yang terdapat pada tautan berikut https://go.dev/doc/install. Kemudian pilih sesuai dengan sistem operasi yang digunakan oleh komputer.

(2) Instalasi Integrated Development Environment (IDE)

Pada pemrograman terdapat banyak sekali IDE yang bisa digunakan, berikut ini adalah beberapa IDE yang umum digunakan untuk pemula.

a. Notepad++ (https://notepad-plus-plus.org/downloads/)
b. Visual Studio Code (https://code.visualstudio.com/)
c. Go Land (https://www.jetbrains.com/go/)

Ikuti setiap petunjuk installasi yang diberikan oleh masing-masing IDE tersebut.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

1.6 Struktur Program menggunakan Go

Dalam kerangka program yang ditulis dalam bahasa pemrograman Go, program utama selalu mempunyai dua komponen berikut:

- **package main** merupakan penanda bahwa file ini berisi program utama.
- **func main()** berisi kode utama dari sebuah program Go.

Komentar, bukan bagian dari kode program, dan dapat ditulis di mana saja di dalam program:

- Satu baris teks yang diawali dengan garis miring ganda ('//') s.d. akhir baris, atau.
- Beberapa baris teks yang dimulai dengan pasangan karakter '/*' dan diakhiri dengan '*/'.

```
1  // Setiap program utama dimulai dengan "package main"
2  package main
3
4  // Impor paket yang dibutuhkan, "fmt" berisi proses I/O standar
5  import “fmt”
6
7  // Kode program utama dalam "fungsi main"
8  func main() {
9     ...
10 }
```

Contoh sebuah program dalam bahasa pemrograman Go (nama file hello.go).

```
1   package main
2   import "fmt"
3   func main() {
4     var greetings = "Selamat datang di dunia DAP"
5     var a, b int
6
7     fmt.Println(greetings)
8     fmt.Scanln(&a, &b)
9     fmt.Printf("%v + %v = %v\n", a, b, a+b)
10  }
```

```
C:\users\go\src\hello>dir
Directory of C:\users\go\src\hello
 6/29/2019   7:15 PM         1,727  hello.go
C:\users\go\src\hello>go build hello.go
C:\users\go\src\hello>dir
Directory of C:\users\go\src\hello
 6/29/2019   7:15 PM         1,727  hello.go
 6/29/2019   7:18 PM     2,198,528  hello.exe
C:\users\go\src\hello>.\hello
Selamat datang di dunia DAP
7 5
7 + 5 = 12
C:\users\go\src\hello>
```

Fakultas Informatika
School of Computing
Telkom University

(1) Koding

- Tidak berbeda dengan penulisan program sumber dalam bahasa lain, program Go harus dibuat menggunakan penyunting teks dan disimpan dalam format teks, bukan dalam format dokumen (doc, docx, atau lainnya).
- Setiap program go disimpan dalam file teks dengan ekstensi *.go, dengan nama bebas. Sebaiknya nama file adalah nama untuk program tersebut.
- Setiap satu program lengkap Go disimpan dalam satu folder tersendiri. Nama folder merupakan nama program tersebut. Karena itu secara prinsip, satu program Go dapat dipecah dalam beberapa file dengan ekstensi *.go selama disimpan dalam folder yang sama.

(2) Kompilasi

Beberapa bahasa pemrograman dirancang untuk diimplementasikan sebagai interpreter dan lainnya sebagai kompilator. Interpreter akan membaca setiap baris instruksi dan kemudian langsung mengeksekusinya, dengan hanya sedikit pemeriksaan apakah penulisan keseluruhan program sudah benar atau belum. Kompilator akan memeriksa keseluruhan program sumber dan kemudian mengubahnya menjadi program eksekutabel, sehingga konsistensi penulisan (seperti penggunaan tipe data) sudah diperiksa sebelum eksekusi. Selain itu karena program dibuat menjadi eksekutabel lebih dahulu, proses optimasi dapat dilakukan sehingga program menjadi sangat efisien.

Go diimplementasikan sebagai kompilator. Berikut adalah contoh sesi yang biasa dilakukan saat mengkompilasi dan mengeksekusi program dalam bahasa Go:

- Panggil shell atau terminal (program/utiliti cmd.exe di Windows)
- Masuk ke dalam (cd) folder program (normalnya ada di C:\Users\go\src\ atau yang sejenis)
- Kemudian panggil perintah go build atau go build file.go untuk mengkompilasi file.go
- Jika gagal, akan muncul pesan eror yang sesuai, pelajari dengan baik pesan tersebut, perbaiki teks program sumber, kemudian ulangi proses build-nya.
- Jika berhasil maka pada folder tersebut akan dibuat program dengan nama yang sama dan diakhiri dengan .exe (untuk Windows)
- Panggil program eksekutabel tersebut dari terminal yang sama. Jangan memanggil program tersebut dengan mengklik eksekutabel tersebut dari folder karena program kalian hanya berbasis teks, bukan/belum dirancang dengan tampilan Windows.

(3) Catatan

Semua proses terkait bahasa Go dilakukan melalui utilitas go. Beberapa opsi dengan utilitas go:

- **go build**: mengkompilasi program sumber yang ada dalam folder menjadi sebuah program.
- **go build file.go**: mengkompilasi program sumber file.go saja.
- **go fmt**: membaca semua program sumber dalam folder dan mereformat penulisannya agar sesuai dengan standar penulisan program sumber Go.
- **go clean**: membersihkan file-file dalam folder sehingga tersisa program sumber nya saja.

Fakultas Informatika
School of Computing
Telkom University

MODUL 2. I/O, TIPE DATA & VARIABEL

2.1 Input dan Output (I/O)

Salah satu Instruksi dasar di dalam pemrograman adalah input/scan/read dan output/print/write.

- **Input**

  Merupakan instruksi dasar untuk membaca data yang diberikan dari pengguna. Data yang diberikan oleh pengguna akan disimpan ke dalam suatu wadah yang disebut variabel. Data ini untuk selanjutnya akan diproses oleh program komputer.

  Pembacaan data ini tentunya memerlukan perangkat/device tambahan yang terhubung oleh komputer. Misalnya: keyboard, mouse, layar sentuh, dan lainnya. Informasi yang diketik oleh pengguna akan disimpan sebagai data komputer di dalam variabel.

  Penulisan instruksi input beragam menyesuaikan bahasa pemrograman yang digunakan (misalnya scan atau read). Penulisan di dalam bahasa pemrograman Go dapat dilakukan seperti contoh berikut ini:

  Tabel 2-1 instruksi_input(variabel1, variabel2, variabel3, ...)

Notasi instruksi dasar	Notasi dalam bahasa Go
input(var_1)	**fmt.Scan**(&var_1)
	fmt.Scanf("%d", &var_1)
input(var_1, nilai_y)	**fmt.Scan**(&var_1, &nilai_y)
Keterangan: • var_1 dan nilai_y merupakan variabel atau wadah yang digunakan untuk menampung data. • Scanf merupakan instruksi input dengan format data tertentu, %d merupakan format untuk data berupa bilangan bulat. • Instruksi Scan lebih direkomendasikan apabila belum memahami string format.	

Fakultas Informatika
School of Computing
Telkom University
informatics lab

- **Output**

  Berbeda dengan Input, maka Output merupakan perintah untuk menampilkan data ke layar monitor* (bisa dari perangkat komputer ataupun mobile). Data yang sudah diproses atau diolah oleh program komputer perlu ditampilkan ke layar sehingga pengguna bisa memperoleh informasi dari hasil pengolahan data yang dilakukan oleh program.

  Kata kunci instruksi yang digunakan juga beragam, menyesuaikan dengan bahasa pemrograman yang digunakan. Contohnya write dan print.

  Catatan*: *Pada konteks ini, perangkat untuk menampilkan data dipersempit menjadi layar komputer, tujuannya untuk membuat mahasiswa lebih mudah memahapi instruksi output.*

Tabel 2-2 instruksi_output(variabel1/data1, variabel2/data2, ...)

Notasi instruksi dasar	Notasi dalam bahasa Go	Tampilan
output("Praktikum",2024)	**fmt.Print**("Praktikum",2024)	Praktikum2024

	fmt.Println("Praktikum",2024)	Praktikum 2024
	fmt.Printf("Praktikum %v",2024)	Praktikum 2024
print("Praktikum") **print**("Algoritma")	**fmt.Println**("Praktikum") **fmt.Println**("Algoritma")	Praktikum Algoritma
	fmt.Print("Praktikum") **fmt.Print**("Algoritma")	PraktikumAlgoritma
Keterangan: • "Praktikum" dan "Algoritma" merupakan data teks. • 2024 merupakan data bilangan bulat. • Instruksi Println menambahkan spasi antar data yang ditampilkan, dan pointer berada di baris baru setelah menampilkan data. Sehingga seolah-olah menambahkan "Enter" atau baris baru diakhir instruksi. • Printf merupakan perintah output dengan format data tertentu.		

Contoh sebuah program dalam bahasa pemrograman Go (nama file hello2.go).

```
1  package main
2  import "fmt"
3  func main() {
4     var mk string = "Algoritma dan Pemrograman"
5     var kode, sks int
6     fmt.Print("Tuliskan kode MK dan SKS: ")
7     fmt.Scan(&kode, &sks)
8     fmt.Println("Kredit MK",kode,"-",mk,"1 adalah",sks,"SKS")
9  }
10
```

Telkom University
informatics lab

```
C:\users\go\src\hello>go build hello2.go
C:\users\go\src\hello>dir
Directory of C:\users\go\src\hello
 6/29/2019   7:15 PM         1,727  hello2.go
 6/29/2019   7:18 PM     2,198,528  hello2.exe
C:\users\go\src\hello>.\hello
Tuliskan kode MK dan SKS: CAK1BAB3 3
Kredit MK CAK1BAB3 – Algoritma dan Pemrograman 1 adalah 3 SKS
C:\users\go\src\hello>
```

2.2 Data, Variabel, dan Instruksi Dasar

Variabel adalah nama dari suatu lokasi di memori, yang data dengan tipe tertentu dapat disimpan.

- Nama variabel dimulai dengan huruf dan dapat dikuti dengan sejumlah huruf, angka, atau garisbawah.

  Contoh: **ketemu**, **found**, **rerata**, **mhs1**, **data_2**, **...**

Tabel 2-3 Pemetaan Tipe Data Dasar Algoritmik ke Tipe Data Bahasa Go

Notasi tipe dasar	Tipe dalam Go	Keterangan
integer	int int8 int32 //rune	bergantung platform 8 bit: -128..127 32 bit: -10^9..10^9

	int64 uint uint8 //byte uint32 uint64	64 bit: -10^19..10^19 bergantung platform 0..255 0..4294967295 0..(2^64-1)
real	float32 float64	32bit: -3.4E+38 .. 3.4E+38 64bit: -1.7E+308 .. 1.7E+308
boolean (atau logikal)	**bool**	**false** dan **true**
karakter	byte //uint8 rune //int32	tabel ASCII/UTF-8 tabel UTF-16
string	string	

- Tipe data yang umum tersedia adalah integer, real, boolean, karakter, dan string. Lihat tabel berikut ini untuk variasi tipe data yang disediakan dalam bahasa Go.
- **Nilai data** yang tersimpan dalam variabel dapat diperoleh dengan menyebutkan langsung nama variabelnya.

  Contoh: Menyebutkan nama **found** akan mengambil nilai tersimpan dalam memori untuk variabel **found**, pastinya.
- **Informasi alamat** atau lokasi dari variabel dapat diperoleh dengan menambahkan prefiks **&** di depan nama variabel tersebut.

  Contoh: **&found** akan mendapatkan alamat memori untuk menyimpan data pada **found**.
- Jika variabel berisi alamat memori, prefiks ***** pada variabel tersebut akan memberikan nilai yang tersimpan dalam memori yang lokasinya disimpan dalam variabel tersebut.

  Contoh: ***mem** akan mendapatkan data di memori yang alamatnya tersimpan di **mem.** Karenanya ***(&found)** akan mendapatkan data dari lokasi memori variabel **found** berada, alias sama saja dengan menyebutkan langsung **found** 8=).
- Operasi yang dapat dilakukan terhadap tipe data di atas adalah:

Tabel 2-4 Daftar Operator dalam Bahasa Go dan Tipe Data Terkait

Operator dalam Go	Tipe data terkait	Keterangan
+	string integer dan real	konkatenasi 2 string operasi penjumlahan
- * /	integer dan real	operasi pengurangan, perkalian, dan pembagian
%	integer	operasi sisa pembagian integer (modulo)
& | ^ &^	integer	operasi **per-bit** AND, OR, XOR, AND-NOT
<< >>	integer dan unsigned integer	operasi geser bit kiri/kanan sebanyak unsigned integer yang diberikan
< <= >= > == !=	selain boolean	komparasi menghasilkan nilai boolean komparasi karakter sesuai dengan posisi karakter tersebut dalam tabel ASCII/UTF-16 komparasi string sesuai dengan operasi karakter per karakter, dimulai dari karakter paling kiri (awal)
&& || !	boolean	operasi **boolean** AND, OR, dan NOT
* &	variabel apasaja	mendapatkan data dari lokasi memori dan mendapatkan lokasi dari variabel

Fakultas Informatika
School of Computing
Telkom University
Informatics Lab

Contoh:

Tabel 2-5 Contoh Operasi Tipe Data dan Hasil Evaluasinya dalam Bahasa Go

Operasi	Hasil
"non suffi" + "cit mundo"	"non sufficit mundo"
2019.01 + 1.0102	2020.0202
2020 / 20	22.22
20.2 * 1.1	101
2020 % 1999	21
2020 & 1111	2104
2020 ^ 1111	1663
2020 >> 2	505
"minutus" < "magnus"	false
2020 >= 1234	true
! false && true	true

- Bahasa Go menganut kesesuaian tipe data yang ketat. Tipe data yang berbeda tidak boleh dicampur dalam satu ekspresi, bahkan tipe data masih yang sejenis, misalnya masih sama-sama integer (**int** dan **int32**). Untuk menyesuaikan tipe data, ada beberapa cara yang dapat dilakukan:
  - ✓ Casting, **tipe(data),** mengubah tipe dari data yang diberikan ke tipe yang diinginkan.
  - ✓ Memanfaatkan fungsi **Sprint** dan **Sscan** dari paket **fmt**.
  - ✓ Memanfaatkan fungsi-fungsi dalam paket **strconv**, seperti **Atoi**, **Itoa**, dan **ParseBool**. Lihat lampiran untuk contoh penggunaan.

Fakultas Informatika
School of Computing
Telkom University

Contoh:

Tabel 2-6 Contoh Penyesuaian (Casting) Tipe Data dalam Bahasa Go

Operasi	Hasil
2020.0 % 19	will be an illegal expression error
int(2020.0) % 19	6

Tabel 2-7 Metode Konversi Tipe Data (Type Casting dan Format String) dalam Bahasa Go

Konversi tipe	Data	Tipe baru	Keterangan
tipe(data)	integer	integer	format data tidak berubah, hanya penyesuaian jumlah bit. Kekurangan bit diisi bit 0 di sebelah kiri (MSB)
	real	real	format data tidak berubah, hanya penyesuaian jumlah bit. Kekurangan bit, maka bit mantisa diisi bit 0.
	real	integer	format data disesuaikan dengan tipe data tujuan
	integer	real	format data disesuaikan dengan tipe data tujuan

fmt.Sprintf("%v",v)	any type	string	tulis output ke string
fmt.Sprintf("%c",v)	karakter	string	tulis karakter ke string
fmt.Sscanf(s,"%v",&v)	string	any type	baca string ke variabel dengan tipe tertentu
fmt.Sscanf(s,"%c",&v)	string	karakter	baca string ke variabel bertipe karakter

- Variabel harus dideklarasikan dulu sebelum digunakan. Variabel juga harus diinisialisasi dulu (diisi data) agar nilai yang tersimpan diketahui dengan jelas dan eksekusi algoritma menjadi terprediksi. Dalam bahasa Go, variabel yang tidak diinisialisasi lebih dahulu otomatis diisi dengan nilai default yang ekuivalen dengan bit 0.
  - ✓ Nilai 0 untuk bilangan integer
  - ✓ 0.0E+0 untuk bilangan real
  - ✓ **false** untuk boolean
  - ✓ Karakter NUL (lihat tabel ASCII) untuk karakter
  - ✓ "" (string kosong, string dengan panjang 0) untuk string
  - ✓ **nil** untuk alamat memori
- Variabel bisa dianalogikan sebagai wadah yang akan digunakan untuk menyimpan data.

Tabel 2-8 Notasi Deklarasi Variabel Algoritmik dan Penulisannya dalam Bahasa Go

Fakultas Informatika School of Computing Telkom University informatics lab

Notasi deklarasi variabel	Penulisan dalam Go	Keterangan
kamus a : tipe	**var** a tipe	a diinisialisasi dengan nilai default
kamus a : tipe **algoritma** a = nilai_awal	**var** a tipe = nilai_awal **var** a = (tipe)nilai_awal	a diinisialisasi dengan nilai_awal
	a := nilai_awal a := (tipe)nilai_awal	secara **implisit**, tipe variabel a ditentukan dari nilai inisialisasinya

Contoh:

```
1  func main() {
2     var a int
3     a = 2019
4     r := 2019.0707
5     b := false
6     c := 'x'
7     s := "a string is a string"
8  }
```

- Instruksi Dasar

Tabel 2-9 Notasi Instruksi Dasar Algoritmik dan Penulisannya dalam Bahasa Go

Notasi instruksi dasar	Penulisan dalam bahasa Go	Keterangan
v1 = e1 v1 = v1 + e1 v1 = v1 – e1 v1 = v1 + 1 v1 = v1 - 1	v1 = e1 v1 += e1 // atau v1 = v1 + e1 v1 -= e1 // atau v1 = v1 – e1 v1++ // atau v1 = v1 + 1 v1-- // atau v1 = v1 - 1	operasi assignment, mengisi data ke lokasi memori (variabel)
input(v1, v2)	fmt.**Scan**(&v1, &v2) fmt.**Scanln**(&v1, &v2) fmt.**Scanf**("%v %v", &v1, &v2)	Pembacaan data memerlukan alamat memori ke mana data akan disimpan.
output(e1, e2)	fmt.**Print**(e1, e2) fmt.**Println**(e1, e2) fmt.**Printf**("%v %v\n", e1, e2)	Penulisan data memerlukan nilai data yang akan ditulis.

Contoh:

```
1  package main
2  import "fmt"
3
4  func main() {
5     var a, b, c float64
6     var hipotenusa bool
7
8     fmt.Scanln( &a, &b, &c )
9     hipotenusa = (c*c) == (a*a + b*b)
10    fmt.Println( "Sisi c adalah hipotenusa segitiga a,b,c: ", hipotenusa)
11 }
```

Telkom University
informatics lab

2.3 Konstanta Simbolik

Konstanta dapat diberi nama untuk memudahkan mengingat maksud dan manfaat dari nilai yang diberi nama tersebut. Seperti PI untuk merepresentasikan konstanta π.

```
1  const PI = 3.14159265358979323846264433
2  const MARKER = "AKHIR"
```

2.4 Tipe Data Character

Tipe data character pada dasarnya merupakan tipe data integer, tetapi digunakan untuk merepresentasikan suatu simbol tertentu berdasarkan tabel acuan ASCII/UTF-8 atau UTF-16.

Tabel 2-10 Representasi Tipe Data Karakter (Char) dalam Bahasa Go

Notasi tipe dasar	Tipe dalam Go	Keterangan
char	byte //uint8 rune //int32	tabel ASCII/UTF-8 tabel UTF-16

Tabel 2-11 Contoh potongan tabel ASCII untuk beberapa bilangan integer

	0	1	2	3	4	5	6	7

32 **40**	SPC (	!)	“ *	# +	$,	% -	& .	' /
48 **56**	0 8	1 9	2 :	3 ;	4 <	5 =	6 >	7 ?
64 **72**	@ H	A I	B J	C K	D L	E M	F N	G O
80 **88**	P X	Q Y	R Z	S [	T \	U]	V ^	W _
96 **104**	` h	a i	b j	c k	d l	e m	f n	g o
112 **120**	p x	q y	r z	s {	t |	u }	v ~	w DEL

Simbol pada tipe data character ditulis dengan menambahkan petik satu, Contohnya bilangan 64 dapat ditulis dengan simbol '@' (dengan petik) dan bilangan 97 dapat ditulis dengan simbol 'a'.

2.5 Contoh Soal Modul 2

Berikut adalah beberapa contoh soal terkait penggunaan tipe data integer, character dan operasinya.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

1) Buatlah program yang digunakan untuk menghitung hasil penjumlahan 5 bilangan bulat.
   Masukan terdiri dari lima bilangan bulat a, b, c, d, dan e.
   Keluaran berupa bilangan hasil penjumlahan lima bilangan bulat a, b, c, d, dan e.
   Contoh masukan dan keluaran:

No.	Masukan	Keluaran
1	3 2 7 10 2	24
2	11 22 33 44 55	165

Jawaban:

```
1   // filename : penjumlahan.go
2   package main
3   import “fmt”
4
5   func main() {
6     var a, b, c, d, e int
7     var hasil int
8     fmt.Scanln( &a, &b, &c, &d, &e)
9     hasil = a + b + c + d + e
10    fmt.Println("Hasil penjumlahan",a,b,c,d,e,"adalah",hasil)
11  }
```

```
C:\users\go\src\hello>go build penjumlahan.go
C:\users\go\src\hello>.\penjumlahan
11 22 33 44 55
Hasil penjumlahan 11 22 33 44 55 adalah 165
```

2) Sebuah program digunakan untuk menghitung persamaan $f(x) = \frac{2}{x+5} + 5$.

Masukan terdiri dari sebuah bilangan bulat.
Keluaran berupa bilangan yang menyatakan nilai dari $f(x)$.
Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	5.2
2	-23	4.888888888888888889

Jawaban:

```
1  package main
2  import "fmt"
3
4  func main(){
5    var x, fx float64
6    fmt.Scan(&x)
7    fx = 2/(x+5) + 5
8    fmt.Println(fx)
9  }
```

3) Tipe karakter sebenarnya hanya apa yang tampak dalam tampilan. Di dalamnya tersimpan dalam bentuk biner 8 bit (byte) atau 32 bit (rune) saja.
Buat program ASCII yang akan membaca 5 buat data integer dan mencetaknya dalam format karakter. Kemudian membaca 3 buah data karakter dan mencetak 3 buah karakter setelah karakter tersebut (menurut tabel ASCII)

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Masukan terdiri dari dua baris. Baris pertama berisi 5 buah data integer. Data integer mempunyai nilai antara 32 s.d. 127. Baris kedua berisi 3 buah karakter yang berdampingan satu dengan yang lain (tanpa dipisahkan spasi).

Keluaran juga terdiri dari dua baris. Baris pertama berisi 5 buah representasi karakter dari data yang diberikan, yang berdampingan satu dengan lain, tanpa dipisahkan spasi. Baris kedua berisi 3 buah karakter (juga tidak dipisahkan oleh spasi).

Contoh masukan dan keluaran:

No.	Masukan	Keluaran
1	66 97 103 117 115 SNO	Bagus TOP

Catatan: Gunakan fmt.**Scanf**("%c", &var) untuk pembacaan satu karakter dan fmt.**Printf**("%c", var) untuk penulisan satu karakter.

Jawaban:

```
1  // filename : ascii.go
2  package main
3  import "fmt"
4  func main(){
5    var c1, c2, c3, c4, c5 byte
```

```
C:\users\go\src\hello>go build ascii.go
C:\users\go\src\hello>.\ascii
11 22 33 44 55
66 97 103 117 115
```

```
6     var b1, b2, b3 int
7
8     fmt.Scan(&c1, &c2, &c3, &c4, &c5)
9     fmt.Scanf("%c",&b1)
10    fmt.Scanf("%c",&b2)
11    fmt.Scanf("%cc",&b3)
12
13    fmt.Printf("%c%c%c%c%c",c1, c2, c3, c4, c5)
14    fmt.Printf("%c%c%c",b1+1, b2+1, b3+1)
15 }
16
```

```
SNO
Bagus
TOP
C:\users\go\src\hello>
```

2.6 Soal Latihan Modul 2

1) Telusuri program berikut dengan cara mengkompilasi dan mengeksekusi program. Silakan masukan data yang sesuai sebanyak yang diminta program. Perhatikan keluaran yang diperoleh. Coba terangkan apa sebenarnya yang dilakukan program tersebut?

```
1  package main
2  import “fmt”
3
4  func main() {
5     var (
6        satu, dua, tiga string
7        temp string
8     )
9     fmt.Print("Masukan input string: ")
10    fmt.Scanln(&satu)
11    fmt.Print("Masukan input string: ")
12    fmt.Scanln(&dua)
13    fmt.Print("Masukan input string: ")
14    fmt.Scanln(&tiga)
15    fmt.Println("Output awal = " + satu + " " + dua + " " + tiga)
16    temp = satu
17    satu = dua
18    dua = tiga
19    tiga = temp
20    fmt.Println("Output akhir = " + satu + " " + dua + " " + tiga)
21 }
```

2) Buatlah program yang digunakan untuk menampilkan data biodata yang diberikan dari masukan pengguna, kemudian menampilkan data tersebut!

   Masukan terdiri dari 3 buah teks yang menyatakan nama, nim dan kelas.

   Keluaran menampilkan resume singkat mahasiswa sesuai masukan yang diberikan.

   Catatan: tidak ada batasan terkait resume yang akan ditampilkan. Silahkan berkreasi.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	Bima 1124431414 IF-48-GAB	Perkenalkan saya adalah Bima, salah satu mahasiswa Prodi S1-IF dari kelas IF-48-GAB dengan NIM 1124431414.

2	Yura 1324234545 IFX-48-12	Perkenalkan saya adalah Yura, salah satu mahasiswa Prodi S1-IF dari kelas IFX-48-12 dengan NIM 1324234545.

3) Sebuah program digunakan untuk menghitung luas lingkaran berdasarkan panjang jari-jari.

Masukan terdiri dari sebuah bilangan riil yang menyatakan jari-jari lingkaran.

Keluaran berupa hasil perhitungan yang menyatakan luas lingkaran.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	7	153.9
2	14	615.8
3	20	1256.6

4) Sebuah digunakan untuk melakukan konversi suhu dari Fahreinheit (F) ke Celcius (C) dengan persamaan sebagai berikut: $F = C \times \frac{9}{5} + 32$

Masukan terdiri dari bilangan bulat yang menyatakan suhu dalam satuan Fahreinheit.

Keluaran berupa suhu dalam satuan Celcius.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	32	0
2	77	25
3	212	100

Fakultas Informatika
Computing
Telkom University
informatics lab

MODUL 3. I/O, TIPE DATA & VARIABEL (LATIHAN 1)

Modul ini merupakan kelanjutan dari modul sebelumnya. Fokus dari modul 3 ini adalah pendalaman materi yang terdapat pada modul 2. Materi pada modul 3 ini sifatnya tambahan dan dibahas secara khusus, tetapi bukanlah menjadi materi utama untuk soal-soal yang ada di modul 3 ini. Soal yang diberikan pada modul ini tetap serupa dengan soal-soal pada modul sebelumnya, yang bersifat lebih umum dengan ruang lingkup masih seputar instruksi input/output, tipe data dan variabel.

3.1 Integer Division dan Modulo

Pembagian pada tipe data integer (**integer division** atau **div**) sedikit berbeda dengan pembagian yang sudah sering kita pelajari dari sekolah dasar. Hasil pembagian akan bertipe data integer, artinya kita akan mengabaikan bilangan yang muncul setelah tanda koma atau floating point. Hasil dari operasi div ini dikenal dengan istilah **quotient**.

Sebagai contoh hasil dari 10 (**dividend**) dibagi dengan 3 (**divisor**) adalah 3.333, sedangkan hasil div adalah 3. Artinya kita mengabaikan floating point dari hasil pembagian tersebut. *Operator "/" di dalam bahasa pemrograman Go akan melakukan operasi integer division apabila tipe data dari kedua operan adalah integer dan pembagian biasa apabila operan bertipe float*.

Contoh:

```
var bil_1 int = 10
var bil_2 int = 3
var hasil = bil_1 / bil_2    // maka hasil akan bernilai 3 apabila ditampilkan
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Modulo atau **modulus** "**mod**" adalah operasi untuk mencari nilai integer dari sisa pembagian pada integer division, biasanya dikenal juga dengan istilah **remainder**. Misalnya untuk operasi 10 dibagi dengan 3, maka sisa pembagian adalah 1. Operator modulo menggunakan "%" di dalam bahasa pemrograman Go.

Contoh:

$$3\sqrt{10} = 3$$
$$\underline{9}$$
$$1$$

```
var bil_1 int = 10
var bil_2 int = 3
var hasil = bil_1 % bil_2    // maka hasil akan bernilai 1 apabila ditampilkan
```

Hubungan antara div dan mod dapat diilustrasikan sebagai berikut:

dividend = quotient x divisor + remainder

Pada persamaan di atas **quotient** dan **remainder** adalah hasil operasi **div** dan **mod** antara **dividend** dan **divisor**.

Contoh penggunaan adalah mencari nilai digit di dalam suatu bilangan.

Semua bilangan apabila di modulus dengan 10 akan didapat satuan atau digit terakhir. Misalnya: 1234 mod 10 akan diperoleh nilai 4, 5677 mod 10 akan diperoleh nilai 7. Apabila 1234 div 10 akan diperoleh 123, sedangkan 5677 div 10 akan didapat nilai 567. Pola ini bisa dimanfaatkan untuk mencari nilai digit-digit yang lain.

Misalnya X adalah 2357, maka diperoleh pola berikut:

Tabel 3-1 Simulasi Ekstraksi Digit Menggunakan Operasi div dan mod

i	Digit ke-i	Operasi	Keterangan
1	2	(X div 1000) mod 10	X div 1000 menghasilkan 2, tidak perlu mod 10
2	3	(X div 100) mod 10	X div 100 menghasilkan 23
3	5	(X div 10) mod 10	X div 10 menghasilkan 235
4	7	(X div 1) mod 10	X div 1 menghasilkan X, jadi tidak perlu dilakukan

3.2 Casting atau Konversi Tipe Data

Pada bahasa pemrograman Go, tipe data bersifat statis, artinya tipe data yang sudah didefiniskan tidak dapat diganti selama program berjalan. Casting merupakan salah satu teknik konversi antar tipe data di dalam bahasa pemrograman.

Contoh:

```
var pi float64 = 3.14
var nilai int = int(pi)
```

Pada contoh tersebut, dilakukan casting terhadap nilai pi 3.14 menjadi bertipe integer pada variabel nilai. Cara casting ini dapat dilakukan dengan menuliskan tipe data tujuan sebagai fungsi yang kemudian nilainya di-assign ke dalam variabel dengan tipe data tujuan.

```
var var_name data_type = data_type(value)
```

Fakultas Informatika
School of Computing
Telkom University
informatika lab

Contoh yang lain adalah casting dengan menggunakan paket **strconv** untuk mengubah tipe data string menjadi integer ataupun sebaliknya. Berikut contoh konversi dari string ke integer:

```
var teks string = "2024"
tahun, err := strconv.Atois(teks)
```

Berikutnya adalah contoh konversi dari integer menjadi string:

```
var bilangan int = "113071049"
var teks string = strconv.Itoa(bilangan)
```

3.3 Contoh Soal Modul 3

1) Buatlah program dalam Bahasa Go untuk menghitung volume kubus berdasarkan panjang sisinya.

   Masukan terdiri dari suatu bilangan bulat positif yang menyatakan panjang sisi kubus.

   Keluaran berupa bilangan yang menyatakan volume dari kubus.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	3	27
2	4	64
3	5	125

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var sisi, volume float64
5     fmt.Scan(&sisi)
6     volume = sisi * sisi * sisi
7     fmt.Println(volume)
8  }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
3
27
E:\DEV\NATIVE\GO>.\Demo_Soal
4
64
E:\DEV\NATIVE\GO>.\Demo_Soal
5
125
```

2) Buatlah program dalam Bahasa Go yang digunakan untuk menghitung luas segitiga, apabila diketahui panjang alas dan tinggi dari segitiga.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Masukan terdiri dari dua bilangan bulat positif yang menyatakan panjang alas dan tinggi dari segitiga.

Keluaran berupa bilangan yang menyatakan luas dari segitiga.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	8 5	20
2	6 7	21
3	12 15	90

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var alas, tinggi, luas float64
5     fmt.Scan(&alas, &tinggi)
6     luas = 0.5 * alas * tinggi
7     fmt.Println(luas)
8  }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
8 5
20
E:\DEV\NATIVE\GO>.\Demo_Soal
6 7
21
E:\DEV\NATIVE\GO>.\Demo_Soal
12 15
90
```

3) Sebuah program digunakan untuk menghitung konversi mata uang dari IDR ke Dolar US dengan Kurs 15,000 IDR / USD.

   Masukan terdiri dari bilangan bulat yang menyatakan uang dalam satuan IDR.

   Keluaran terdiri dari bilangan yang menyatakan uang dalam satuan USD.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	15000	1
2	75000	5
3	300000	20

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var rupiah, dolar int
5     fmt.Scan(&rupiah)
6     dolar = rupiah / 15000
7     fmt.Println(dolar)
8  }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
15000
1
E:\DEV\NATIVE\GO>.\Demo_Soal
75000
5
E:\DEV\NATIVE\GO>.\Demo_Soal
300000
20
E:\DEV\NATIVE\GO>.\Demo_Soal
1
0
```

3.4 Soal Latihan Modul 3

1) Sebuah program digunakan untuk menghitung nilai x pada persamaan $f(x) = \frac{2}{x+5} + 5$, apabila diberikan nilai $f(x)$

Masukan terdiri dari sebuah bilangan riil yang menyatakan $f(x)$.

Keluaran berupa bilangan yang menyatakan nilai dari x

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5.2	5
2	4.125	11

2) Buat program **Bola** yang menerima input jari-jari suatu bola (bilangan bulat). Tampilkan Volume dan Luas kulit bola. $volumebola = \frac{4}{3}\pi r^3$ dan $luasbola = 4\pi r^2$ ($\pi \approx 3.1415926535$).

(Contoh input/output, **Teks bergaris bawah** adalah input dari user):

Jejari = **5** Bola dengan jejari 5 memiliki volume 523.5988 dan luas kulit 314.1593

School of Computing
Telkom University

3) Tahun kabisat adalah tahun yang habis dibagi 400 atau habis dibagi 4 tetapi tidak habis dibagi 100. Buatlah sebuah program yang menerima input sebuah bilangan bulat dan memeriksa apakah bilangan tersebut merupakan tahun kabisat (**true**) atau bukan (**false**).

(Contoh input/output, **Teks bergaris bawah** adalah input dari user):

1	Tahun: **2016** Kabisat: true
2	Tahun: **2000** Kabisat: true
3	Tahun: **2018** Kabisat: false

4) Dibaca nilai temperatur dalam derajat Celsius. Nyatakan temperatur tersebut dalam Fahrenheit

$$Celsius = (Fahrenheit - 32) \times \frac{5}{9}\ Reamur = Celcius \times \frac{4}{5}\ Kelvin = (Fahrenheit + 459.67) \times \frac{5}{9}$$

(Contoh input/output, **Teks bergaris bawah** adalah input dari user):

Temperatur Celsius: **50** Derajat Fahrenheit: 122

Lanjutkan program di atas, sehingga temperatur dinyatakan juga dalam derajat Reamur dan Kelvin.

(Contoh input/output, **Teks bergaris bawah** adalah input dari user):

Temperatur Celsius: **50**
Derajat Reamur: 40
Derajat Fahrenheit: 122
Derajat Kelvin: 323

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 4. I/O, TIPE DATA & VARIABEL (LATIHAN 2)

Modul ini merupakan kelanjutan dari modul sebelumnya. Fokus dari modul 4 ini adalah pendalaman terkait materi yang terdapat pada modul 2 dan 3 sebelumnya.

4.1 Contoh Soal Modul 4

1) Sebuah program digunakan untuk mengkonversi detik ke jam, menit dan detik.

   Masukan terdiri dari suatu bilangan bulat positif yang menyatakan waktu dalam satuan detik.

   Keluaran terdiri dari suatu teks yang menyatakan jam, menit dan detik.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	3661	1 jam 1 menit dan 1 detik
2	7322	2 jam 2 menit dan 2 detik
3	3600	1 jam 0 menit dan 0 detik

   Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var detik, jam, menit int
5     fmt.Scan(&detik)
6     jam = detik / 3600
7     menit = (detik % 3600) / 60
8     detik = detik % 60
9     fmt.Println(jam, "jam", menit, "menit dan", detik,"detik")
10 }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
3661
1 jam 1 menit dan 1 detik
E:\DEV\NATIVE\GO>.\Demo_Soal
7322
2 jam 2 menit dan 2 detik
E:\DEV\NATIVE\GO>.\Demo_Soal
3600
1 jam 0 menit dan 0 detik
```

2) Sebuah program digunakan untuk menentukan apakah setiap digit pada suatu bilangan terurut membesar atau tidak.

   Masukan terdiri dari suatu bilangan bulat positif dengan jumlah digit adalah 3 (atau antara 100 sampai dengan 999)

   Keluaran adalah boolean yang menyatakan bilangan terurut membesar atau tidak.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	362	false
2	256	true
3	189	true

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var bilangan, d1, d2, d3 int
5     fmt.Scan(&bilangan)
6     d1 = bilangan / 100
7     d2 = bilangan % 100 / 10
8     d3 = bilangan % 100 % 10
9     fmt.Println(d1 <= d2 && d2 <= d3)
10 }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
362
false
E:\DEV\NATIVE\GO>.\Demo_Soal
256
true
E:\DEV\NATIVE\GO>.\Demo_Soal
189
true
E:\DEV\NATIVE\GO>.\Demo_Soal
555
true
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Sebuah program digunakan untuk menghitung BMI atau Body Mass Indeks, yang mana BMI merupakan hasil bagi dari berat badan dengan kuadrat dari tinggi badan.

   Masukan terdiri dari dua bilangan riil yang menyatakan berat (kg) dan tinggi badan (m).

   Keluaran berupa nilai BMI berdasarkan berat dan tinggi badan yang diberikan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	70 1.75	22.85
2	60 1.6	23.43
3	80 1.8	24.69

Jawaban

```go
package main
import "fmt"
func main() {
    var beratBadan, tinggiBadan, bmi float64
    fmt.Scan(&beratBadan, &tinggiBadan)
    bmi = beratBadan / (tinggiBadan * tinggiBadan)
    fmt.Printf("%.2f", bmi)
}
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
70 1.175
50.70
E:\DEV\NATIVE\GO>.\Demo_Soal
60 1.6
23.44
E:\DEV\NATIVE\GO>.\Demo_Soal
80 1.8
24.69
```

4.2 Soal Latihan Modul 4

1) Sebuah program digunakan untuk menghitung total harga setelah memperoleh diskon dengan besaran tertentu.

   Masukan terdiri dari dua baris. Baris pertama adalah bilangan bulat yang menyatakan total belanja awal, sedangkan baris kedua merupakan bilangan bulat yang menyatakan besarnya diskon dalam satuan persen.

   Keluaran berupa total belanja akhir setelah dipotong oleh diskon.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	100000 10	90000
2	200000 20	160000
3	150000 15	127500

2) Buatlah program yang digunakan untuk menentukan berat badan seseorang apabila diketahui nilai BMI dan tinggi badannya.

   Masukan terdiri dari dua buah bilangan riil yang menyatakan nilai BMI dan tinggi badan dalam satuan meter.

   Keluaran berupa berat badan seseorang dalam satuan kilogram.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	22.85 1.75	70

Fakultas Informatika School of Computing Telkom University informatics lab

2	23.43 1.6	60
3	24.69 1.8	80

3) Dalam sebuah peta, terdapat tiga titik yang membentuk segitiga. Titik A, B, dan C masing-masing memiliki koordinat dalam sistem kartesius 2 dimensi. Tugas Anda adalah menghitung panjang sisi-sisi segitiga yang dibentuk oleh titik-titik tersebut dan menentukan sisi terpanjang dari segitiga tersebut menggunakan teorema Pythagoras.

Masukan terdiri dari tiga baris, yang mana masing-masing berisi dua bilangan riil yang menyatakan koordinat titik A, B, dan C dalam format x y.

Keluaran Sebuah bilangan riil yang menyatakan panjang sisi terpanjang dari segitiga yang dibentuk oleh titik-titik tersebut. Hasil harus ditampilkan dengan dua angka di belakang koma.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	1.0 1.0 4.0 1.0 1.0 5.0	4.47
2	0.0 0.0 3.0 0.0 3.0 4.0	5

Petunjuk: Gunakan **math.Pow** untuk pemangkatan dan **math.Sqrt** untuk menghitung akar kuadrat. Sisi terpanjang harus ditentukan dari hasil perhitungan panjang sisi segitiga.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 5 & 6. FOR-LOOP

5.1 Paradigma Perulangan

Perulangan merupakan salah satu struktur kontrol yang memungkinkan suatu instruksi yang sama dilakukan berulang kali dalam waktu atau jumlah yang lama. Tanpa instruksi perulangan, maka suatu instruksi akan ditulis dalam jumlah yang sangat banyak.

Sebagai contoh adalah menuliskan suatu teks "CAK1BAB3 Algoritma Pemrograman 1" sebanyak 1000 baris. Maka tidak mungkin kita menuliskan instruksi fmt.Println("CAK1BAB3 Algoritma Pemrograman 1") sebanyak 1000x (walaupun memungkinkan menggunakan copy-paste).

Berikut ilustrasinya:

Tabel 5-1 Perbandingan Penulisan Kode Program Tanpa dan Menggunakan Perulangan

Tanpa perulangan	Dengan perulangan

fmt.**Println**("CAK1BAB3 Algoritma Pemrograman 1")
fmt.**Println**("CAK1BAB3 Algoritma Pemrograman 1")
fmt.**Println**("CAK1BAB3 Algoritma Pemrograman 1")
...
...
...
dst
	// instruksi perulangan 1000x
	fmt.**Println**("CAK1BAB3 Algoritma Pemrograman 1")
	// batas instruksi perulangan 1000x

Fakultas Informatika
School of Computing
Telkom University

Terlihat pada ilustrasi di atas, penulisan dengan instruksi perulangan jauh lebih singkat dan jelas dibandingkan dengan tanpa perulangan.

Penting! Pastikan bahwa instruksi perulangan yang digunakan pasti bisa membuat proses perulangan berhenti, apabila tidak maka program akan terus berjalan mengulangi instruksi tanpa akan pernah berhenti.

5.2 Karakteristik For-Loop (Perulangan berdasarkan iterasi)

Salah satu instruksi perulangan yang paling mudah adalah **for-loop**, yang mana dengan instruksi ini dapat digunakan untuk mengulangi instruksi sebanyak $\boldsymbol{n}$ kali (iterasi). Batasan besar nilai dari $\boldsymbol{n}$ menyesuaikan dengan batasan dari tipe data integer yang digunakan.

Tabel 5-2 Notasi Perulangan For-Loop pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go

for inisialisasi **to** n **do**
  // instruksi yang akan diulangi
endfor
	for inisialisasi; kondisi; update {
	  // instruksi yang akan diulangi
	}

Keterangan: • Gunakan tanda **titik koma ";" (semicolon)** untuk memisahkan inisialisasi, kondisi dan update pada instruksi for-loop.	

- Berikan tab atau 4 spasi untuk instruksi yang akan diulangi. Tujuannya untuk mempermudah pembacaan kode dan mengetahui bagian kode mana yang akan diulangi.

Instruksi for-loop memiliki beberapa komponen, yaitu:

1) **inisialisasi** merupakan assignment **variabel iterasi** yang bertipe integer. Pada contoh di atas biasanya **variabel iterasi = 0** atau **1**, artinya iterasi dimulai dari 0 atau 1.
2) **kondisi** merupakan suatu operasi bernilai boolean yang menyatakan kapan perulangan harus dilakukan. Pada contoh di atas **kondisi** adalah **variabel iterasi <= n** (kurang dari atau sama dengan)
3) **update** merupakan ekspresi yang menyatakan perubahan nilai dari **variabel iterasi**. Pada contoh di atas update adalah **variabel iterasi = variabel iterasi + 1**.

5.3 Implementasi menggunakan Go

Sebagai contoh penulisan for-loop dengan menggunakan bahasa Go adalah sebagai berikut, yang mana kita akan gunakan contoh menampilkan teks "CAK1BAB3 Algoritma Pemrograman 1" sebanyak 1000x.

```
1   // filename: Alpo1.go
2   package main
3   import "fmt"
4
5   func main() {
6     var iterasi, n int
7     n = 1000
8
9     for iterasi = 1; iterasi <= n; iterasi = iterasi + 1 {
10      fmt.Println(iterasi, "CAK1BAB3 Algoritma Pemrograman 1")
11    }
12  }
```

```
C:\users\go\src\hello>go build Alpro1.go
C:\users\go\src\hello>.\Alpro1
1 CAK1BAB3 Algoritma Pemrograman 1
2 CAK1BAB3 Algoritma Pemrograman 1
3 CAK1BAB3 Algoritma Pemrograman 1
4 CAK1BAB3 Algoritma Pemrograman 1
5 CAK1BAB3 Algoritma Pemrograman 1
6 CAK1BAB3 Algoritma Pemrograman 1
7 CAK1BAB3 Algoritma Pemrograman 1
8 CAK1BAB3 Algoritma Pemrograman 1
...
...
1000 CAK1BAB3 Algoritma Pemrograman 1
C:\users\go\src\hello>
```

5.4 Contoh Soal Modul 5

1) Buatlah program untuk menampilkan baris bilangan dari a sampai dengan b.

   Masukan terdiri dari dua bilangan bulat a dan b, yang mana $a \leq b$.

   Keluaran berupa baris bilangan dari a sampai dengan b.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	2 5	2 3 4 5
2	6 6	6
3	-5 7	-5 -4 -3 -2 -1 0 1 2 3 4 5 6 7

Jawaban

```
1   package main
2   import "fmt"
3
4   func main() {
5      var a, b int
6      var j int
7      fmt.Scan(&a, &b)
8      for j = a; j <=b; j+=1 {
9         fmt.Print(j," ")
10     }
11  }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
2 5
2 3 4 5
E:\DEV\NATIVE\GO>.\Demo_Soal
6 6
6
E:\DEV\NATIVE\GO>.\Demo_Soal
-5 7
-5 -4 -3 -2 -1 0 1 2 3 4 5 6 7
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

2) Buatlah program yang digunakan untuk menampilkan sejumlah n luas segitiga, apabila diketahui sisi alas dan tinggi dari masing-masing segitiga.

Masukan terdiri dari $n+1$ baris. Baris pertama merupakan suatu bilangan bulat n, selanjutnya n baris berikutnya masing-masing merupakan panjang sisi alas dan tinggi dari segitiga

Keluaran terdiri dari n baris, yang masing-masing menyatakan luas dari segitiga.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5 11 2 32 14 6 2 15 15	11 224 6 112.5 350

	20 35	
2	3 12 32 231 234 43 34	192 27027 731

Jawaban

```
1   package main
2   import "fmt"
3
4   func main() {
5      var j, alas, tinggi, n int
6      var luas float64
7      fmt.Scan(&n)
8      for j = 1; j <=n; j+=1 {
9         fmt.Scan(&alas, &tinggi)
10        luas = 0.5 * float64(alas * tinggi)
11        fmt.Println(luas)
12     }
13  }
14
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
5
11 2
11
32 14
224
6 2
6
15 15
112.5
20 35
350

E:\DEV\NATIVE\GO>.\Demo_Soal
3
12 32
192
231 234
27027
43 34
731
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program yang digunakan untuk menghitung hasil perkalian dua buah bilangan tanpa menggunakan operator kali "*".
Masukan terdiri dari dua bilangan bulat positif.

Keluaran terdiri dari sebuah bilangan yang menyatakan hasil perkalian dari dua bilangan pada masukan.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	2 100	200
2	7 6	42

Jawaban

```
1   package main
2   import "fmt"
3
4   func main(){
5      var j, v1, v2 int
6      var hasil int
7      fmt.Scan(&v1, &v2)
8      hasil = 0
9      for j = 1; j <= v2; j+=1 {
10        hasil = hasil + v1
11     }
12     fmt.Println(hasil)
13  }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
2 10
20
E:\DEV\NATIVE\GO>.\Demo_Soal
7 6
42
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

5.5 Soal Latihan Modul 5

1) Buatlah program untuk menjumlahkan sekumpulan bilangan.

   Masukan terdiri dari suatu bilangan bulat positif n.

   Keluaran berupa bilangan hasil penjumlahan dari 1 sampai dengan n.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	3	6
2	1	1

3	5	15

2) Buatlah program yang digunakan untuk menghitung volume sejumlah n kerucut, apabila diketahui panjang jari-jari alas kerucut dan tinggi dari kerucut.

   Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat n, selanjutnya n baris berikutnya masing-masing merupakan panjang jari-jari alas kerucut dan tinggi dari kerucut.

   Keluaran terdiri dari beberapa baris, yang masing-masingnya menyatakan volume dari n kerucut.

   Contoh masukan dan keluaran

No	Masukan	Keluaran
1	1 3 4	37.69911184307752
2	3 1 1 2 2 3 3	1.0471975511965976 8.377580409572781 28.274333882308138

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program yang digunakan untuk menghitung hasil pemangkatan dari dua buah bilangan. Program dibuat dengan menggunakan operator perkalian dan struktur kontrol perulangan.

   Masukan terdiri dari dua bilangan bulat positif.

   Keluaran terdiri dari suatu bilangan yang menyatakan hasil bilangan pertama dipangkatkan dengan bilangan kedua.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	4 2	16
2	2 10	1024
3	10 3	1000

4) Buatlah program yang digunakan untuk menghitung hasil faktorial dari suatu bilangan.

   Masukan terdiri dari suatu bilangan bulat non negatif.

   Keluaran terdiri dari hasil faktorial dari bilangan bulat n.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	0	1
2	1	1
3	5	120
4	10	3628800

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 7. ASESMEN PRAKTIKUM 1

7.1 Asesmen Praktikum 1

Asesmen Praktikum 1 merupakan tes untuk materi dari modul ke-2 sampai dengan modul ke-7. Mahasiswa akan mengerjakan soal tes secara mandiri di laboratorium.

Materi meliputi penggunaan:

1) Input/output.
2) Tipe data dan operasinya.
3) Sekuensial dalam penulisan proses.
4) Struktur kontrol perulangan dengan For-Loop.

7.2 Contoh Soal Asesmen Praktikum 1

1) [**Penjumlahan**] Buatlah sebuah program yang digunakan untuk menjumlahkan bilangan dari x sampai dengan y, yang mana nilai x selalu lebih kecil atau sama dengan y.

   Masukan terdiri dari dua buah bilangan bulat positif x dan y (x <= y).

   Keluaran berupa hasil penjumlahan semua bilangan dari x sampai dengan y (inklusif)

   Contoh masukan dan keluaran:

Fakultas Informatika
School of Computing
Telkom University
informatics lab

No	Masukan	Keluaran
1	4 5	9
2	2 100	5049

   Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var x, y, j, jumlah int
5     fmt.Scan(&x, &y)
6     jumlah = 0
7     for j = x; j <= y; j += 1 {
8        jumlah = jumlah + j
9     }
10    fmt.Println(jumlah)
11 }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
4 5
9
E:\DEV\NATIVE\GO>.\Demo_Soal
2 100
5049
```

Jawaban versi 2 menggunakan rumus $S = \frac{n}{2} \times (x + y)$, yang mana n adalah banyaknya anggota deret, x adalah sukur pertama dan y adalah suku terakhir.

```
1  package main
2  import "fmt"
3  func main() {
4     var x, y, n int
5     var S float64
6     fmt.Scan(&x, &y)
7     n = (y - x) + 1
8     S = float64(n * (x + y)) / 2
9     fmt.Println(S)
10 }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
4 5
9
E:\DEV\NATIVE\GO>.\Demo_Soal
2 100
5049
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

2) [**Kuadrat**] Sebuah program digunakan untuk mencetak kuadrat dari bilangan 1 sampai dengan N.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran berupa baris bilangan, yang masing-masingnya adalah kuadrat dari bilangan 1 sampai dengan N.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	4	1 4 9 16
2	10	1 4 9 16 25 36 49 64 81 100

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var N, j int
5     fmt.Scan(&N)
6     for j = 1; j <= N; j += 1 {
7        fmt.Print(j*j, " ")
8     }
9  }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
```

```
4
1 4 9 16
E:\DEV\NATIVE\GO>.\Demo_Soal
10
1 4 9 16 25 36 49 64 81 100
```

3) [**Tukar Mata Uang**] Pada masa kekhalifahan Harun Al Rasyid dikenal 4 mata uang sebagai alat penukar barang sehari-hari, yaitu dinar, dirham, fals, dan qirat. Satu dinar terbuat dari emas sebanding dengan 10 dirham, satu dirham terbuat dari perak sebanding dengan 10 fals, dan satu fals terbuat dari tembaga sebanding dengan 6 qirat.

   Buatlah program untuk menghitung dengan cepat, berapa dinar, dirham, fals, dan qirat jika seseorang pada zaman itu ingin menukar uang dari satuan qirat.

   Masukan terdiri atas sebuah bilangan bulat yang menyatakan uang dalam satuan qirat.

   Keluaran terdiri atas 4 bilangan bulat yang menyatakan uang dalam satuan dinar, dirham, fals, dan qirat.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	1000	1 6 6 4
2	679	1 1 3 1
3	100	0 1 6 4

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var dinar, dirham, fals, qirat int
5     fmt.Scan(&qirat)
6     dinar = qirat / 600
7     dirham = qirat % 600 / 60
8     fals = qirat % 600 % 60 / 6
9     qirat = qirat % 600 % 60 % 6
10    fmt.Println(dinar, dirham, fals, qirat)
11 }
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
1000
1 6 6 4
E:\DEV\NATIVE\GO>.\Demo_Soal
679
1 1 3 1
E:\DEV\NATIVE\GO>.\Demo_Soal
100
0 1 6 4
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 9. IF-THEN

9.1 Paradigma Percabangan

Sebelumnya telah dipelajari bahwa setiap baris kode program akan dieksekusi satu persatu secara sekuensial. Artinya kode program dari baris ke-1 hingga baris terakhis akan dieksekusi satu persatu. Bagaimana jika kita ingin **baris kode program yang dieksekusi itu berdasarkan syarat atau suatu ketentuan** tertentu? Misalnya baris pertama dan terakhir saja yang dijalankan. Maka di dalam pemrograman hal tersebut mungkin untuk dilakukan, struktur kontrol yang digunakan adalah **if-then**.

9.2 Karakterisik If-Then

Penulisan struktur kontrol percabangan dengan menggunakan if-then pada dasarnya terdiri dari dua bagian, yaitu:

1) **Kondisi**, yaitu sesuatu syarat atau ketentuan dari suatu percabangan. **Kondisi** ini harus bernilai **boolean**, baik itu variabel ataupun operasi tipe data.
2) **Aksi**, yaitu kumpulan instruksi yang akan dilakukan apabila **kondisi terpenuhi atau bernilai true**. Artinya baris aksi tidak secara default dieksekusi, tetapi ada syarat yang harus terpenuhi terlebih dahulu sehingga aksi dapat dieksekusi.

Sebagai contoh ilustrasi penulisannya adalah sebagai berikut:

Fakultas Informatika
School of Computing
Informatics Lab

Tabel 9-1 Notasi Percabangan Tunggal (If-Then) pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go

if kondisi **then**
  // aksi
endfor
	if kondisi {
	  // aksi
	}

Penulisan aksi diberi tab atau 4 spasi (sama seperti), untuk mempermudah pembacaan bagian baris kode mana yang merupakan aksi dan mana yang tidak.

9.3 Implementasi menggunakan Go

Sebagai contoh, misalnya terdapat suatu program yang akan menampilkan hasil operasi pembagian (a/b). Hasil pembagian hanya akan ditampilkan apabila pembagi **tidaklah bernilai 0**.

```
1  // filename: ifthen1.go
2  package main
3  import "fmt"
4
5  func main() {
6    var a, b, hasil float64
7
```

```
8       fmt.Scan(&a, &b)
9       if b != 0 {
10          hasil = a / b
11          fmt.Println("Hasil pembagian adalah", hasil)
12      }
13      fmt.Println("Program selesai")
14  }
```

```
C:\users\go\src\hello>go build ifthen1.go
C:\users\go\src\hello>.\ifthen1
5 2
Hasil pembagian adalah 2.5
Program selesai
C:\users\go\src\hello>.\ifthen1
5 0
Program selesai
```

Baris ke-10 dan 11 hanya akan dieksekusi apabila kondisi pada baris ke-9 bernilai true. Pada contoh ketika nilai b adalah 2, maka program akan menampilkan hasil pembagian, sedangkan ketika b adalah 0, maka tidak ada hasil operasi yang ditampilkan.

9.4 Contoh Soal Modul 9

1) Sebuah program digunakan untuk menghitung nilai absolut atau mutlak dari suatu bilangan.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Masukan terdiri dari suatu bilangan bulat.

Keluaran berupa nilai absolut atau mutlak dari bilangan yang diberikan.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	10	10
2	-3	3
3	5	5
4	0	0
5	-9999	9999

Jawaban

```
1   package main
2   import "fmt"
3   func main() {
4     var bilangan int
5     fmt.Scan(&bilangan)
6     if bilangan < 0 {
7       bilangan = -bilangan
8     }
```

```
    fmt.Println(bilangan)
}
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
10
10
E:\DEV\NATIVE\GO>.\Demo_Soal
-3
3
E:\DEV\NATIVE\GO>.\Demo_Soal
5
5
E:\DEV\NATIVE\GO>.\Demo_Soal
-9999
9999
```

2) Buatlah program Go yang digunakan untuk menentukan apakah suatu bilangan yang diberikan adalah bilangan positif atau bukan.

Masukan terdiri dari suatu bilangan bulat.

Keluaran terdiri dari teks yang menyatakan bilangan bulat adalah "positif" atau "bukan positif".

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	10	positif
2	-3	bukan positif
3	5	positif
4	0	bukan positif

Jawaban

```
package main
import "fmt"
func main() {
    var bilangan int
    var teks string
    fmt.Scan(&bilangan)
    teks = "bukan positif"
```

```
    if bilangan > 0 {
        teks = "positif"
    }
    fmt.Println(teks)
}
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
10
positif
E:\DEV\NATIVE\GO>.\Demo_Soal
-3
bukan positif
E:\DEV\NATIVE\GO>.\Demo_Soal
5
positif
E:\DEV\NATIVE\GO>.\Demo_Soal
0
bukan positif
```

3) Buatlah suatu program dengan menggunakan bahasa pemrograman Go, yang mana program digunakan untuk menentukan apakah bilangan yang diberikan adalah bilangan genap negatif atau bukan.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Masukan terdiri dari suatu bilangan bulat.

Keluaran terdiri dari suatu boolean yang menyatakan true apabila bilangan yang diberikan adalah genap negatif, atau false apabila bukan genap negatif.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	10	false
2	-3	false
3	-4	true
4	0	false
5	-2	true

Jawaban:

```
package main
import "fmt"
func main() {
    var bilangan int
    var hasil bool
    fmt.Scan(&bilangan)
    hasil = bilangan % 2 == 0 && bilangan < 0
    fmt.Println(hasil)
}
```

```
E:\DEV\NATIVE\GO>go build Demo_Soal.go
E:\DEV\NATIVE\GO>.\Demo_Soal
10
false
E:\DEV\NATIVE\GO>.\Demo_Soal
-3
false
E:\DEV\NATIVE\GO>.\Demo_Soal
-4
true
E:\DEV\NATIVE\GO>.\Demo_Soal
0
false
E:\DEV\NATIVE\GO>.\Demo_Soal
-2
true
```

Catatan: Tidak semua soal yang terlihat seperti menggunakan percabangan perlu diselesaikan dengan percabangan. Pada soal ini, nilai true dan false dapat langsung dihasilkan dari operasi perbandingan dan logika.

9.5 Soal Latihan Modul 9

1) Sebuah program Go digunakan untuk menentukan jumlah motor yang diperlukan seseorang untuk melakukan touring. Satu motor hanya cukup untuk dua orang, yaitu satu pengemudi dan satu tumpangan. Setiap motor diperioritaskan untuk terisi dua orang.

   Masukan terdiri dari bilangan bulat yang menyatang jumlah orang yang akan melakukan touring.

   Keluaran berupa jumlah motor yang diperlukan peserta touring.

   Contoh masukan dan keluaran:

Fakultas Informatika
School of Computing
Telkom University
informatics lab

No	Masukan	Keluaran
1	10	5
2	1	1
3	25	13
4	9	5

2) Buatlah suatu program dengan menggunakan bahasa pemrograman Go, yang mana program digunakan untuk menentukan apakah bilangan yang diberikan adalah bilangan genap negatif atau bukan.

   Masukan terdiri dari suatu bilangan bulat.

   Keluaran terdiri dari suatu teks yang menyatakan bilangan adalah "genap negatif" atau "bukan".

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	10	bukan
2	-4	genap negatif
3	0	bukan

4	-2	genap negatif

3) Buatlah program yang digunakan untuk menentukan suatu bilangan adalah faktor dari bilangan yang lain. Suatu bilangan a adalah faktor dari b apabila bilangan a habis membagi bilangan b.

Masukan terdiri dari dua bilangan bulat positif x dan y.

Keluaran terdiri dari dua baris boolean, yang mana baris pertama adalah boolean yang menyatakan x adalah faktor dari y. Baris kedua adalah boolean yang menyatakan y adalah faktor dari x.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	10 5	false true
2	3 21	true false
3	4 4	true true

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 10. ELSE-IF

10.1 Paradigma Percabangan

Sebelumnya telah dipelajari bahwa setiap baris kode program akan dieksekusi satu persatu secara sekuensial. Artinya kode program dari baris ke-1 hingga baris terakhis akan dieksekusi satu persatu. Bagaimana jika kita ingin **baris kode program yang dieksekusi itu berdasarkan syarat atau suatu ketentuan** tertentu? Sebagai analogi misalnya ketika kita berada dipersimpangan jalan ke kiri atau ke kanan. Maka di dalam pemrograman hal tersebut mungkin untuk dilakukan, struktur kontrol yang digunakan adalah **else-if**.

10.2 Karakterisik Else-If

Pada dasarnya else-if tidak jauh berbeda dengan struktur kontrol percabangan menggunakan if-then. Perbedaannya terletak pada adanya **aksi lain** yang akan dieksekusi apabila kondisi tidak terjadi atau bernilai false.

Penulisan struktur kontrol percabangan dengan menggunakan else-if terdiri dari dua bagian, yaitu:

1) **Kondisi**, yaitu sesuatu syarat atau ketentuan dari suatu percabangan. **Kondisi** ini harus bernilai **boolean**, baik itu variabel ataupun operasi tipe data.
2) **Aksi**, yaitu kumpulan instruksi yang akan dilakukan apabila **kondisi terpenuhi atau bernilai true**.
3) **Aksi lain**, yaitu kumpulan instruksi yang akan dilakukan apabila **kondisi terpenuhi atau bernilai false.** Artinya aksi dan aksi lain ini merupakan pilihan, yang mana hanya salah satu aksi saja yang akan dieksekusi sesuai dengan nilai dari kondisi.

Fakultas Informatika
School of Computing
Telkom University
Informatics lab

Sebagai contoh ilustrasi penulisannya adalah sebagai berikut:

Tabel 10-1 Notasi Percabangan If-Else pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go

if kondisi **then**
  // aksi
else
  // aksi lain
endfor
	if kondisi {
	  // aksi
	}**else**{
	  // aksi lain
	}

Penulisan aksi diberi tab atau 4 spasi (sama seperti), untuk mempermudah pembacaan bagian baris kode mana yang merupakan aksi dan mana yang tidak.

Untuk penulisan dengan kondisi yang yang lebih banyak maka, notasi dapat ditulis ke dalam bentuk seperti berikut ini:

Tabel 10-2 Notasi Percabangan Bertingkat (Nested If-Else) pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go
if kondisi_1 **then**	**if** kondisi_1 {

// aksi 1
else
if kondisi_2 **then**
// aksi 2
else
if kondisi_3 **then**
// aksi 3
else
// aksi lain
endfor
endfor
endfor
	// aksi 1
	}**else**{
	if kondisi_2 {
	// aksi 2
	}**else**{
	if kondisi_3 {
	// aksi 3
	}**else**{
	// aksi lain
	}
	}
	}

Setiap aksi ke-i hanya akan dieksekusi apabila kondisi ke-i bernilai true atau benar, sedangkan aksi lain hanya akan dieksekusi apabila semua kondisi dari i = 1 s.d n bernilai false. Bentuk penulisan lain yang lebih sederhana adalah sebagai berikut ini:

Tabel 10-3 Notasi Percabangan Jamak (Else-If) pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go

if kondisi_1 **then**
// aksi 1
else if kondisi_2 **then**
// aksi 2
else if kondisi_3 **then**
// aksi 3
else
// aksi lain
endfor
	if kondisi_1 {
	// aksi 1
	}**else if** kondisi_2 {
	// aksi 2
	}**else if** kondisi_3 {
	// aksi 3
	}**else**{
	// aksi lain
	}

Fakultas Informatika
School of Computing
Telkom University
informatics lab

10.3 Implementasi menggunakan Go

Sebagai contoh, misalnya terdapat suatu program yang akan menampilkan hasil operasi pembagian (a/b). Hasil pembagian hanya akan ditampilkan apabila pembagi **tidaklah bernilai 0**. Apabila pembagi bernilai 0 maka akan ditampilkan suatu pesan "Variabel b bernilai nol".

```
1   // filename: elseif1.go
2   package main
3   import "fmt"
4
5   func main() {
6       var a, b, hasil float64
7
8       fmt.Scan(&a, &b)
9       if b != 0 {
10          hasil = a / b
11          fmt.Println("Hasil pembagian adalah", hasil)
12      }else{
13          fmt.Println("Variabel b bernilai nol")
```

```
    }
    fmt.Println("Program selesai")
}
```

```
C:\users\go\src\hello>go build elseif1.go
C:\users\go\src\hello>.\elseif1
5 2
Hasil pembagian adalah 2.5
Program selesai
C:\users\go\src\hello>.\elseif1
5 0
Variabel b bernilai nol
Program selesai
```

Pada contoh di atas terlihat bahwa, baris ke-13 hanya akan dieksekusi apabila kondisi di baris ke-9 bernilai false, artinya b bernilai 0.

Contoh konversi (nilai, tubes, kehadiran) menjadi indeks nilai.

```
if nilai > 75 && adaTubes {
   indeks = 'A'
} else if nilai > 65 {
   indeks = 'B'
} else if nilai > 50 && pctHadir > 0.7 {
   indeks = 'C'
} else {
   indeks = 'F'
}
fmt.Printf( “Nilai %v dengan kehadiran %v%% dan buat tubes=%v, mendapat indeks %c\n”, nilai, pctHadir,
adaTubes, indeks )
```

Telkom University informatics lab

10.4 Contoh Soal Modul 10

1) Buatlah program dengan bahasa Go yang digunakan untuk menentukan apakah seorang penduduk bisa membuat KTP atau tidak. Syarat utama membuat KTP adalah berusia minimal 17 tahun dan memiliki kartu keluarga.

   Masukan terdiri dari dua baris. Baris pertama adalah bilangan bulat yang menyatakan usia, sedangkan baris kedua adalah boolean yang menyatakan memiliki KTP atau tidak.

   Keluaran berupa teks "bisa membuat KTP" atau "belum bisa membuat KTP" sesuai dengan masukan yang diberikan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	17 true	bisa membuat KTP
2	20 false	belum bisa membuat KTP
3	15 true	belum bisa membuat KTP

Jawaban:

```
1  package main
2  import "fmt"
3  func main() {
4     var usia int
5     var kk bool
6     fmt.Scan(&usia, &kk)
7     if usia >= 17 && kk {
8        fmt.Println("bisa membuat KTP")
9     }else{
10       fmt.Println("belum bisa membuat KTP")
11    }
12 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
17
true
bisa membuat KTP
gppras@SR8 GO % ./Demo_Soal
20
false
belum bisa membuat KTP
gppras@SR8 GO % ./Demo_Soal
15
true
belum bisa membuat KTP
```

Fakultas Informatika
School of Computing
Telkom University
Informatika Ide

2) Buatlah suatu program Go yang digunakan untuk menentukan suatu alfabet yang diberikan adalah vokal atau konsonan.

Masukan terdiri dari suatu huruf.

Keluaran berupa teks yang menyatakan huruf adalah adalah "vokal" atau "konsonan" atau "bukan huruf".

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	A	vokal
2	f	konsonan
3	1	bukan huruf
4	$	bukan huruf

Jawaban:

```go
package main
import "fmt"
func main() {
   var x rune
   var huruf, vKecil, vBesar bool
   fmt.Scanf("%c", &x)
   huruf = (x >= 'a' && x <= 'z') || (x >= 'A' && x <= 'Z')
   vKecil = x == 'a' || x == 'i' || x == 'u' || x == 'e' || x == 'o'
   vBesar = x == 'A' || x == 'I' || x == 'U' || x == 'E' || x == 'O'
   if huruf && (vKecil || vBesar) {
      fmt.Println("vokal")
   }else if huruf && !(vKecil || vBesar){
      fmt.Println("konsonan")
   }else{
      fmt.Println("bukan huruf")
   }
}
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
A
vokal
gppras@SR8 GO % ./Demo_Soal
f
konsonan
gppras@SR8 GO % ./Demo_Soal
1
bukan huruf
gppras@SR8 GO % ./Demo_Soal
$
bukan huruf
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program untuk menentukan apakah digit dalam suatu bilangan terurut membesar, mengecil, atau tidak terurut. Bilangan hanya terdiri dari empat digit saja, atau lebih besar atau sama dengan 1000 dan kecil sama atau sama dengan 9999.

   Perhatikan contoh sesi interaksi program berikut (**teks bergaris bawah** adalah input/read):

No.	Contoh masukan dan keluaran
1	Bilangan: **2489** Digit pada bilangan 2489 terurut membesar
2	Bilangan: **3861** Digit pada bilangan 3861 tidak terurut
3	Bilangan: **9651** Digit pada bilangan 9651 terurut mengecil

Catatan: Asumsi digit pada bilangan tidak ada yang sama.

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var bilangan, d1, d2, d3, d4 int
5     var teks string
6     fmt.Print("Bilangan: ")
7     fmt.Scan(&bilangan)
8     d4 = bilangan % 10
9     d3 = (bilangan % 100) / 10
10    d2 = (bilangan % 1000) / 10
11    d1 = bilangan / 1000
12    if d1 < d2 && d2 < d3 && d3 < d4 {
13       teks = "terurut membesar"
14    }else if d1 > d2 && d2 > d3 && d3 > d4{
15       teks = "terurut mengecil"
16    }else{
17       teks = "tidak terurut"
18    }
19    fmt.Println("Digit pada bilangan", bilangan, teks)
20 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
Bilangan: 2489
Digit pada bilangan 2489 tidak terurut
gppras@SR8 GO % ./Demo_Soal
Bilangan: 3861
Digit pada bilangan 3861 tidak terurut
gppras@SR8 GO % ./Demo_Soal
Bilangan: 9651
Digit pada bilangan 9651 tidak terurut
gppras@SR8 GO % ./Demo_Soal
Bilangan: 5555
Digit pada bilangan 5555 tidak terurut
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

10.5 Soal Latihan Modul 10

1) PT POS membutuhkan aplikasi perhitungan biaya kirim berdasarkan berat parsel. Maka, **buatlah program BiayaPos untuk menghitung biaya pengiriman tersebut dengan ketentuan sebagai berikut!**

   Dari berat parsel (dalam gram), harus dihitung total berat dalam kg dan sisanya (dalam gram). Biaya jasa pengiriman adalah Rp. 10.000,- per kg. Jika sisa berat tidak kurang dari 500 gram, maka tambahan biaya kirim hanya Rp. 5,- per gram saja. Tetapi jika kurang dari 500 gram, maka tambahan biaya akan dibebankan sebesar Rp. 15,- per gram. Sisa berat (yang kurang dari 1kg) digratiskan biayanya apabila total berat ternyata lebih dari 10kg.

   Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

No.	Contoh masukan dan keluaran

1	Berat parsel (gram): **8500** Detail berat: 8 kg + 500 gr Detail biaya: Rp. 80000 + Rp. 2500 Total biaya: Rp. 82500
2	Berat parsel (gram): **9250** Detail berat: 9 kg + 250 gr Detail biaya: Rp. 90000 + Rp. 3750 Total biaya: Rp. 93750
3	Berat parsel (gram): **11750** Detail berat: 11 kg + 750 gr Detail biaya: Rp. 110000 + Rp. 3750 Total biaya: Rp. 110000

2) Diberikan sebuah nilai akhir mata kuliah (NAM) [0..100] dan standar penilaian nilai mata kuliah (NMK) sebagai berikut:

NAM	NMK
NAM>80	A
72.5 < NAM <= 80	AB
65 < NAM <= 72.5	B
57.5 < NAM <= 65	BC
50 < NAM <= 57.5	C
40 < NAM <= 50	D
NAM <=40	E

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Program berikut menerima input sebuah bilangan riil yang menyatakan NAM. Program menghitung NMK dan menampilkannya.

```
1   package main
2   import "fmt"
3   func main() {
4     var nam float64
5     var nmk string
6     fmt.Print("Nilai akhir mata kuliah: ")
7     fmt.Scan(&nam)
8     if nam > 80 {
9       nam = "A"
10    }
11    if nam > 72.5 {
12      nam = "AB"
13    }
14    if nam > 65 {
15      nam = "B"
16    }
17    if nam > 57.5 {
18      nam = "BC"
19    }
20    if nam > 50 {
21      nam = "C"
22    }
```

```
23      if nam > 40 {
24          nam = "D"
25      } else if nam <= 40 {
26          nam = "E"
27      }
28      fmt.Println("Nilai mata kuliah: ", nmk)
29  }
```

Jawablah pertanyaan-pertanyaan berikut:

a. Jika **nam** diberikan adalah 80.1, apa keluaran dari program tersebut? Apakah eksekusi program tersebut sesuai spesifikasi soal?

b. Apa saja kesalahan dari program tersebut? Mengapa demikian? Jelaskan alur program seharusnya!

c. Perbaiki program tersebut! Ujilah dengan masukan: 93.5; 70.6; dan 49.5. Seharusnya keluaran yang diperoleh adalah 'A', 'B', dan 'D'.

3) Sebuah bilangan bulat **b** memiliki faktor bilangan **f** > 0 jika **f** habis membagi **b**. Contoh: 2 merupakan faktor dari bilangan 6 karena 6 habis dibagi 2.

Buatlah program yang menerima input sebuah bilangan bulat **b** dan **b** > 1. Program harus dapat mencari dan menampilkan semua faktor dari bilangan tersebut!

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

Bilangan: **12** Faktor: 1 2 3 4 6 12	Bilangan: **7** Faktor: 1 7

Fakultas Informatika
Telkom University
informatics lab

Bilangan bulat **b** > 0 merupakan bilangan prima **p** jika dan hanya jika memiliki persis dua faktor bilangan saja, yaitu 1 dan dirinya sendiri.

Lanjutkan program sebelumnya. Setelah menerima masukan sebuah bilangan bulat **b** > 0. Program tersebut mencari dan menampilkan semua faktor bilangan tersebut. Kemudian, program menentukan apakah **b** merupakan bilangan prima.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

Bilangan: **12** Faktor: 1 2 3 4 6 12 Prima: false	Bilangan: **7** Faktor: 1 7 Prima: true

MODUL 11. SWITCH-CASE

11.1 Karakterisik Switch-Case

Pada dasarnya switch-case merupakan alternatif dari penggunaan else-if, tetapi pemilihan aksi tidak dilakukan berdasarkan kondisi, tetapi suatu nilai atau operasi tipe data dasar yang tidak menghasilkan boolean.

Penulisan switch case terdiri dari komponen berikut:

a. **Ekspresi**, merupakan operasi tipe data yang menghasilkan **nilai** selain tipe data boolean,

b. **Nilai**, pilihan case dari **ekspresi** yang dideklarasikan pada instruksi switch-case.

c. **Aksi**, merupakan kumpulan instruksi yang akan dieksekusi sesuai dengan **nilai** yang dihasilkan dari **ekspresi** yang terdapat pada switch-case. **Aksi lain** hanya akan dieksekusi apabila tidak ada pilihan nilai yang sesuai dengan hasil ekspresi.

Berikut adalah notasi dari switch-case

Tabel 11-1 Notasi Pemilihan Switch-Case pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go

depend on expresi
nilai_1:
  // aksi 1
nilai_2:
  // aksi 2
nilai 3:
  // aksi 3
nilai 4:
  // aksi 4
.. dst.
  // dst.
end
	switch ekspresi {
	case nilai_1:
	  // aksi 1
	case nilai_2:
	  // aksi 2
	case nilai_3:
	  // aksi 3
	case nilai_4:
	  // aksi 4
	default:
	  // aksi lain
	}

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Kata kunci **default** digunakan untuk menyatakan pilihan untuk **aksi lain**. Selain itu terdapat juga penggunaan switch-case sebagai pengganti if-then, yang mana pada setiap case yang didefinisikan adalah kondisi dari struktur kontrol percabangan,

```
switch {
  case kondisi_1 : // aksi kondisi 1
  case kondisi_2 : // aksi kondisi 2
  case kondisi_2 : // aksi kondisi 3
  default : // aksi lain
}
```

11.2 Implementasi menggunakan Go

Sebagai contoh sederhana adalah program yang digunakan untuk menentukan batas atas dan bawah suatu indeks nilai mutu.

```
switch indeks {
case 'A':
   batasA = 100
   batasB = 75
case 'B':
   batasA = 75
   batasB = 65
case 'C':
   batasA = 65
   batasB = 50
default:
   batasA = 50
   batasB = 0
}
fmt.Printf( “Rentang nilai %v adalah: %v..%v\n”, indeks, batasB, batasA )
```

```
switch {
case nilai > 75 && adaTubes:
   indeks = 'A'
case nilai > 65:
   indeks = 'B'
case nilai > 50 && pctHadir > 0.7:
   indeks = 'C'
default:
   indeks = 'F'
}
fmt.Printf( “Nilai %v dengan kehadiran %v%% dan buat tubes=%v, mendapat indeks %c\n”, nilai, pctHadir, adaTubes, indeks )
```

Telkom University
informatics lab

Contoh yang lain, misalnya suatu program digunakan untuk menampilkan hasil operasi aritmatika berdasarkan kriteria tertentu.

```
// filename: switchcase1.go
package main
import "fmt"

func main() {
   var a int
   var b int
   fmt.Scan(&a, &b)
   switch (a + b) % 4  {
   case 0:
      fmt.Println("Hasil dari", a, "+", b, "% 4 adalah", (a + b) % 4)
      fmt.Println("Operasi: Penjumlahan")
      fmt.Println("Hasil:", a, "+", b, "=", a+b)
   case 1:
      fmt.Println("Hasil dari", a, "+", b, "% 4 adalah", (a + b) % 4)
      fmt.Println("Operasi: Pengurangan")
      fmt.Println("Hasil:", a, "-", b, "=", a-b)
   case 2:
      fmt.Println("Hasil dari", a, "+", b, "% 4 adalah", (a + b) % 4)
      fmt.Println("Operasi: Perkalian")
      fmt.Println("Hasil:", a, "*", b, "=", a*b)
   case 3:
```

```
23          fmt.Println("Hasil dari", a, "+", b, "% 4 adalah", (a + b) % 4)
24          fmt.Println("Operasi: Pembagian")
25          if b != 0 {
26              fmt.Println("Hasil:", a, "/", b, "=", a/b)
27          } else {
28              fmt.Println("Error: Pembagi tidak boleh nol.")
29          }
30      default:
31          fmt.Println("Error: Pilihan tidak valid.")
32      }
33  }
```

```
C:\users\go\src\hello>go build switchcase1.go
C:\users\go\src\hello>switchcase1
10 6
Hasil dari 10 + 6 % 4 adalah 0
Operasi: Penjumlahan
Hasil: 10 + 6 = 16
C:\users\go\src\hello>switchcase1
7 4u
Hasil dari 7 + 4 % 4 adalah 3
Operasi: Pembagian
Hasil: 7 / 4 = 1
C:\users\go\src\hello>switchcase1
3 0
Hasil dari 3 + 0 % 4 adalah 3
Operasi: Pembagian
Error: Pembagi tidak boleh nol.
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

11.3 Contoh Soal Modul 11

1) Sebuah algoritma digunakan untuk melakukan konversi waktu dari bentuk 24 jam ke dalam bentuk 12 jam.

   Masukan terdiri dari satu bilangan bulat dengan interval 0 s.d. 23 (inklusif) yang merepresentasikan jam dalam bentuk 24 jam.

   Keluaran terdiri dari teks yang menyatakan format jam pada masukan dalam bentuk 12 jam.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	13	1 PM
2	0	12 AM
3	12	12 PM

   Jawaban:

```go
package main
import "fmt"
func main() {
   var jam12, jam24 int
   var label string
   fmt.Scan(&jam24)
   switch {
   case jam24 == 0:
      jam12 = 12
      label = "AM"
   case jam24 < 12:
      jam12 = jam24
      label = "AM"
   case jam24 == 12:
      jam12 = 12
      label = "PM"
   case jam24 > 12:
      jam12 = jam24 - 12
      label = "PM"
   }
   fmt.Println(jam12, label)
}
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
13
1 PM
gppras@SR8 GO % ./Demo_Soal
0
12 AM
gppras@SR8 GO % ./Demo_Soal
12
12 PM
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

2) Buatlah program dengan bahasa Go yang digunakan untuk menentukan apakah tanaman tersebut termasuk tanaman karnivora atau tidak. Jika ya, apakah tanaman tersebut asli indonesia atau tidak.

   Masukan terdiri dari satu string, yakni nama tanaman tersebut.

   Keluaran berupa teks "Termasuk T

   anaman Karnivora" atau "Tidak termasuk Tanaman Karnivora", dan dilanjutkan “Asli Indonesia” atau “Bukan Asli Indonesia”. Keluaran sesuai dengan masukan yang diberikan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	nepenthes	Termasuk Tanaman Karnivora Asli Indonesia
2	venus	Termasuk Tanaman Karnivora

		Bukan Asli Indonesia
3	karedok	Tidak termasuk Tanaman Karnivora

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var nama_tanaman string
5     fmt.Scan(&nama_tanaman)
6     switch nama_tanaman {
7        case "nepenthes", "drosera":
8           fmt.Println("Termasuk Tanaman Karnivora.")
9           fmt.Println("Asli Indonesia.")
10       case "venus", "sarracenia":
11          fmt.Println("Termasuk Tanaman Karnivora.")
12          fmt.Println("Tidak Asli Indonesia.")
13       default:
14          fmt.Println("Tidak termasuk Tanaman Karnivora.")
15    }
16 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
nepenthes
Termasuk Tanaman Karnivora.
Asli Indonesia.
gppras@SR8 GO % ./Demo_Soal
venus
Termasuk Tanaman Karnivora.
Tidak Asli Indonesia.
gppras@SR8 GO % ./Demo_Soal
karedok
Tidak termasuk Tanaman Karnivora.
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program dalam bahasa Go yang menentukan tarif parkir berdasarkan jenis kendaraan dan durasi parkir. Terdapat tiga jenis kendaraan yang dapat diparkir dengan tarif berbeda berdasarkan durasi parkir. Untuk motor, tarifnya adalah Rp 7.000 untuk durasi 1-2 jam, dan Rp 9.000 jika durasi lebih dari 2 jam. Mobil dikenakan tarif Rp 15.000 untuk 1-2 jam, dan Rp 20.000 untuk durasi lebih dari 2 jam. Sementara itu, truk dikenakan tarif Rp 25.000 untuk 1-2 jam, dan Rp 35.000 jika lebih dari 2 jam.

   Masukan terdiri dari 2 baris, baris pertama adalah string yang menyatakan jenis kendaraan sedangkan baris kedua merupakan bilangan bulat yang menyatakan durasi parkir dalam jam.

   Keluaran berupa string yang menampilkan tarif parkir berdasarkan jenis kendaraan dan durasi parkir kendaraan atau "Jenis kendaraan atau durasi parkir tidak valid" jika jenis kendaraan selain dari motor, mobil dan truk.

   Contoh masukan dan keluaran (<u>Teks bergaris bawah</u> adalah input/read):

No	Masukan	Keluaran
1	Masukkan jenis kendaraan (Motor/Mobil/Truk): **Motor** Masukkan durasi parkir (dalam jam): **2** Tarif Parkir: Rp 7000	
2	Masukkan jenis kendaraan (Motor/Mobil/Truk): **Mobil** Masukkan durasi parkir (dalam jam): **4** Tarif Parkir: Rp 20000	
3	Masukkan jenis kendaraan (Motor/Mobil/Truk): **Motor** Masukkan durasi parkir (dalam jam): **3** Tarif Parkir: Rp 9000	
4	Masukkan jenis kendaraan (Motor/Mobil/Truk): **Truk** Masukkan durasi parkir (dalam jam): **1** Tarif Parkir: Rp 25000	
5	Masukkan jenis kendaraan (Motor/Mobil/Truk): **Sepeda** Masukkan durasi parkir (dalam jam): **2** Jenis kendaraan atau durasi parkir tidak valid Tarif Parkir: Rp 0	

Jawaban:

```
1  package main
2  import "fmt"
3  func main() {
4    var kendaraan string
5    var durasi int
6    var tarif int
7    fmt.Print("Masukkan jenis kendaraan (Motor/Mobil/Truk): ")
8    fmt.Scan(&kendaraan)
9    fmt.Print("Masukkan durasi parkir (dalam jam): ")
10   fmt.Scan(&durasi)
11   switch {
12   case kendaraan == "Motor" && durasi >= 1 && durasi <= 2:
13     tarif = 7000
14   case kendaraan == "Motor" && durasi > 2:
15     tarif = 9000
16   case kendaraan == "Mobil" && durasi >= 1 && durasi <= 2:
17     tarif = 15000
18   case kendaraan == "Mobil" && durasi > 2:
19     tarif = 20000
20   case kendaraan == "Truk" && durasi >= 1 && durasi <= 2:
21     tarif = 25000
22   case kendaraan == "Truk" && durasi > 2:
23     tarif = 35000
24   default:
25     fmt.Println("Jenis kendaraan atau durasi parkir tidak valid")
26   }
27   fmt.Printf("Tarif Parkir: Rp %d\n", tarif)
28 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
Masukkan jenis kendaraan (Motor/Mobil/Truk): Motor
Masukkan durasi parkir (dalam jam): 2
Tarif Parkir: Rp 7000
gppras@SR8 GO % ./Demo_Soal
Masukkan jenis kendaraan (Motor/Mobil/Truk): Mobil
Masukkan durasi parkir (dalam jam): 4
Tarif Parkir: Rp 20000
gppras@SR8 GO % ./Demo_Soal
Masukkan jenis kendaraan (Motor/Mobil/Truk): Motor
Masukkan durasi parkir (dalam jam): 3
Tarif Parkir: Rp 9000
gppras@SR8 GO % ./Demo_Soal
Masukkan jenis kendaraan (Motor/Mobil/Truk): Truk
Masukkan durasi parkir (dalam jam): 1
Tarif Parkir: Rp 25000
gppras@SR8 GO % ./Demo_Soal
Masukkan jenis kendaraan (Motor/Mobil/Truk): Sepeda
Masukkan durasi parkir (dalam jam): 2
Jenis kendaraan atau durasi parkir tidak valid
Tarif Parkir: Rp 0
```

11.4 Soal Latihan Modul 11

Fakultas Informatika
School of Computing
Telkom University
informatics lab

1) Buatlah program dengan bahasa Go yang digunakan untuk menentukan apakah kadar pH pada air yang diinput termasuk Air yang layak untuk diminum atau tidak.

   Masukan terdiri dari satu float, yakni kadar pH tersebut.

   Keluaran berupa teks seperti contoh "Air Layak Minum" (ph >= 6.5 dan ph <= 8.6) atau "Air Tidak Layak Minum" (ph < 6.5 dan ph > 8.6), dan “Input tidak valid, rentang pH 0 - 14” jika nilai float diatas 14. Keluaran sesuai dengan masukan yang diberikan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	8.6	Air layak minum
2	9	Air tidak layak minum
3	16	Nilai pH tidak valid. Nilai pH harus antara 0 dan 14.

2) Buatlah program dalam bahasa Go untuk menghitung tarif parkir berdasarkan jenis kendaraan dan durasi parkir yang dimasukkan oleh pengguna. Ada tiga jenis kendaraan: motor, mobil, dan truk, dengan masing-masing memiliki tarif parkir yang berbeda. Motor dikenakan tarif sebesar Rp 2.000 per jam, mobil sebesar Rp 5.000 per jam, dan truk sebesar Rp 8.000 per jam.
   Program harus dapat menentukan tarif per jam berdasarkan jenis kendaraan, lalu mengalikan tarif tersebut dengan jumlah jam parkir untuk menghasilkan total biaya parkir. Jika durasi parkir kurang dari 1 jam, maka durasi tetap dianggap 1 jam.

   Masukan berupa jenis kendaraan dan durasi parkir dalam jam.
   Keluaran berupa total biaya parkir berdasarkan jenis kendaraan dan durasi parkir.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	motor 3 jam	Rp 6000
2	mobil 1 jam	Rp 5000
3	truk 5 jam	Rp 40000

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program dengan bahasa Go yang digunakan untuk mengidentifikasi pola aritmatika berdasarkan bilangan yang diinputkan dan melakukan operasi matematika yang sesuai. Beberapa ketentuan kategori diantaranya:

   a. Bilangan Ganjil : Menghitung penjumlahan antara bilangan yang diinput dengan bilangan berikutnya

   b. Bilangan Genap : Menghitung perkalian antara bilangan yang diinput dengan bilangan berikutnya

   c. Bilangan Kelipatan 5 : Menghitung hasil kuadrat dari bilangan yang diinputkan

   d. Bilangan Kelipatan 10 : Membagi bilangan yang diinputkan dengan bilangan 10

   Masukan terdiri dari satu bilangan bulat.

   Keluaran berupa teks "Kategori: Bilangan Ganjil" dilanjutkan dengan "Hasil penjumlahan dengan bilangan berikutnya %(input) + %(input+1) = %(hasil)" untuk Bilangan Ganjil. "Kategori: Bilangan Genap" dilanjutkan dengan "Hasil perkalianan dengan bilangan berikutnya %(input) * %(input+1) = %(hasil)" untuk Bilangan Genap. "Kategori: Bilangan Kelipatan 5" dilanjutkan dengan "Hasil kuadrat dari %(input) ^2 = %(hasil)" untuk Bilangan Kelipatan 5. "Kategori: Bilangan Kelipatan 10" dilanjutkan dengan "Hasil pembagian antara %(input) / 10 = %(hasil)" untuk Bilangan Kelipatan 10. Keluaran sesuai dengan masukan yang diberikan.

   Contoh masukan dan keluaran:

Fakultas Informatika
School of Computing
Telkom University
informatics lab

No	Masukan	Keluaran
1	5	Kategori: Bilangan Ganjil Hasil penjumlahan dengan bilangan berikutnya 5 + 6 = 11
2	8	Kategori: Bilangan Genap Hasil perkalian dengan bilangan berikutnya 8 * 9 = 72
3	25	Kategori: Bilangan Kelipatan 5 Hasil kuadrat dari 25 ^2 = 625
4	20	Kategori: Bilangan Kelipatan 10 Hasil pembagian antara 20 / 10 = 2

MODUL 12. WHILE-LOOP

12.1 Paradigma Perulangan

Perulangan merupakan salah satu struktur kontrol yang memungkinkan suatu instruksi yang sama dilakukan berulang kali dalam waktu atau jumlah yang lama. Tanpa instruksi perulangan, maka suatu instruksi akan ditulis dalam jumlah yang sangat banyak. Sebelumnya pada modul ke-5 dan 6 telah dipelajari instruksi perulangan dengan for-loop. Instruksi for-loop memungkinkan kita melakukan berulangan sebanyak *n* iterasi, akan tetapi pada banyak kasus yang melibatkan perulangan, **tidak semua perulangan diketahui jumlah iterasinya di awal**. Perulangan seperti ini disebut juga dengan istilah **perulangan dengan kondisi**.

Contoh perulangan jenis ini di kehidupan dunia nyata adalah sebagia berikut:

- *"Menulis teks tertentu selama tinta pena masih ada"*.
  Terdapat kondisi "tinta pena masih ada" sebagai syarat perulangan.
- *"Saya makan suap demi suap selama saya masih lapar".*
  Terdapat kondisi "saya masih lapar" sebagai syarat perulangan.

Penting! Pastikan bahwa instruksi perulangan yang digunakan pasti bisa membuat proses perulangan berhenti, apabila tidak maka program akan terus berjalan mengulangi instruksi tanpa akan pernah berhenti.

Terdapat dua bentuk perulangan dengan kondisi ini, yaitu **while-loop** dan **repeat-until**. Masing-masing kita akan bahas di modul yang berbeda.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

12.2 Karakterisik While-Loop

Struktur kontrol perulangan menggunakan while-loop memiliki bentuk yang hampir serupa dengan penulisan if-then pada percabangan, yaitu memiliki **kondisi** dan **aksi**. Hal yang membedakan adalah aksi akan dilakukan secara berulang-ulang selama kondisi bernilai true.

1) **Kondisi**, merupakan nilai atau operasi tipe data yang menghasilkan tipe data boolean. Kondisi ini merupakan syarat terjadinya perulangan. Artinya perulangan terjadi apabila kondisi bernilai true.
2) **Aksi**, merupakan kumpulan instruksi yang akan dieksekusi secara berulang-ulang selama kondisi bernilai true. *Salah satu instruksi dari aksi harus bisa membuat kondisi yang awalnya bernilai true manjadi false, tujuannya adalah untuk membuat perulangan berhenti.*

Pada penulisan notasinya secara umum bahasa pemrograman menggunakan kata kunci **while**, tetapi khusus di bahasa pemrograman Go, kata kunci yang digunakan adalah **for**. Walaupun berbeda dari kata kunci yang digunakan, **secara struktur penulisannya tetap sama**, sehingga tetap mudah untuk membedakan instruksi **for** yang digunakan adalah **for-loop** atau **while-loop**.

Berikut adalah notasi dari while-loop:

Tabel 12-1 Notasi Perulangan While-Loop pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go
while kondisi **do**	**for** kondisi {

// aksi **endwhile**	// aksi }

12.3 Implementasi menggunakan Go

Contoh persoalan yang melibatkan perulangan dengan kondisi ini adalah:

Program yang akan menampilkan hasil penjumlahan dua bilangan yang diterima dari masukan pengguna secara terus menerus. Proses ini akan berlangsung terus selama hasil penjumlahan adalah genap.

```
1   // filename: whileloop1.go
2   package main
3   import "fmt"
4
5   func main() {
6      var a int
7      var b int
8      fmt.Scan(&a, &b)
9      for (a + b) % 2 == 0 {
10        fmt.Println("Hasil penjumlahan",a+b)
11        fmt.Scan(&a, &b)
12     }
13     fmt.Println("Program selesai")
14  }
```

```
C:\users\go\src\hello>go build whileloop1.go
C:\users\go\src\hello>whileloop1
10 6
Hasil penjumlahan 16
5 5
Hasil penjumlahan 10
3 9
Hasil penjumlahan 12
9 2
Program selesai
C:\users\go\src\hello>whileloop1
4 5
Program selesai
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Pada contoh di atas, instruksi di baris ke-10 dan 11 akan berhenti dieksekusi berulang ketika kondisi di baris ke-9 bernilai false, yaitu ketika a dan b bernilai 9 dan 2 (9 + 2 = 11, yaitu ganjil).

Contoh penggunaan bentuk **while-loop** untuk menghitung $y = \sqrt{x}$ adalah sebagai berikut:

Tabel 12-2 Contoh Implementasi While-Loop untuk Menghitung Akar Bilangan dalam Algoritma dan Bahasa Go

	Notasi algoritma	Penulisan dalam bahasa Go
1	e <- 0.0000001	e = 0.0000001
2	x <- 2.0	x = 2.0
3	y <- 0.0	y = 0.0
4	y1 <- x	y1 = x
5	**while** y1-y > e **or** y1-y < -e **do**	**for** y1-y > e || y1-y < -e {
6	y <- y1	y = y1

7	y1 <- 0.5*y + 0.5*(x/y)	y1 = 0.5*y + 0.5*(x/y)
8	**endwhile**	}
9	**output**("sqrt(",x,")=",y)	fmt.**Printf**("sqrt(%v)=%v\n", x, y)

12.4 Contoh Soal Modul 12

1) Buatlah program yang digunakan untuk menampilkan deret bilangan Faktorial dari sautu bilangan.

   Masukan terdiri dari sebuah bilangan bulat non negatif n.

   Keluaran berupa deret bilangan dari Faktorial n. Perhatikan contoh masukan dan keluaran yang diberikan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	0	1
2	5	5 x 4 x 3 x 2 x 1
3	10	10 x 9 x 8 x 7 x 6 x 5 x 4 x 3 x 2 x 1
4	1	1

Fakultas Informatika

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var n, j int
5     fmt.Scan(&n)
6     j = n
7     for j > 1 {
8        fmt.Print(j, " x ")
9        j = j - 1
10    }
11    fmt.Println(1)
12 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
0
1
gppras@SR8 GO % ./Demo_Soal
5
5 x 4 x 3 x 2 x 1
gppras@SR8 GO % ./Demo_Soal
10
10 x 9 x 8 x 7 x 6 x 5 x 4 x 3 x 2 x 1
gppras@SR8 GO % ./Demo_Soal
1
1
```

2) Buatlah program Go yang digunakan untuk login ke dalam suatu aplikasi. Asumsi token untuk yang valid adalah "12345abcde".

Masukan terdiri dari suatu token. Selama token yang diberikan salah, maka program akan meminta token secara terus menerus hingga token yang diberikan benar.

Keluaran adalah teks yang menyatakan "Selamat Anda berhasil login".

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	Qwe12312 231234 13213 123lijwe 12345abcde	Selamat Anda berhasil login
2	12345abcde	Selamat Anda berhasil login

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var token string
5     fmt.Scan(&token)
6     for token != "12345abcde" {
7        fmt.Scan(&token)
8     }
9     fmt.Println("Selamat Anda berhasil login")
10 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
Qwe12312
231234
13213
123lijwe
12345abcde
Selamat Anda berhasil login
gppras@SR8 GO % ./Demo_Soal
12345abcde
Selamat Anda berhasil login
```

3) Buatlah program dalam bahasa Go yang digunakan untuk mencetak N bilangan pertama dalam deret Fibonacci.

Masukan terdiri dari bilangan bulat positif N dengan nilai besar atau sama dengan 2.

Keluaran terdiri dari sejumlah N bilangan yang menyatakan N deret bilangan Fibonacci yang pertama.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	0 1 1 2 3
2	2	0 1
3	10	0 1 1 2 3 5 8 13 21 34

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var N, s1, s2, j, temp int
5     fmt.Scan(&N)
6     s1 = 0
7     s2 = 1
8     j = 0
9     for j < N {
10       fmt.Print(s1," ")
11       temp = s1 + s2
12       s1 = s2
13       s2 = temp
14       j = j + 1
15    }
16 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
5
0 1 1 2 3
gppras@SR8 GO % ./Demo_Soal
2
0 1
gppras@SR8 GO % ./Demo_Soal
10
0 1 1 2 3 5 8 13 21 34
```

School of Computing Telkom University
informatics lab

12.5 Soal Latihan Modul 12

1) Buatlah sebuah program dalam bahasa Go yang digunakan untuk menghitung berapa banyak seseorang pengguna gagal melakukan login, karena kesalahan memberikan username dan password.

   Masukan terdiri dari dua teks yang berisi username dan password, apabila username dan password salah, maka program akan meminta masukan ulang. Apabila username dan password sudah benar maka program akan **menampilkan** informasi berapa kali percobaan login yang gagal dilakukan. Asumsi username dan password yang benar adalah "Admin" dan "Admin" tanpa tanda petik.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	User123 user123 User admin Admin admin Admin Admin123 Admin Admin	4 percobaan gagal login
2	Admin Admin	0 percobaab gagal login

2) Buatlah program yang digunakan untuk mencacah setiap digit yang terdapat di dalam suatu bilangan bulat positif.

Masukan terdiri dari suatu bilangan bulat positif.

Keluaran terdiri dari nilai digit pada bilangan tersebut. Tampilkan dari digit terakhir (paling kanan) sampai dengan digit pertama (paling kiri).

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	2	2
2	2544	4 4 5 2
3	3423554654	4 5 6 4 5 5 3 2 4 3

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program untuk untuk mencari hasil integer division dari dua bilangan. Gunakan perulangan dan tidak diperbolehkan menggunakan operator pembagian.

Masukan terdiri dari dua bilangan bulat positif x dan y, yang mana x >= x.

Keluaran terdiri dari hasil dari operasi x div dengan y.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5 2	2

2	10 7	1
3	120	30

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 13. REPEAT-UNTIL

13.1 Paradigma Perulangan

Perulangan merupakan salah satu struktur kontrol yang memungkinkan suatu instruksi yang sama dilakukan berulang kali dalam waktu atau jumlah yang lama. Tanpa instruksi perulangan, maka suatu instruksi akan ditulis dalam jumlah yang sangat banyak. Pada modul 12 sebelumnya telah dipelajari terkait penggunaan struktur kontrol perulangan dengan while-loop, selanjutnya perulangan juga dapat dilakukan menggunakan **repeat-until**.

Penggunaan repeat-until pada dasarnya sama dengan while-loop di mana perulangan berdasarkan kondisi. Perbedaan terletak pada kondisi yang digunakan, pada while-loop kondisi yang harus didefinisikan adalah kondisi perulangannya, atau kapan perulangan itu terjadi, sedangkan pada repeat-until kondisi yang harus didefinisikan merupakan kondisi berhenti, atau kapan perulangan tersebut harus dihentikan.

Kondisi perulangan dan kondisi berhenti memiliki keterhubungan sifat komplemen, sehingga apabila kita mengetahui kondisi perulangannya, maka cukup dengan menambahkan operator negasi atau not untuk mengubah menjadi kondisi berhenti. Hal ini berlaku juga sebaliknya, komplemen dari kondisi berhenti adalah kondisi perulangan.

Pahami beberapa contoh yang diberikan berikut ini:

- Statement while-loop: *"Menulis teks tertentu **selama** tinta pena masih ada"*.
  Statement repeat-until: *"Menulis teks tertentu **sampai** tinta pena habis"*.
  Komplemen dari kondisi "tinta pena masih ada" adalah "tinta pena habis".
- Statement while-loop: *"Saya makan suap demi suap **selama** saya masih lapar"*.
  Statement repeat-until: *"Saya makan suap demi suap **sampai** saya merasa kenyang"*.
  Komplemen dari kondisi "saya masih lapar" adalah. "*saya merasa kenyang*".

Fakultas Informatika
School of Computing
Telkom University
informatics lab

13.2 Karakteristik Repeat-Until

Komponen dari repeat-until sama dengan while-loop, yaitu terdapat kondisi dan aksi, hanya struktur penulisannya saja yang berbeda.

1) **Aksi**, merupakan kumpulan instruksi yang akan dilakukan perulangan. Aksi minimal dijalankan sekali, baru dilakukan pengecekan kondisi berhenti setelahnya. Apabila kondisi bernilai true, maka perulangan dihentikan.
2) **Kondisi/berhenti**, merupakan kondisi berhenti dari perulangan, harus bernilai false selama perulangan dilakukan.

Notasi repeat-until memiliki banyak sekali keragaman kata kunci di dalam bahasa pemrograman. Penggunaan repeat-until sebenarnya berasal dari keluarga bahasa pemrograman Pascal. Pada keluarga bahasa pemrograman C/C++ digunakan do-while, sedangkan pada bahasa Go tidak ada instruksi eksplisit untuk repeat-until.

Tabel 13-1 Notasi Perulangan Repeat-Until pada Pseudocode dan Bahasa Go

Notasi dalam pseudocode	Notasi dalam bahasa Go

repeat
  // aksi
until kondisi
	// versi dengan kata kunci break
	for kondisi = false; !kondisi; {
	  // aksi
	  kondisi = // update nilai kondisi
	}

13.3 Implementasi menggunakan Go

Sebagai contoh, misalnya terdapat suatu program yang digunakan untuk mengecek username dan password yang digunakan pengguna ketika login adalah "admin" dan "admin12345".

```
1   // filename: repeatuntil1.go
2   package main
3   import "fmt"
4
5   func main() {
6     var usr, pwd int
7     var kondisi bool
8     for kondisi = false; !kondisi; {
9       fmt.Scan(&usr, &pwd)
10      konsidi = usr == "admin" && pwd == "admin12345"
11    }
12    fmt.Println("Selamat, Anda berhasil login ")
13  }
14
```

```
C:\users\go\src\hello>go build repeatuntil1.go
C:\users\go\src\hello>repeatuntil1
user admin
admin admin
admin123 admin123
admin admin12345
Selamat, Anda berhasil login
C:\users\go\src\hello>repeatuntil1
admin admin12345
Selamat, Anda berhasil login
```

Contoh penggunaan bentuk **repeat-until** untuk mencetak deret bilangan Fibonacci:

Tabel 13-2 Contoh Implementasi Repeat-Until untuk Mencetak Deret Fibonacci dalam Algoritma dan Bahasa Go

	Notasi Algoritma	Penulisan dalam bahasa Go
1	maxF <- 100	maxF := 100
2	f0 <- 0	f0 := 0
3	f1 <- 1	f1 := 1
4	f2 <- 1	f2 := 1
5	**output**("Bilangan pertama:", f1)	fmt.**Println**("Bilangan pertama:", f1)
6	**repeat**	**for** selesai:=**false**; !selesai; {
7	f0 <- f1	f0 = f1
8	f1 <- f2	f1 = f2
9	f2 <- f1 + f0	f2 = f1 + f0

10	**output**("Bilangan berikutnya:",f1)	fmt.**Println**("Bilangan berikutnya:",f1)
11	**until** f2 > maxF	selesai = f2 > maxF
12		}

Perhatian: Karena pernyataan kondisi ada di bawah pada bentuk repeat-until, **apapun kondisinya**, badan loop **pasti akan pernah dieksekusi** minimum satu kali!

Kode program dengan bahasa Go di bawah menggunakan algoritma yang sangat mirip dengan algoritma di atas, dengan perbedaan pada digunakannya bentuk while-loop. Umumnya keluaran kedua algoritma sama, **kecuali** saat **maxF** diinisialisasi dengan nilai 0 atau lebih kecil!

```
1   maxF = 100
2   f0 = 0
3   f1 = 1
4   f2 = 1
5   fmt.Println("Bilangan pertama:", f1 )
6   for f2 <= maxF {
7      f0 = f1
8      f1 = f2
9      f2 = f1 + f0
10     fmt.Println("Bilangan berikutnya:", f1 )
11  }
```

informatics lab

13.4 Contoh Soal Modul 13

1) Buatlah program menggunakan bahasa Go yang menerima input kata dan mencetaknya sebanyak jumlah pengulangan yang diinginkan oleh pengguna. Program akan dihentikan ketika jumlah kata yang dicetak mencapai jumlah yang diinginkan oleh pengguna.

   Masukan berupa suatu kata dan jumlah pengulangan yang diinginkan oleh pengguna.

   Keluaran berupa kata yang diinputkan pengguna dan dicetak sebanyaknya jumlah pengulangan yang diinginkan oleh pengguna.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	pagi 3	pagi pagi pagi
2	kursi 5	kursi kursi kursi kursi kursi

Jawaban:

```
1   package main
2   import "fmt"
3   func main() {
4     var word string
5     var repetitions int
6     fmt.Scan(&word, &repetitions)
7     counter := 0
8     for done := false; !done; {
9       fmt.Println(word)
10      counter++
11      done = (counter >= repetitions)
12    }
13  }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
pagi 3
pagi
pagi
pagi
gppras@SR8 GO % ./Demo_Soal
kursi 7
kursi
kursi
kursi
kursi
kursi
kursi
kursi
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

2) Buatlah program dalam bahasa Go yang meminta pengguna untuk memasukkan bilangan bulat positif. Program akan terus meminta input hingga pengguna memasukkan bilangan bulat positif.

   Masukan berupa bilangan bulat positif, apabila bukan maka program akan terus meminta masukan hingga bilangan yang diberikan adalah bilangan bulat positif.

   Keluaran berupa satu baris keluaran yang menunjukkan n bilangan adalah bilangan bulat positif.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	-5 -2	5 adalah bilangan bulat positif

	-1 0 5	
2	17	17 adalah bilangan bulat positif

Jawaban:

```
1   package main
2   import "fmt"
3   func main() {
4       var number int
5       var continueLoop bool
6       for continueLoop = true; continueLoop; {
7           fmt.Scan(&number)
8           continueLoop = number <= 0
9       }
10      fmt.Printf("%d adalah bilangan bulat positif\n", number)
11  }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
17
17 adalah bilangan bulat positif
gppras@SR8 GO % ./Demo_Soal
-5
-2
-1
0
5
5 adalah bilangan bulat positif
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3) Buatlah program yang digunakan untuk melakukan pengecekan apakah suatu bilangan merupakan kelipatan dari bilangan lainnya.

   Masukan terdiri dari dua buah bilangan bulat positif X dan Y.

   Keluaran terdiri dari perulangan pengurangan kelipatan dengan hasil akhir boolean yang menyatakan apakah bilangan X merupakan kelipatan dari Y.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5 2	3 1 -1 false
2	15 3	12 9 6 3

		0 true
3	25 5	20 15 10 5 0 true

Jawaban:

```
1  package main
2  import "fmt"
3  func main() {
4     var x int
5     var y int
6     var selesai bool
7     fmt.Scan(&x, &y)
8     for selesai = false; !selesai; {
9        x = x - y
10       fmt.Println(x)
11       selesai = x <= 0
12    }
13    fmt.Println(x == 0)
14 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
5 2
3
1
-1
false
gppras@SR8 GO % ./Demo_Soal
15 3
12
9
6
3
0
true
gppras@SR8 GO % ./Demo_Soal
25 5
20
15
10
```

5
0
true

13.5 Soal Latihan Modul 13

1) Buatlah program yang digunakan untuk menghitung banyaknya digit dari suatu bilangan.

   Masukan berupa bilangan bulat positif.

   Keluaran berupa bilangan bulat yang menyatakan banyaknya digit dari bilangan yang diberikan pada masukan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1
2	234	3
3	78787	5
4	1894256	7

Fakultas Informatika
School of Computing
Telkom University

2) Buatlah program yang digunakan untuk mendapatkan bilangan bulat optimal dari bilangan yang telah diinputkan. Melakukan penjumlahan tiap perulangan mencapai pembulatan keatas dari bilangan yang diinputkan.

   Masukan berupa bilangan desimal.

   Keluaran terdiri dari bilangan hasil penjumlahan tiap perulangannya sampai pembulatan keatas dari bilangan yang diinputkan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	0.2	0.3 0.4 0.5 0.6 0.7 0.8 0.9 1
2	2.7	2.8 2.9 3

3) Sebuah organisasi amal sedang mengumpulkan donasi untuk mendukung kegiatan sosial mereka. Setiap donatur dapat memberikan sumbangan dalam jumlah tertentu. Program ini akan terus meminta input dari pengguna untuk jumlah donasi hingga total donasi mencapai atau melebihi target yang telah ditentukan.

   Masukan pada baris pertama berupa bilangan bulat yang merupakan target donasi yang harus dicapai. Masukan pada baris berikut dan seterusnya merupakan bilangan bulat yang menyatakan donasi oleh setiap donatur, masukan terus diterima hingga target tercapai.

   Keluaran berupa bilangan hasil total penjumlahan tiap perulangannya serta jumlah donatur.

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	300 100 50 200	Donatur 1: Menyumbang 100. Total terkumpul: 100 Donatur 2: Menyumbang 50. Total terkumpul: 150 Donatur 3: Menyumbang 200. Total terkumpul: 350 Target tercapai! Total donasi: 350 dari 3 donatur.
2	500 150 100 50 300	Donatur 1: Menyumbang 150. Total terkumpul: 150 Donatur 2: Menyumbang 100. Total terkumpul: 250 Donatur 3: Menyumbang 50. Total terkumpul: 300 Donatur 4: Menyumbang 300. Total terkumpul: 600 Target tercapai! Total donasi: 600 dari 4 donatur.
3	200 300	Donatur 1: Menyumbang 300. Total terkumpul: 300 Target tercapai! Total donasi: 300 dari 1 donatur.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

MODUL 14. KOMPOSISI

14.1 Pengantar Komposisi

Pada modul komposisi ini kita akan mempelajari soal-soal dengan tingkat kesulitan lebih, yang mana semua materi yang telah dipelajari saling dikombinasikan. Apabila sebelumnya penggunaan struktur kontrol saling dipisahkan, maka pada soal yang sekarang, memungkinkan melibatkan lebih dari satu jenis struktur kontrol.

Berikut contoh-contoh soal dengan komposisi struktur kontrol.

14.2 Contoh Soal Modul 14

1) Buatlah program yang digunakan untuk menampilkan sejumlah bilangan ganjil.

   Masukan terdiri dari sebuah bilangan bulat.

Keluaran berupa barisan bilangan ganjil dari 1 hingga bilangan bulat yang diberikan pada masukan.

Contoh masukan dan keluaran.

No	Masukan	Keluaran
1	3	1 3
2	2	1
3	7	1 3 5 7
4	10	1 3 5 7 9

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var bilangan, j int
5     fmt.Scan(&bilangan)
6     for j = 1; j <= bilangan; j +=1 {
7        if j % 2 != 0 {
8           fmt.Print(j," ")
9        }
10    }
11 }
```

School of Computing
Telkom University
informatics lab

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
3
1 3
gppras@SR8 GO % ./Demo_Soal
2
1
gppras@SR8 GO % ./Demo_Soal
7
1 3 5 7
gppras@SR8 GO % ./Demo_Soal
10
1 3 5 7 9
```

2) Buatlah program Go yang digunakan untuk mencari nilai terbesar dan terkecil antara tiga bilangan yang diberikan.

Masukan terdiri dari 3 bilangan bulat.

Keluaran terdiri dari dua bilangan yang menyatakan bilangan terbesar dan terkecil dari tiga bilangan yang diberikan.

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	1 2 3	Terbesar 3 Terkecil 1

2	5 5 5	Terbesar 5 Terkecil 5
3	12 31 -43	Terbesar 31 Terkecil -43

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var b1, b2, b3, max, min int
5     fmt.Scan(&b1, &b2, &b3)
6     if b1 > b2 {
7        max = b1
8        min = b2
9     }else {
10       max = b2
11       min = b1
12    }
13    if max < b3 {
14       max = b3
15    }
16    if min > b3 {
17       min = b3
18    }
19    fmt.Println("Terbesar", max)
20    fmt.Println("Terkecil", min)
21 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
1 2 3
Terbesar 3
Terkecil 1
gppras@SR8 GO % ./Demo_Soal
5 5 5
Terbesar 5
Terkecil 5
gppras@SR8 GO % ./Demo_Soal
12 31 -43
Terbesar 31
Terkecil -43
```

3) Buatlah program yang digunakan untuk menampilkan faktor bilangan dari suatu nilai. Faktor bilangan dari x adalah himpunan bilangan yang habis membagi bilangan x.

   Masukan terdiri dari suatu bilangan bulat positif.

   Keluaran berupa baris bilangan yang menyatakan semua faktor bilangan dari bilangan yang diberikan pada masukan.

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 5
2	12	1 2 3 4 6 12
3	20	1 2 4 5 10 20
4	72	1 2 3 4 6 8 9 12 18 24 36 72

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var bilangan, j int
5     fmt.Scan(&bilangan)
6     for j = 1; j <= bilangan; j += 1 {
7        if bilangan % j == 0 {
8           fmt.Print(j," ")
9        }
10    }
11 }
```

```
gppras@SR8 GO % go build Demo_Soal.go
gppras@SR8 GO % ./Demo_Soal
5
1 5
gppras@SR8 GO % ./Demo_Soal
12
1 2 3 4 6 12
gppras@SR8 GO % ./Demo_Soal
20
1 2 4 5 10 20
gppras@SR8 GO % ./Demo_Soal
72
1 2 3 4 6 8 9 12 18 24 36 72
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

14.3 Soal Latihan Modul 14

1) Buatlah sebuah program Go yang digunakan untuk menghitung banyaknya bilangan ganjil dari 1 hingga n.

   Masukan terdiri dari suatu bilangan bulat positif n.

   Keluaran terdiri dari suatu teks yang menyatakan banyaknya bilangan ganjil yang terdapat antara 1 hingga n.

   Catatan: Gunakan perulangan untuk pengecekan bilangan, bukan menggunakan operasi aritmatika.

   Contoh masukan dan keluaran

No	Masukan	Keluaran
1	3	Terdapat 2 bilangan ganjil

2	2	Terdapat 1 bilangan ganjil
3	7	Terdapat 4 bilangan ganjil
4	10	Terdapt 5 bilangan ganjil

2) Sebuah program digunakan untuk menentukan sebuah bilangan adalah prima atau bukan. Bilangan dikatakan prima apabila hanya memiliki faktor yaitu satu dan bilangan itu sendiri. Sebagai catatan bilangan satu bukanlah bilangan prima.

   Masukan terdiri dari suatu bilangan bulat positif.

   Keluaran berupa teks yang menyatakan bilangan adalah "prima" atau "bukan prima".

   Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	prima
2	12	bukan prima
3	19	prima
4	72	bukan prima

3) Siswa kelas IPA di salah satu sekolah menengah atas di Indonesia sedang mengadakan praktikum kimia. Di setiap percobaan akan menggunakan 4 tabung reaksi, yang mana susunan warna cairan di setiap tabung akan menentukan hasil percobaan. Siswa diminta untuk mencatat hasil percobaan tersebut. Percobaan dikatakan berhasil apabila susunan warna zat cair pada gelas 1 hingga gelas 4 secara berturutan adalah 'merah', 'kuning', 'hijau', dan 'ungu' selama 5 kali percobaan berulang.

   Buatlah sebuah program yang menerima input berupa warna dari ke 4 gelas reaksi sebanyak 5 kali percobaan. Kemudian program akan menampilkan **true** apabila urutan warna sesuai dengan informasi yang diberikan pada paragraf sebelumnya, dan **false** untuk urutan warna lainnya.

   Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

```
Percobaan 1: merah kuning hijau ungu
Percobaan 2: merah kuning hijau ungu
Percobaan 3: merah kuning hijau ungu
Percobaan 4: merah kuning hijau ungu
Percobaan 5: merah kuning hijau ungu
BERHASIL: true

Percobaan 1: merah kuning hijau ungu
Percobaan 2: merah kuning hijau ungu
Percobaan 3: merah kuning hijau ungu
Percobaan 4: ungu kuning hijau merah
Percobaan 5: merah kuning hijau ungu
BERHASIL: false
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

4) Suatu pita (string) berisi kumpulan nama-nama bunga yang dipisahkan oleh spasi dan '– ', contoh pita diilustrasikan seperti berikut ini.

Pita: mawar – melati – tulip – teratai – kamboja – anggrek

Buatlah sebuah program yang menerima input sebuah bilangan bulat positif (dan tidak nol) N, kemudian program akan meminta input berupa nama bunga secara berulang sebanyak N kali dan nama tersebut disimpan ke dalam pita. *(Petunjuk: gunakan operasi penggabungan string dengan operator "+")*.

Tampilkan isi pita setelah proses input selesai.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

N: **3** Bunga 1: **Kertas** Bunga 2: **Mawar** Bunga 3: **Tulip** Pita: Kertas – Mawar – Tulip -	N : **0** Pita :

Modifikasi program sebelumnya, proses input akan berhenti apabila user mengetikkan 'SELESAI'. Kemudian tampilkan isi pita beserta banyaknya bunga yang ada di dalam pita.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

Fakultas Informatika

Bunga 1: **Kertas** Bunga 2: **Mawar** Bunga 3: **Tulip** Bunga 4: **SELESAI** Pita: Kertas – Mawar – Tulip - Bunga: 3	Bunga 1: **SELESAI** Pita : Bunga: 0

5) Setiap hari Pak Andi membawa banyak barang belanjaan dari pasar dengan mengendarai sepeda motor. Barang belanjaan tersebut dibawa dalam kantong terpal di kiri-kanan motor. Sepeda motor tidak akan oleng jika selisih berat barang di kedua kantong sisi tidak lebih dari 9 kg.

Buatlah program Pak Andi yang menerima input dua buah bilangan riil positif yang menyatakan berat total masing-masing isi kantong terpal. Program akan terus meminta input bilangan tersebut hingga salah satu kantong terpal berisi 9 kg atau lebih.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

```
Masukan berat belanjaan di kedua kantong: 5.5 1.0
Masukan berat belanjaan di kedua kantong: 7.1 8.5
Masukan berat belanjaan di kedua kantong: 2 6
Masukan berat belanjaan di kedua kantong: 9 5.8
Proses selesai.
```

Pada modifikasi program tersebut, program akan menampilkan **true** jika selisih kedua isi kantong lebih dari atau sama dengan 9 kg. Program berhenti memproses apabila total berat isi kedua kantong melebihi 150 kg atau salah satu kantong beratnya negatif.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

```
Masukan berat belanjaan di kedua kantong: 5 10
Sepeda motor pak Andi akan oleng: false
Masukan berat belanjaan di kedua kantong: 55.6 70.2
Sepeda motor pak Andi akan oleng: true
Masukan berat belanjaan di kedua kantong: 72.3 66.9
Sepeda motor pak Andi akan oleng: false
Masukan berat belanjaan di kedua kantong: 59.5 98.7
Proses selesai.
```

6) Diberikan sebuah persamaan sebagai berikut ini.

$$f(k) = \frac{(4k+2)^2}{(4k+1)(4k+3)}$$

Buatlah sebuah program yang menerima input sebuah bilangan sebagai **K**, kemudian menghitung dan menampilkan nilai f(K) sesuai persamaan di atas.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

```
Nilai K = 100
Nilai f(K) = 1.0000061880
```

$\sqrt{2}$ merupakan bilangan irasional. Meskipun demikian, nilai tersebut dapat dihampiri dengan rumus berikut:

$$\sqrt{2} = \prod_{k=0}^{\infty} \frac{(4k+2)^2}{(4k+1)(4k+3)}$$

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Modifikasi program sebelumnya yang menerima input integer K dan menghitung $\sqrt{2}$ untuk K tersebut. Hampiran $\sqrt{2}$ dituliskan dalam ketelitian 10 angka di belakang koma.

Perhatikan contoh sesi interaksi program seperti di bawah ini (**teks bergaris bawah** adalah input/read):

1	Nilai K = **10** Nilai akar 2 = 1.4062058441
2	Nilai K = **100** Nilai akar 2 = 1.4133387072
3	Nilai K = **1000** Nilai akar 2 = 1.4141252651

MODUL 15. ASESMEN PRAKTIKUM 2

15.1 Asesmen Praktikum 2

Asesmen Praktikum 2 merupakan tes untuk materi dari modul ke-9 sampai dengan modul ke-14. Mahasiswa akan mengerjakan soal tes secara mandiri di laboratorium.

Materi meliputi penggunaan:

1) Struktur kontrol percabangan: If-Then, Else-IF, atau Switch-Case.
2) Struktur kontrol perulangan berdasarkan kondisi: While-Loop atau Repeat-Until.
3) Komposisi struktur kontrol.

15.2 Contoh Soal Asesmen Praktikum 2

1) [**Segitiga**] Buatlah program dengan bahasa Go untuk menentukan apakah tiga sisi yang diinputkan membentuk segitiga sama sisi, sama kaki, sembarang, siku-siku, atau bukan segitiga. Tiga buah sisi dikatakan membentuk segitiga apabila jumlah panjang dua sisinya lebih besar dari panjang sisi yang tersisa.

   Masukan berupa tiga bilangan bulat positif a, b, dan c.

   Keluaran berupa string "Segitiga sama sisi" jika ketiga sisi sama panjang, "**Segitiga sama kaki**" jika dua sisi sama panjang, "**Segitiga siku-siku**" jika memenuhi teorema Pythagoras, "**Segitiga sembarang**" jika ketiga sisi berbeda panjang, dan "**Bukan segitiga**" apabila ketiga sisi tersebut tidak memenuhi syarat sebagai segitiga.

Fakultas Informatika
School of Computing
Telkom University
Informatics lab

   Contoh masukan dan keluaran

No	Masukan	Keluaran
1	3 3 3	Segitiga sama sisi
2	5 5 8	Segitiga sama kaki
3	3 4 5	Segitiga siku-siku
4	4 5 6	Segitiga sembarang
5	1 2 3	Bukan segitiga

   Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4     var a, b, c int
5     fmt.Scan(&a, &b, &c)
6     if a > b {
7        a, b = b, a
8     }
9     if b > c {
```

```
    b, c = c, b
  }
  if a > b {
    a, b = b, a
  }
  if a+b > c {
    if a == b && b == c {
      fmt.Println("Segitiga sama sisi")
    } else if a == b || b == c || a == c {
      fmt.Println("Segitiga sama kaki")
    } else if a*a+b*b == c*c {
      fmt.Println("Segitiga siku-siku")
    } else {
      fmt.Println("Segitiga sembarang")
    }
  }else{
    fmt.Println("Bukan segitiga")
  }
}
```

2) [**ASCII**] Buatlah program dalam bahasa Go yang dapat membaca dua buah karakter ASCII. Kemudian periksalah apakah karakter tersebut sama tanpa memperhatikan huruf besar atau kecil. Artinya, karakter "A" dianggap sama dengan "a".

Masukan berupa dua buah karakter yang dipisahkan oleh sebuah spasi.

Keluaran berupa string "Ya" jika karakter sama, atau "Tidak" jika berbeda.

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	A b	Tidak
2	V v	Ya
3	B b	Ya
4	[ ]	Tidak

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Jawaban

```
package main
import "fmt"
func main() {
  var a, b rune
  fmt.Scanf("%c %c", &a, &b)
  if ((a >= 'a' && a <= 'z') || (a >= 'A' && a <= 'Z')) && ((b >= 'a' && b <= 'z') || (b >= 'A' && b <= 'Z')) {
    if (b-a == 32) || (a-b == 32) || (a-b == 0) {
      fmt.Println("Ya")
    } else {
      fmt.Println("Tidak")
```

```
    }
  } else {
    fmt.Println("Tidak")
  }
}
```

3) [**Contains String**] Buatlah sebuah program untuk menghitung banyaknya string “go” pada serangkaian karakter yang diinputkan.

Masukan terdiri dari serangkaian karakter yang diakhiri oleh tanda titik ".".

Keluaran berupa jumlah total bilangan bulat dari banyaknya string “go” yang ditemukan.

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	Inigolang.	1
2	Gilinggiling.	0
3	Inigolanggolanggo.	3

Jawaban

```
package main
import "fmt"
func main() {
  var c byte
  var counter , flag int
  fmt.Scanf("%c", &c)
  counter = 0
  flag = 0
  for c != '.' {
    if c == 'g'{
      flag = 1
    }else if flag == 1 && c == 'o' {
      flag = 0
      counter = counter + 1
    }else{
      flag = 0
    }
    fmt.Scanf("%c",&c)
  }
  fmt.Println(counter)
}
```

4) [**Deret Aritmatika**] Buatlah sebuah program untuk menentukan apakah sekumpulan bilangan bulat yang diberikan membentuk deret aritmatika. Jika deret tersebut valid sebagai deret aritmatika, program akan menghitung dan menampilkan jumlah total dari seluruh suku dalam deret tersebut.

Masukan terdiri dari dua baris. Baris pertama berupa sebuah bilangan bulat n, yang menunjukkan jumlah elemen dalam deret, sedangkan baris kedua adalah n bilangan bulat yang menjadi elemen-elemen deret tersebut.

Keluaran berupa string "Deret tidak valid" jika n < 3, jumlah total dari seluruh suku dalam deret jika merupakan deret aritmatika, dan begitu juga sebaliknya.

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	3 7 14	Deret tidak valid
2	5 2 4 6 8 10	Jumlah total: 30
3	4 1 3 5 8	Bukan deret aritmatika
4	3 9 6 3	Jumlah total: 18

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4    var n, a, b, c, total int
5    var selisih int
6    var aritmatika bool
7    fmt.Scan(&n)
8    fmt.Scan(&a)
9    fmt.Scan(&b)
10   total = a + b
11   aritmatika = true
12   for i:=3; i <= n; i++{
13     fmt.Scan(&c)
14     if (c - b) != selisih {
15       aritmatika = false
16     }
17   }
18   if n < 3 {
19     fmt.Println("Deret tidak valid")
20   } else if aritmatika {
21     fmt.Printf("Jumlah total: %d\n", total)
22   } else {
23     fmt.Println("Bukan deret aritmatika")
24   }
25 }
```

5) [**Pola X**] Buatlah sebuah program untuk menampilkan bola berbentuk X sesuai besar iterasi n.

Masukan berupa bilangan bulat ganjil n.

Keluaran terdiri dari sebuah pola X dengan perulangan iterasi dari n.

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	1	1
2	7	1 1 2 2 3 3 4 5 5 6 6 7 7
3	5	1 1 2 2 3 4 4 5 5

Jawaban

```
1  package main
2  import "fmt"
3  func main() {
4    var x, baris, kolom int
5    fmt.Scan(&x)
6    for baris = 1; baris <= x; baris++ {
7      for kolom = 1; kolom <= x; kolom++ {
8        if (baris == kolom) || (kolom+baris-1 == x) {
9          fmt.Print(baris, " ")
10       } else {
11         fmt.Print("  ")
12       }
13     }
14     fmt.Println()
15   }
16 }
```

MODUL 16. SKEMA PEMROSESAN SEKUENSIAL (PENGAYAAN)

16.1 Pengantar Skema Pemrosesan Sekuensial

Dengan dipersenjatai bentuk perulangan dan bentuk percabangan, banyak problem komputasi yang dapat diselesaikan. Berikut ini beberapa skema (pola) yang umum ditemukan untuk pemrosesan data (secara sekuensial).

16.2 Pembacaan Data Tanpa Marker pada Akhir Rangkaian Data

Pola ini memperlihatkan bahwa semua data yang diberikan pada masukan adalah data yang harus diproses. Berikut contoh polanya

Tabel 16-1 Pola Pembacaan Data Tanpa Marker pada Notasi Algoritma dan Bahasa Go

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(n)	fmt.**Scanln**(&n)
2	i = 1	i = 0
3	**while** i <= n **do**	**for** i < n {
4	**input**(dat)	fmt.**Scan**(&dat)
5	*{ kode untuk memproses dat }*	*// kode untuk memproses dat*
6	i = i + 1	i = i + 1
7	**endwhile**	}

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Pada pola di atas, terlihat bahwa variabel ***dat*** yang diperoleh pada baris ke-4 adalah data valid yang pasti diproses pada baris ke-5. Baris ke-5 bisa berisi potongan kode apapun yang digunakan untuk memproses variabel ***dat*** tersebut.

Berikut contohnya pada potongan program pencarian nilai maksimum:

Tabel 16-2 Contoh Penerapan Pola Pembacaan Data untuk Pencarian Nilai Maksimum dalam Notasi Algoritma dan Bahasa Go

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(n)	fmt.**Scan**(&n)
2	max = *{BILANGAN_KECIL}*	max = // *BILANGAN_KECIL*
3	i = 1	i = 1
4	**while** i <= n **do**	**for** i <= n {
5	**input**(dat)	fmt.**Scan**(&dat)
6	**if** dat > max **then**	**if** dat > max {
7	max = dat	max = dat
8	**endif**	}
9	i =i + 1	i = i + 1
10	**endwhile**	}
11	**output**("Data terbesar", max)	fmt.**Println**("Data terbesar", max)

16.3 Pembacaan Data dengan Marker pada Akhir Rangkaian Data

Pada pola denga marker, terdapat data yang dipersiapkan khusus untuk menghentikan perulangan. Artinya semua data yang diberikan pada masukan adalah data yang valid, kecuali data yang terakhir, karena digunakan untuk menghentikan perulangan. Berikut contoh polanya:

Tabel 16-3 Pola Pembacaan Data Menggunakan Marker pada Notasi Algoritma dan Bahasa Go

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(dat)	fmt.**Scanln**(&dat)
2	**while** dat != MARKER **do**	**for** dat != MARKER {
3	*{kode untuk memproses dat}*	*// kode untuk memproses dat*
4	**input**(dat)	fmt.**Scanln**(&dat)
5	**endwhile**	}

Apabila kita memperhatikan pola dengan marker di atas, terlihat bahwa selalu ada pengkondisian pada while yang akan selalu menyecek nilai ***dat***. Semua nilai pada variabel ***dat***, baik yang diperoleh pada baris ke-1 ataupun baris ke-4 akan selalu dicek pada baris ke-2. Apabila variabel ***dat*** tidak berisi ***marker***, maka ***dat*** dapat diproses pada baris ke-3, sebaliknya apabila ***dat*** adalah ***marker***, maka perulangan akan berhenti, dan ***dat*** tidak akan diproses pada baris ke-3.

Perhatikan contoh potongan program mencari nilai maksimum berikut ini:

Tabel 16-4 Contoh Penerapan Pola Pembacaan Data Menggunakan Marker untuk Pencarian Nilai Maksimum dalam Algoritma dan Bahasa Go

Fakultas Informatika School of Computing Telkom University Informatics lab

	Notasi Algoritma	Notasi dalam bahasa Go
1	max = *{BILANGAN_KECIL}*	max = *// BILANGAN_KECIL*
2	**input**(dat)	fmt.**Scan**(&dat)
3	**while** dat != MARKER **do**	**for** dat != MARKER {
4	**if** dat > max **then**	**if** dat > max {
5	max = dat	max = dat
6	**endif**	}
7	**input**(dat)	fmt.**Scan**(&dat)
8	**endwhile**	}
9	**output**("Data terbesar", max)	fmt.**Println**("Data terbesar", max)

Nilai ***marker*** bisa nilai berapapun, biasanya diberikan pada soal atau kita biasanya bisa memberikan nilai berdasarkan asumsi.

16.4 Kemungkinan Rangkaian Data Kosong, Hanya Ada Marker

Pola dengan marker pada contoh di atas memungkinkan terjadi bahwa data pertama yang diberikan pada masukan adalah marker, artinya tidak ada satu datapun yang valid. Kemungkinan ini disebut juga rangkaian data kosong atau kasus kosong.

Tabel 16-5 Pola Pembacaan Data Menggunakan Marker dengan Penanganan Kasus Data Kosong pada Notasi Algoritma dan Bahasa Go

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(dat)	fmt.**Scanln**(&dat)
2	**if** dat == MARKER **then**	**if** dat == MARKER {
3	*{kode untuk data kosong}*	*// kode untuk data kosong*
4	**else**	}**else**{
5	**while** dat != MARKER **do**	**for** dat != MARKER {
6	*{kode untuk memproses dat}*	*// kode untuk memproses dat*
7	**input**(dat)	fmt.**Scanln**(&dat)
8	**endwhile**	}
9	**endif**	}

Pada pola di atas ditambahkan struktur kontrol pengkondisian atau percabangan apabila ada kasus kosong yang terjadi. Berikut ini adalah contoh kode untuk mencari nilai maksimum.

Tabel 16-6 Contoh Penerapan Pola Pembacaan Data Berbasis Marker dengan Penanganan Data Kosong untuk Pencarian Nilai Maksimum

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(dat)	fmt.**Scanln**(&dat)
2	**if** dat == MARKER **then**	**if** dat == MARKER {
3	**output**("tidak ada data")	fmt.Println("tidak ada data")
4	**else**	}**else**{
5	max = *{BILANGAN KECIL }*	max = *// BILANGAN KECIL*
6	**while** dat != MARKER **do**	**for** dat != MARKER {
7	**if** dat > max **then**	**if** dat > max {
8	max = dat	max = dat
9	**endif**	}
10	**input**(dat)	fmt.**Scanln**(&dat)
11	**endwhile**	}
12	**output**("Data terbesar", max)	fmt.**Println**("Data terbesar", max)
13	**endif**	}

Fakultas Informatika
Telkom University
informatics lab

16.5 Elemen Pertama Perlu Diproses Tersendiri/Kasus Khusus

Pada pola ini data pertama diproses terlebih dahulu secara khusus sebelum perulangan dilakukan. Apabila melihat contoh pencarian nilai maksimum di atas, terlihat bahwa nilai variabel ***max*** selalu diinisialisasi oleh sebuah nilai ***BILANGAN KECIL*** berapapun. Kekurangan dari pendekatan ini adalah kita harus mengetahui secara pasti nilai-nilai yang mungkin ada pada variabel ***dat***, yanga mana nilai pada variabel ***dat*** tersebut **TIDAK BOLEH** lebih kecil dibandingkan nilai dari ***BILANGAN KECIL*** yang digunakan saat inisialisasi. Sebagai contoh mencari nilai temperatur atau suhu maksimum. Pada kasus ini, berapa nilai dari ***BILANGAN KECIL*** yang akan digunakan? Tentu kita akan kesulitan menentukannya, karena temperatur bisa bernilai negatif.

Oleh karena itu, dengan menggunakan pola Kasus Khusus ini, penentuan ***BILANGAN KECIL*** tersebut data dapat terselesaikan.

Tabel 16-7 Pola Pembacaan Data Berbasis Marker dengan Pemrosesan Data Pertama dan Penanganan Kasus Kosong

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(dat)	fmt.**Scanln**(&dat)
2	**if** dat == MARKER **then**	**if** dat == MARKER {
3	*{kode untuk data kosong}*	*// kode untuk data kosong*
4	**else**	}**else**{
5	*{proses data pertama}*	*{proses data pertama}*
6	**input**(dat)	fmt.**Scan**(&dat)
7	**while** dat != MARKER **do**	**for** dat != MARKER {
8	*{kode untuk memproses dat}*	*// kode untuk memproses dat*
9	**input**(dat)	fmt.**Scanln**(&dat)
10	**endwhile**	}
11	**endif**	}

Pada pola tersebut, terlihat data atau elemen pertama yang diperoleh pada baris ke-1 diproses pada baris ke-5, selanjutnya data ke-2 didapat pada baris ke-6.

Berikut contoh potongan program untuk mencari nilai maksimum:

Tabel 16-8 Contoh Penerapan Pola Pembacaan Berbasis Marker dengan Inisialisasi Data Pertama untuk Pencarian Nilai Maksimum

	Notasi Algoritma	Notasi dalam bahasa Go
1	**input**(dat)	fmt.**Scanln**(&dat)
2	**if** dat == MARKER **then**	**if** dat == MARKER {
3	**output**("tidak ada data")	fmt.**Println**("tidak ada data")
4	**else**	}**else**{
5	max = dat	max = dat
6	**input**(dat)	fmt.**Scan**(&dat)
7	**while** dat != MARKER **do**	**for** dat != MARKER {
8	**if** dat > max **then**	**if** dat > max {
9	max = dat	max = dat
10	**endif**	}
11	**input**(dat)	fmt.**Scanln**(&dat)
12	**endwhile**	}
13	**output**("Data terbesar", max)	fmt.**Println**("Data terbesar", max)
14	**endif**	}

Fakultas Informatika School of Computing Telkom University informatics lab

16.6 Soal Latihan Modul 16

1) Diberikan sejumlah bilangan riil yang diakhiri dengan marker 9999, cari rerata dari bilangan-bilangan tersebut.

2) Diberikan string x dan n buah string. x adalah data pertama yang dibaca, n adalah data bilangan yang dibaca kedua, dan n data berikutnya adalah data string. Buat algoritma untuk menjawab pertanyaan berikut:

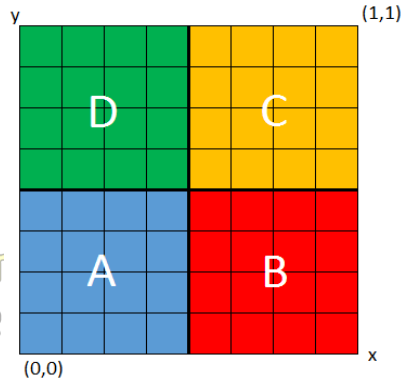
   a. Apakah string x ada dalam kumpulan n data string tersebut?

   b. Pada posisi ke berapa string x tersebut ditemukan?

   c. Ada berapakah string x dalam kumpulan n data string tersebut?

   d. Adakah sedikitnya dua string x dalam n data string tersebut?

3) Empat daerah A, B, C, dan D yang berdekatan ingin mengukur curah hujan. Keempat daerah tersebut digambarkan pada bidang berikut:



Fakultas
School of Cor
Telkom Unive
informatics lab

Gamber 16-1 Ilustrasi denah daerah yang digunakan untuk mengukur curah hujan

Misal curah hujan dihitung berdasarkan banyaknya tetesan air hujan. Setiap tetesan berukuran 0.0001 ml curah hujan. Tetesan air hujan turun secara acak dari titik (0,0) sampai (1,1). Jika diterima input yang menyatakan banyaknya tetesan air hujan. Tentukan curah hujan untuk keempat daerah tersebut.

Buatlah program yang menerima input berupa banyaknya tetesan air hujan. Kemudian buat koordinat/titik (x, y) secara acak dengan menggunakan fungsi **rand.Float64()**. Hitung dan tampilkan banyaknya tetesan yang jatuh pada daerah A, B, C dan D. Konversikan satu tetesan berukuran 0.0001 milimeter.

Catatan: Lihat lampiran untuk informasi menggunakan paket math/rand untuk menggunakan **rand.Float64()** yang menghasilkan bilangan riil acak [0..1].

Berikut contoh masukan dan keluarannya:

No	Masukan	Keluaran
1	10000000	Curah hujan daerah A: 250.0066 milimeter Curah hujan daerah B: 249.8981 milimeter Curah hujan daerah C: 249.9930 milimeter Curah hujan daerah D: 250.1023 milimeter

4) Berdasarkan formula Leibniz, nilai π dapat dinyatakan sebagai deret harmonik ganti sebagai berikut:

$$1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots = \frac{\pi}{4}$$

Suku ke-*i* dinyatakan sebagai S_i dan **jumlah deret** adalah S. Apabila diketahui suku pertama $S_1 = 1$, suku kedua $S_2 = \frac{-1}{3}$. Temukan rumus untuk suku ke $-$ ***i*** atau S_i.

Berdasarkan rumus tersebut, buatlah program yang menghitung S untuk 1000000 suku pertama.

Perhatikan contoh sesi interaksi program di bawah ini (**teks bergaris bawah** adalah input/read):

```
N suku pertama: 1000000
Hasil PI: 3.1415951
```

Setelah jalan, modifikasi program tersebut agar menyimpan nilai dua suku yang bersebelahan, S_i dan S_{i+1}. Buatlah agar program tersebut sekarang berhenti apabila selisih dari kedua suku tersebut tidak lebih dari 0.00001.

Perhatikan contoh sesi interaksi program di bawah ini (**teks bergaris bawah** adalah input/read):

```
N suku pertama: 1000000
Hasil PI: 3.1415876535
Hasil PI: 3.1415976535
Pada i ke: 200002
```

5) Monti bekerja pada sebuah kedai pizza, saking ramainya kedai tersebut membuat Monti tidak ada waktu untuk bersantai. Suatu ketika saat sedang menaburkan topping pada pizza yang diletakkan pada wadah berbentuk persegi, terpikirkan oleh Monti cara menghitung berapa **banyak topping yang dia butuhkan**, dan cara menghitung **nilai π**.

Ilustrasi seperti gambar yang diberikan di bawah, topping adalah lingkaran-lingkaran kecil. Ada yang tepat berada di atas pizza, dan ada yang jatuh di dalam kotak tetapi berada di luar pizza.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Apabila luas pizza yang memiliki radius r adalah $LuasPizza = \pi r^2$ dan luas wadah pizza yang memiliki panjang sisi $d = 2r$ adalah $LuasWadah = d^2 = 4r^2$, maka diperoleh perbandingan luas kedua bidang tersebut

$$\frac{LuasPizza}{LuasWadah} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Persamaan lingkaran adalah $(x - x_c)^2 + (y - y_c)^2 = r^2$ dengan titik pusat lingkaran adalah (x_c, y_c). Suatu titik sembarang (x, y) dikatakan berada di dalam lingkaran apabila memenuhi ketidaksamaan:

$$(x - x_c)^2 + (y - y_c)^2 \leq r^2$$

Pada ilustrasi topping berbentuk bulat kecil merah dan biru pada gambar adalah titik-titik (x, y) acak pada sebuah wadah yang berisi pizza. Dengan jumlah yang sangat banyak dan ditaburkan merata (secara acak), maka kita bisa mengetahui berapa banyak titik/topping yang berada tepat di dalam pizza menggunakan ketidaksamaan di atas.

Buatlah program yang menerima input berupa banyaknya topping yang akan ditaburkan, kemudian buat titik acak (x, y) dari bilangan acak riil pada kisaran nilai 0 hingga 1 sebanyak topping yang diberikan. Hitung dan tampilkan berapa banyak topping yang jatuh tepat di atas pizza.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Titik pusat pizza adalah (0.5, 0.5) dan jari-jari pizza adalah 0.5 satuan wadah.

Perhatikan contoh sesi interaksi program di bawah ini (**teks bergaris bawah** adalah input/read):

Banyak Topping: **1234567** Topping pada Pizza: 969000	Banyak Topping: **10000000** Topping pada Pizza: 7856565

Apabila topping yang ditaburkan oleh Monti secara merata berjumlah yang sangat banyak, maka topping akan menutupi keseluruhan wadah pizza. **Luas Pizza sebanding dengan topping yang berada pada pizza**, sedangkan **Luas Wadah sebanding dengan banyaknya topping yang ditaburkan**. Dengan menggunakan rumus perbandingan luas yang diberikan di atas, maka nilai konstanta π dapat dihitung.

Modifikasi program di atas sehingga dapat menghitung dan menampilkan nilai konstanta π.

Perhatikan contoh sesi interaksi program di bawah ini (teks bergaris bawah adalah input/read):

Banyak Topping: **1234567** Topping pada Pizza: 969206	Banyak Topping: **10** Topping pada Pizza: 5

PI : 3.1402297324	PI : 2.0000000000
Banyak Topping: **256** Topping pada Pizza: 198 PI : 3.0937500000	Banyak Topping: **5000** Topping pada Pizza: 3973 PI : 3.1784000000

Fakultas Informatika
School of Computing
Telkom University
informatics lab

DAFTAR PUSTAKA

1. Buku Ajar: Yunanto, P. E., Bunyamin, & Al Faraby, S. (2023). *Pengenalan Algoritma Pemrograman dalam Bahasa Go dan Python*. Tel-U Press.
2. Tutorial: https://go.dev/tour/welcome/1
3. Dokumentasi GoLang: https://go.dev/
4. Daftar paket standar yang tersedia dalam bahasa Go: https://pkg.go.dev/std
5. Situs tutorial dari pihak ketiga:
    a. https://gobyexample.com,
    b. https://golangbot.com,
    c. https://golangtutorial.com,
    d. https://www.tutorialspoint.com/go/index.htm ,
6. Situs latihan pemrograman:
    a. https://hackerrank.com,
    b. https://spoj.com

Fakultas Informatika
School of Computing
Telkom University

informatics lab

LAMPIRAN

A. Paket-Paket Penting

1. Paket "math"

Berikut adalah konstanta dan fungsi yang tersedia dalam paket math. Manfaat fungsi tersebut sudah sangat jelas, kecuali:

a. Fungsi **Modf(x)(b,p)** mengembalikan bagian bilangan bulat b dan bilangan pecahan p dari suatu bilangan riil x.

b. Fungsi **Pow(x,y)** menghitung x pangkat y (x^y)

c. Fungsi **Pow10(n)** menghitung 10 pangkat n (10^n)

```
const E = 2.718281828458945235...
const Pi= 3.145926535897932384...
const Sqrt2= 1.41421356237309504880...
```

```
func Cos(x float64) float64
func Sin(x float64) float64
func Tan(x float64) float64
func Abs(x float64) float64
func Floor(x float64) float64
func Round(x float64) float64
func Trunc(x float64) float64
func Log10(x float64) float64
func Log2(x float64) float64
func Modf(x float64) (int float64, frac float64)
func Pow(x,y float64) float64
func Pow10(n int) float64
func Sqrt(x float64) float64
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Contoh:

```
package main

import (
   "fmt"
   "math"
)

func main() {
   sinX := math.Sin(1)
   cosX := math.Cos(1)
   fmt.Println( (sinX*sinX + cosX*cosX), math.Sqrt(2) )
}
```

```
$ go build
$ ./contohmath
1 1.4142135623730951
$
```

2. Paket "math/rand"

Paket ini berisi pembangkit nilai-acak semu berbasis rumus matematika:

a. Prosedure **Seed(s)** untuk menginisialisasi awal nilai acak.

b. Fungsi **Int()** menghasilkan nilai acak integer dalam rentang [-231 s.d. 231].

c. Fungsi **Intn(n)** menghasilkan nilai integer acak dalam rentang [0 .. n), yaitu antara 0 s.d. n-1.

d. Fungsi **Float64()** menghasilkan nilai riil acak dalam rentang [0 .. 1.0).

```
func Seed(seed int64)
func Int() int
func Intn(n int) int
func Float64() float64
```

Contoh:

```
package main

import (
   "fmt"
   "math/rand"
)

func main() {
   rand.Seed(1)
   for i:=1; i<10; i++ {
      fmt.Print( rand.Intn(10), " " )
   }
   fmt.Println()
}
```

```
$ go build
$ ./contohrand
1 7 7 9 1 8 5 0 6
$ ./contohrand
1 7 7 9 1 8 5 0 6
$
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3. Paket "time"

Paket ini berisi berbagai fungsi untuk mengambil informasi tanggal dan waktu dari sistem:

a. Tipe **Time** mewakili waktu sampai satuan nanosekon.

b. Fungsi **Date(...)** untuk membuat data waktu dari parameter yang diberikan.

c. Fungsi **Now()** untuk mengambil data waktu dari sistem.

d. Metoda **t.Date() (y,m,d)** untuk mengambil tanggal dari data **Time t**.

e. Metoda **t.Clock() (h,m,s)** untuk mengambil waktu dari data **Time t**.

f. Metoda **t.UnixNano()** untuk mengambil data nanosekon dari data **Time t**.

g. Metoda **t.Unix()** untuk mengambil data detik dari data **Time t**.

```
type Time
func Date( yy int, mo Month, dd, hh, mm, ss, ns int, loc *Location) Time
func Now() Time
func (t Time)Date() (year int, month Month, day int)
func (t Time)Clock() (hour, min, sec int)
func (t Time)UnixNano() int64
func (t Time)Unix() int64
```

Contoh:

```
package main
import (
   "fmt"
   "math/rand"
    “time”
)
func main() {
   curtime := time.Now()
   rand.Seed(curtime.UnixNano())
   for i:=1; i<10; i++ {
      fmt.Print( rand.Intn(10), “ “ )
   }
   fmt.Println()
}
```

```
$ go build
$ ./contohrandtime
6 2 2 2 8 0 4 0 6
$ ./contohrandtime
2 6 2 5 4 9 8 3 1
$ ./contohrandtime
7 5 1 2 4 7 8 6 9
$
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

4. Paket "os"

Paket ini berisi akses untuk layanan sistem operasi.

a. Argumen yang diberikan waktu mengeksekusi program, termasuk nama program itu sendiri.
   var Args[]string

b. Tipe data untuk handel file.
   type File

c. Handel file untuk input standar, diantaranya digunakan oleh fmt.Scan.
   var Stdin

d. Handel file untuk output standar, diantaranya digunakan oleh fmt.Print.
   var Stdout

e. Handel file untuk output error standar, diantaranya digunakan oleh log.Print.
   var Stderr

f. Fungsi Create membuat file baru atau menimpah file lama.
   func Create(n string) (*File, error)

g. Fungsi Open membuka file yang sudah ada.
   func Open(n string) (*File, error)

h. Fungsi Close menutup file yang sudah dibuka oleh Create/Open.
   func (f *File)Close() error

i. Metoda Read membaca data dalam bytes dengan mengembalikan jumlah data yang berhasil dibaca.
func (f *File)Read(b []byte) (int, error)ALGORITMA PEMROGRAMAN 86

j. Metoda Write menulis data dalam bytes dengan mengembalikan jumlah data yang berhasil ditulis.
func (f *File)Write(b []byte) (int, error)

Contoh:

```
package main
import "log"
import "os"
func main() {
   fin, err := os.Open(os.Args[1])
   if err != nil {
      log.Fatal(err)
   } else {
      defer fin.Close()
   }
   fout, err := os.Create(os.Args[2])
   if err != nil {
      log.Fatal(err)
   } else {
      defer fout.Close()
   }
   data := make([]byte, 1000000)
   n, _ := fin.Read(data)
   fout.Write(data[:n])
}
```

```
$ go build
$ ls -l
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 15:35 sumber.dat
$ ./contohos sumber.dat tujuan.dat
$ ls -l
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 15:35 sumber.dat
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 17:13 tujuan.dat
$
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

5. Paket "fmt"

Ini merupakan paket yang paling sering dipakai karena menyediakan proses standar input/output teks. Terdapat tiga variasi instruksi; dua ala bahasa Pascal, dan satu ala bahasa C. Juga terdapat tiga target proses I/O, yaitu ke/dari file standar (layar monitor + keyboard), ke/dari file teks umum, dan ke/dari data string.

a. Membaca teks dari **os.Stdin**, input standar yang biasanya berasal dari keyboard atau dari pengalihan input.

func Scan(a ...interface{}) (n int, err error)

func Scanf(format string, a ...interface{}) (n int, err error)

func Scanln(a ...interface{}) (n int, err error)

b. Membaca teks dari file umum, biasanya file yang telah dibuka oleh **os.Open**

func Fscan(r io.Reader, a ...interface{}) (n int, err error)

func Fscanf(r io.Reader, fmt string, ...interface{}) (int, error)

func Fscanln(r io.Reader, a ...interface{}) (n int, err error)

c. Mengkonversi data string dengan proses **Scan**

func Sscan(s string, a ...interface{}) (n int, err error)

func Sscanf(s string, fmt string, a ...interface{}) (int, error)

func Sscanln(s string, a ...interface{}) (n int, err error)

d. Menulis teks ke **os.Stdout**, output standar yang biasanya layar monitor atau dapat dialihoutput ke media lain.

func Print(a ...interface{}) (n int, err error)

func Printf(format string, a ...interface{}) (n int, err error)

func Println(a ...interface{}) (n int, err error)

e. Menulis teks ke file umum, biasanya file yang telah dibuka oleh **os.Create**

func Fprint(w io.Writer, a ...interface{}) (n int, err error)

func Fprintf(w io.Writer, fmt string, ...interface{}) (int, error)

func Fprintln(w io.Writer, a ...interface{}) (n int, err error)

f. Menkonversi data ke bentuk string dengan proses **Print.**

func Sprint(a ...interface{}) string

func Sprintf(format string, a ...interface{}) string

func Sprintln(a ...interface{}) string

Fakultas Informatika
School of Computing
Telkom University
Informatics Lab

Berikut ini beberapa kode pemformat yang dapat digunakan dalam string format:

- **%v** adalah bentuk format yang paling umum, mengambil nilai dari argumen yang diberikan dan mencetak tampilan sesuai dengan tipe data dari argumen tersebut. **%+v** dapat digunakan jika nama field yang terkait dari record/struct juga ingin dicetak.

- **%%** mencetak simbol persen (%).
- **%t** mencetak nilai boolean: **true** atau **false**.
- **%b** mencetak bilangan dengan basis-2 (biner, tiap bit).
- **%c** mencetak karakter (ASCII atau Unicode) dari bilangan integer. **%v** akan selalu mencetak nilai
- integernya, tidak akan mencetak karakter.
- **%d** mencetak bilangan dengan basis-10 (bilangan integer yang biasa kita lihat)
- **%o** mencetak bilangan dengan basis-8 (oktal, per 3 bit)
- **%X** mencetak bilangan dengan basis-16 (heksadesimal, per 4 bit)
- **%E** mencetak bilangan riil menggunakan notasi ilmiah (dalam bentuk x.yE±z).
- **%f** mencetak bilangan riil menggunakan titik desimal. Lebar tampilan dan presisi dapat diberikan menggunakan format **%w.pf**, **w** adalah lebar tampilan dan **p** presisi yang ditampilkan.
- **%G** mencetak bilangan riil serapih mungkin.

- **%s** mencetak string

Contoh

```
package main
import "fmt"
import "os"
func main() {
   if len(os.Args) != 3 {
      fmt.Printf("Run as: %v read|write filename\n", os.Args[0])
      fmt.Printf("eg. %v write short.txt\n", os.Args[0])
   } else if os.Args[1] == "write" {
      if file, err := os.Open(os.Args[2]); err == nil {
         file.Close()
         fmt.Printf("%v is already exist\n", os.Args[2])
      } else if file, err := os.Create(os.Args[2]); err != nil {
         fmt.Printf("%v when creating file %v\n", err, os.Args[2])
      } else {
         var oneword string
         defer file.Close()
         fmt.Print("Enter a word to store, '9999' to exit: ")
         fmt.Scanln(&oneword)
         for oneword != "9999" {
            fmt.Fprintln(file, oneword)
            fmt.Print("Enter another word to store, '9999' to exit: ")
            fmt.Scanln(&oneword)
         }
      }
   } else if os.Args[1] == "read" {
      if file, err := os.Open(os.Args[2]); err != nil {
         fmt.Printf("%v when opening file %v\n", err, os.Args[2])
      } else {
         var oneword string
         defer file.Close()
         _, err := fmt.Fscanln(file, &oneword)
         for err == nil {
            fmt.Println("Word from file:", oneword)
            _, err = fmt.Fscanln(file, &oneword)
         }
      }
   } else {
      fmt.Printf("Command is neither 'write' nor 'read'\n")
   }
}
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

6. Paket "log"

Layanan yang diberikan paket ini adalah pembuatan log, misalnya dalam kasus terjadi kesalahan eksekusi. Ada tiga jenis log:

a. Print: yang hanya mencetak pesan beserta waktu kejadian ke log.

```
func Print(a ...interface{})
func Printf(format string, a ...interface{})
func Println(a ...interface{})
```

b. **Panic**: yang sama seperti **Print**, tetapi kemudian melakukan eksekusi **panic()**, yaitu menghentikan eksekusi normal, kemudian menjalankan semua operasi **defer**. Operasi **defer** dijalankan dari operasi **defer** terakhir s.d. **defer** yang pertama atau sampai menemui operasi **recover()**.

```
func Panic(a ...interface{})
func Panicf(format string, a ...interface{})
func Panicln(a ...interface{})
```

c. **Fatal**: melakukan operasi **Print** seperti diatas, tetapi kemudian langsung terminasi program melalui **os.Exit(1).**

```
func Fatal(a ...interface{})
func Fatalf(format string, a ...interface{})
func Fataln(a ...interface{})
```

Contoh penggunaan paket **log** ini dapat dilihat pada **contohos** untuk paket **os**.

7. **Paket "io"**

Paket ini tersedia untuk operasi file bukan teks. Beberapa fungsi yang berguna adalah:

a. Untuk menduplikasi data dari satu (handel) file ke (handel) file lain. File **dest** dapat dibuat menggunakan **os.Create** dan file **src** dapat dibuka menggunakan **os.Open**.

```
func Copy(dst File, src File) (written int64, err error)
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

b. Untuk membaca file. Bandingkan dengan perintah **os.Read** sebelumnya

```
func ReadAtLeast(r File, buf []byte, min int) (n int, err error)
```

c. Untuk menuliskan string s kedalam file.

```
func WriteString(w File, s string) (n int, err error)
```

Contoh

```
package main
/* Ini hanya contoh
  Program yang baik seharusnya memeriksa berbagai kemungkinan error:
  Argumen yang salah,
  Ada/tidaknya file masukan,
  Output akan menimpah file yang sudah ada,
  Gagal copy, ...
*/

import "os"
import "io"

func main() {
  if len(os.Args) == 3 {
    ifile,_ := os.Open(os.Args[1])
    ofile,_ := os.Create(os.Args[2])
    defer ifile.Close()
    defer ofile.Close()
    io.Copy(ofile, ifile)
  }
}
```

```
$ go build
$ ls -l
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 15:35 sumber.dat
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 17:13 tujuan.dat
$ ./contohio sumber.dat duplikat.dat
$ ls -l
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 15:35 sumber.dat
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 17:13 tujuan.dat
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 19:24 duplikat.dat
$
```

8. Paket "net/http"

Paket ini berisi fungsi dan prosedur untuk membuat layanan berbasis web. Berikut ini adalah beberapa fungsi/prosedur yang sering digunakan:

a. Mendengarkan permintaan layanan TCP pada alamat IP dan nomor port tertentu. Permintaan yang datang akan dikirim ke handler. Handler dapat diset nil jika digunakan handler default.

   func ListenAndServe(addr string, handler Handler) error

b. Prosedur handler mencatat handler yang mana yang diaktifkan sesuai dengan permintaan yang diterima.

   func Handle(pattern string, handler Handler)

c. Handler berupa fungsi dapat langsung dideklarasi dan dicatatkan pada prosedur handleFunc. HandleFunc akan mencatat fungsi tersebut perlu diaktifkan jika sesuai dengan permintaan.

   func HandleFunc(pattern string, handler func(ResponseWriter, *Request))

Fakultas Informatika
School of Computing
Telkom University
informatics lab

d. Fungsi FileServer merupakan handler khusus yang sudah tersedia untuk melayani akses ke sistem file melalui web.

   func FileServer(root FileSystem) Handler

e. Tipe Dir adalah implementasi dari tipe FileSystem yang berisi antarmuka yang mendukung operasi Open.

   type FileSystem

   type Dir

   func (d Dir) Open(name string) (File, error)

Contoh penggunaan. Hasilnya dapat dilihat di web melalui URL http://localhost:8080/

```
package main
import "net/http"
func main() {
   panic(http.ListenAndServe(":8080", http.FileServer(http.Dir("."))))
}
```

```
$ go build
$ ls -l
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 15:35 sumber.dat
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 17:13 tujuan.dat
```

```
-rw-rw-r--. 1 jimmyt jimmyt 336 Jul 2 19:24 duplikat.dat
$ ./contohweb
```

9. Paket "strconv"

Paket ini dapat digunakan untuk mengkonversi data string dari/ke tipe yang lain, yang tidak dapat dilakukan dengan cara casting.

a. Konversi dari string representasi nilai boolean menjadi data dengan tipe boolean. Nilai boolean true dapat diperoleh dari string "1", "t", "TRUE", atau "true". Begitu juga nilai boolean false dapat diperoleh dari string "0", "f", "FALSE", atau "false".

```
func ParseBool(str string) (bool, error)
```

b. Konversi dari string representasi nilai bilangan riil menjadi data dengan tipe float64. Bitsize digunakan untuk menentukan ketelitian yang diinginkan, 32 atau 64 (bit).

```
func ParseFloat(s string, bitSize int) (float64, error)
```

c. Konversi dari string representasi nilai bilangan integer menjadi data dengan tipe int.

```
func Atoi(s string) (int, error)
```

d. Konversi data integer ke representasi string, mirip dengan fmt.Sprintf("%v",val)

```
func Itoa(i int) string
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Konversi sebaliknya dari tipe data lain ke tipe string dapat dilakukan menggunakan fmt.Sprint. Lihat paket fmt untuk detilnya. Contoh konversi data:

```
package main
import "fmt"
import "strconv"
func main() {
  var (
    b bool
    f float64
    i int
    s string
  )
  b, _ = strconv.ParseBool("T")
  f, _ = strconv.ParseFloat("2018.0909", 64)
  i, _ = strconv.Atoi("2018")
  s = strconv.Itoa(i)
  fmt.Println(b, f, i, s)
}
```

```
$ go build
$ ./contohconv
true 2018.0909 2018 2018
```

10. Paket "strings"

Operasi terhadap data dengan tipe **string**, selain operasi primitif **len()** dan operator primitif **+**

a. Fungsi untuk memecah string menjadi beberapa potongan berdasarkan marker **sep**. Potongan string dikembalikan sebagai **slice** dari **string**.

```
func Split(s, sep string) []string
```

b. Fungsi untuk mencari potongan string subs didalam string yang diberikan. Fungsi ini mengembalikan lokasi/indeks awal dimana potongan tersebut ditemukan atau -1 jika tidak ada.

```
func Index(s, substring string) int
```

c. Fungsi untuk mencari potongan string subs didalam string yang diberikan. Sama seperti diatas, tetapi fungsi ini mengembalikan nilai boolean **true** jika ada dan **false** jika tidak ada.

```
func Contains(s, substring string) bool
```

d. Mengkonversi setiap huruf dalam string menjadi huruf kecil, atau sebaliknya huruf kapital.

```
func ToLower(s string) string
func ToUpper(s string) string
```

e. Menghapus spasi dibagian depan dan belakang suatu string.

```
func TrimSpace(s string) string
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Contoh

```
package main
import "fmt"
import "strings"

func main() {
   many := strings.Split("Telkom University", "i")
   pos := strings.Index("Telkom University", "kom")
   found := strings.Contains("Telkom University", "Uni")
   lower := strings.ToLower("Gopher")
   upper := strings.ToUpper("Gopher")
   trimmed := strings.TrimSpace(" \t\n Hello, Gophers \n\t\r\n")
   fmt.Println(many, pos, found, lower, upper, trimmed)
}
```

```
$ go build
$ ./contohstr
[Telkom Un vers ty] 3 true gopher GOPHER Hello, Gophers
$
```

11. Paket "archive/zip"

Paket zip adalah satu contoh paket yang sudah tersedia dalam perkakas Go untuk kompresi, pengolahan citra, kripto, dlsb.

a. Fungsi **OpenReader** digunakan untuk membaca isi file zip.

   func OpenReader(name string) (*ReadCloser, error)

b. Tipe **ReadCloser** menyediakan slice **File** yang berisi pasangan-pasangan nama file dan data file tersebut dalam file .zip. Selain itu tipe ini mendukung antarmuka yang mempunyai metoda **Close**().

   type ReadCloser

   func (rc *ReadCloser) Close() error

c. Fungsi **NewWriter** dapat digunakan untuk membuat file kompresi .zip baru atau menimpah yang lama. Fungsi ini mengembalikan handle Writer terhadap .zip file yang dapat digunakan untuk membuat/menyimpan file didalam .zip file.

   func NewWriter(w io.Writer) *Writer

Contoh ala perkakas zip dan unzip. Program yang sesungguhnya perlu menguji hasil operasi terhadap file, apakah terjadi error atau tidak; seperti file tidak ada, direktori tidak ada, dlsb.

Contoh

Fakultas Informatika
School of Computing
Telkom University
informatics lab

```
package main
import "archive/zip"
import "io"
import "os"

func main() {
   switch os.Args[1] {
   case "x":
      xzip, _ := zip.OpenReader(os.Args[2])
      defer xzip.Close()
      for _, file := range xzip.File {
         ifl, _ := file.Open()
         ofl, _ := os.Create(file.Name)
         io.Copy(ofl, ifl)
         ofl.Close()
         ifl.Close()
      }
   case "a":
      f, _ := os.Create(os.Args[2])
      defer f.Close()
      azip := zip.NewWriter(f)
      defer azip.Close()
      for _, file := range os.Args[3:] {
         ifl, _ := os.Open(file)
         ofl, _ := azip.Create(file)
         io.Copy(ofl, ifl)
         ifl.Close()
      }
   }
```

```
}
```

```
$ go build
$ ./contohzip a arsip.zip readme.txt data.dat
$
```

12. Paket "testing"

Paket ini digunakan untuk mendukung proses uji unit, uji tolok-ukur, dan uji keluaran dari program aplikasi Go. Program pengujian dibuat dalam folder yang sama dan dengan nama **package** yang sama pula, tetapi dengan nama file yang mempunyai akhir **_test.go**. Akhiran tersebut digunakan oleh perkakas Go sebagai penanda program pengujian dan proses pengujian dieksekusi dengan memanggil **go test** sebagai pengganti **go build** dan **go run**.

a. Uji unit dilakukan dalam fungsi bernama yang diawali dengan **Test** dan mempunyai parameter ***testing.T**

   ```
   func TestXxx(t *testing.T)
   ```

b. Uji tolok-ukur dilakukan dalam fungsi bernama yang diawali dengan Benchmark dan mempunyai parameter ***testing.B**

   ```
   func BenchmarkXxx(b *testing.B)
   ```

c. Uji keluaran dilakukan dalam fungsi bernama yang diawali dengan Example dan tanpa parameter.

   ```
   func ExampleXXX()
   ```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

Pengujian dapat melaporkan melalui metoda berikut, yang ada dalam antarmuka T dan B

a. **T.Error()** untuk melaporkan hasil uji gagal, tetapi proses tetap dilanjutkan.
b. **T.Fatal()** untuk melaporkan hasil uji gagal dan langsung menghentikan proses pengujian.
c. **T.Log()** untuk melaporkan suatu catatan, tidak terkait kegagalan pengujian.

Contoh

```
package hiworld

import (
   "testing"
   "fmt"
)
func TestHiDefault( t *testing.T ) {
   if GetHi() != "Hello World!" {
      t.Error( "Default was", GetHi(), "should be 'Hello World!'" )
   }
}
func TestHiBad( t *testing.T ) {
   SetHi( "Welcome to the world" )
   if GetHi() == "Welcome to me" {
      t.Error( "Different values on purpose" )
```

```
    }
}
func BenchmarkHello(b *testing.B) {
    for i := 0; i < b.N; i++ {
        fmt.Sprintf("hello")
    }
}
func ExampleOutput() {
    SetHi( "World in my hand" )
    fmt.Println( GetHi() )
    // Output:
    // World in my hand
}
```

```
$ go test
PASS
ok hello2/hiworld 0.002s
$ go test -bench Hello
goos: linux
goarch: amd64
pkg: hello2/hiworld
BenchmarkHello-4 20000000 92.6 ns/op
PASS
ok hello2/hiworld 1.947s
```

B. Perintah Go

Bahasa pemrograman Go agak unik dalam arti perkakas pendukung terintregitas dalam satu perkakas saja, yaitu perintah **go**

1. **go version**: Menampilkan versi dan platform implementasi perkakas Go
2. **go help**: Menampilkan ringkasan informasi penggunaan perkakas Go
3. **go env GOPATH**: Menampilkan lokasi folder untuk menyimpan program Go
    a. Program sumber di **$GOPATH/src**
    b. Program biner/eksekutabel di **$GOPATH/bin**
    c. Pustaka paket program di **$GOPATH/pkg**
4. **go env GOROOT**: Menampilkan lokasi instalasi perkakas Go
5. **go build**: Melakukan proses kompilasi dalam folder *current*. Jika sukses, akan menghasilkan program eksekutabel. (Yaitu file dengan ekstensi *.exe* untuk os Windows)
6. **go clean**: Membersihkan sisa-sisa hasil kompilasi sebelumnya
7. **go run filename.go**: Mengkompilasi dan langsung mengeksekusi program tersebut. Program eksekutabel tidak disimpan, sehingga memberikan ilusi seolah Go adalah sebuah interpreter,

Fakultas Informatika
School of Computing
Telkom University
informatics lab

8. **go fmt**: Me-reformat program sumber yang ada di folder *current*, sehingga memenuhi standar format penulisan program sumber Go

9. **go test**: Melakukan uji (tahap unit), baik uji logik maupun tolok-ukur (*benchmark*). Program
   sumber uji harus mempunyai eksetensi **_test.go**.

10. **go install**: Untuk menginstall program hasil kompilasi pada lokasi yang dituju, biasanya di **$GOPATH/bin/**

11. **go get**: Untuk mengunduh dan menginstal paket dari pihak ketiga atau versi baru/lain perkakas Go. Paket akan diinstal di **$GOPATH/pkg/**, sedangkan perkakas akan disimpan di **$GOPATH/bin/.**

    a. **go get golang.org/dl/go1.12.5**: Untuk mengunduh Go versi 1.12.5

    b. **go get lokasi_paket**: Untuk mengunduh dan menginstal paket dari pihak ketiga. Lihat instalasi paket GUI dibawah untuk contoh.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

C. Instalasi Go Perkakas

1. Unduh perkakas Go dengan versi yang sesuai untuk perangkat keras dan sistem operasi yang digunakan di https://golang.org/dl dan ikuti petunjuk cara instalasinya. Selanjutnya, semua program sumber dan hasil kompilasinya akan disimpan di %GOPATH%

    a. $HOME/go, dengan subfolder src untuk program sumber, bin untuk instal program eksekutabel hasil kompilasi, dan pkg untuk instal paket hasil kompilasi atau paket pihak ketiga.

    b. C:\Users\username\go, dimana username adalah nama user yang sedang login .

2. Untuk menghapus perangkat lunak Go dari sistem, silakan hapus folder %GOROOT% berikut:

    a. Linux dan Mac: folder /usr/local/go

    b. Windows: folder C:\Go

3. Untuk menginstal perkakas versi lain (terbaru) dari Go, melalui command prompt/terminal/shell lakukan perintah Go berikut untuk menginstal versi 1.12.6:

   ```
   go get golang.org/dl/go1.12.6
   go1.12.6 download
   ```

D. Instalasi Paket Pihak ke-3 (Contoh paket GUI)

Kunjungi situs dimana paket tersebut tersedia, umumnya ada di https://github.com/. Baca dan ikuti petunjuk instalasinya. Paket GUI yang berada di https:/github.com/andlabs/ui dapat diinstal dengan menggunakan perintah go get sbb:

```
go get -u github.com/andlabs/ui
```

Contoh program untuk mencoba hasil instal paket GUI tersebut:

```
package main
import "strconv"
import "github.com/andlabs/ui"
const STRETCHED = true
const GREETING = "Colek kiborna, teras pencet tombolna "
func gui() {
   input := ui.NewEntry()
   exbutton := ui.NewButton("Rengse")
   greeting := ui.NewLabel(GREETING)
   vbox := ui.NewVerticalBox()
   vbox.Append(greeting, STRETCHED)
   vbox.Append(input, !STRETCHED)
   vbox.Append(exbutton, STRETCHED)
   window := ui.NewWindow("Sampurasun Dunya", 200, 100, false)
   window.SetMargined(true)
   window.SetChild(vbox)
   input.OnChanged(func(*ui.Entry) {
      greeting.SetText(GREETING + strconv.Itoa(len(input.Text())))
   })
   exbutton.OnClicked(func(*ui.Button) {
      ui.Quit()
   })
   window.OnClosing(func(*ui.Window) bool {
      ui.Quit()
      return true
   })
   window.Show()
}

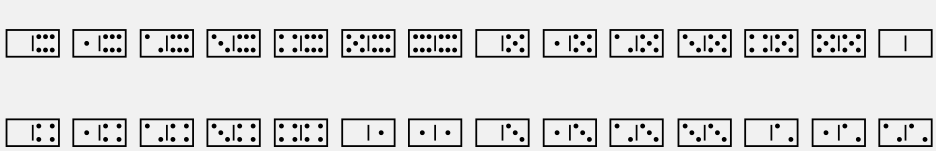
func main() {
   err := ui.Main(gui)
   if err != nil {
      panic(err)
   }
}
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

E. Contoh Program

(**Permainan Solitaire Gapleh**) Kartu domino yang biasa digunakan bermain gapleh berisi 28 kartu seperti yang ditampilkan dalam gambar di bawah ini. Bagian depan kartu terbagi oleh sebuah garis menjadi dua sisi. Tiap sisi dapat memiliki titik dari 0 s.d. 6. Jika kedua sisi mempunyai jumlah titik yang sama, maka disebut kartu balak. Ada 7 kartu

balak; 0-0, 1-1, s.d. 6-6. Nilai total kartu adalah jumlah titik di kedua sisi, sehingga nilainya bervariasi dari 0 s.d. 12.



Kartu domino dikocok, kemudian pemain dibagi 7 kartu dari tumpukan kartu. Satu kartu dari tumpukan dibuka untuk memulai permainan. Pemain harus memilih salah satu kartunya yang mempunyai gambar yang sama dengan salah satu ujung rantai kartu dan dipasangkan diujung rantai kartu tersebut.

Nilai yang diperoleh pemain adalah berapa banyak kartu yang dia bisa tempatkan dalam rantai kartu, 0..7. Maksimum nilai per ronde adalah 7.

Program Anda harus dapat:

1. Mengocok kartu
2. Mencatat nilai dan jumlah ronde yang dimainkan
3. Pada setiap ronde, membagikan 7 kartu dan membuka satu kartu dari tumpukan untuk memulai permainan.
4. Memeriksa kartu pilihan pemain (-7..-1, 0, 1, ..7, dan 9) yang -1 .. -7 untuk meletakkan kartu diujung kiri rantai, 1..7 untuk ujung kanan rantai, 0 untuk selesai ronde, 9 untuk selesai permainan

Fakultas Informatika
School of Computing
Telkom University
Informatics Lab

Contoh masukan dan keluaran (**<u>teks bergaris bawah</u>** adalah input/masukan):

Welcome to the game of Gaple	
Your score is 0/0	At the beginning scores=0, games=0
Dealing ...	Shuffle the tiles
Your tiles: (3,6) (1,4) (5,5) (2,5) (5,6) (2,3) (3,4)	
Starting train is (0,4)	
Decision? **<u>+7</u>**	7th/last tile=(3,4) put on right end
Train is (0,3)	The train (0,4) (4,3), or just (0,3)
Decision? **<u>+1</u>**	1st tile=(3,6)
Train is (0,6)	The train (0,4) (4,3) (3,6), or just (0,6)
Decision? **<u>+5</u>**	5th tile=(5,6)
Train is (0,5)	The train (0,4) (4,3) (3,6)(6,5), or just (0,5)
Decision? **<u>+3</u>**	3rd tile=(5,5)
Train is (0,5)	The train (0,4) (4,3) (3,6) (6,5) (5,5)
Decision? **<u>+4</u>**	4th tile=(2,5)
Train is (0,2)	The train (0,4) (4,3) (3,6) (6,5) (5,5) (5,2)
Decision? **<u>+6</u>**	6th tile=(2,3)
Train is (0,3)	The train (0,4)(4,3)(3,6)(6,5)(5,5)(5,2)(2,3)
Decision? **<u>0</u>**	No more legal move
You earn 6 points	



Kontak Kami :

@dky2921g

@informaticslab

@informaticslab_telu

informaticslab@telkomuniversity.ac.id

informatics.labs.telkomuniversity.ac.id